

SECURITY AUDIT TOOL

Benutzerhandbuch

Firewall-Validation Portable

Auswertung mit `index.html` und Pflege des Regelkatalogs mit dem Regel-Editor `rules-editor.html` für OPNsense- und pfSense-Konfigurationen.

KATALOG

1.2.0 (34 Regeln)

STAND

9. Oktober 2026

BETRIEB

lokal im Browser, offline

LIZENZ

GPL-3.0-or-later mit Urhebernennung

AUTOR

© 2026 Christian Jelitte

Inhaltsverzeichnis

1 Überblick

- 1.1 Was das Werkzeug tut
- 1.2 Die Bausteine und wie sie zusammenspielen
- 1.3 Datenschutz und Arbeitsweise
- 1.4 Was das Werkzeug nicht ersetzt
- 1.5 Wie du dieses Handbuch benutzt

2 Schnellstart in fünf Schritten

TEIL A – AUSWERTUNG MIT INDEX.HTML

3 Voraussetzungen und Vorbereitung

- 3.1 Browser und Ordner
- 3.2 Konfiguration exportieren

4 Konfiguration laden

- 4.1 Was beim Laden geschieht

5 Der Einzelbericht

- 5.1 Kopf und Scorecard
- 5.2 Die Befundübersicht
- 5.3 Detailbefunde und Filter
- 5.4 Kartenarten im Vergleich

6 Score, Status und Schweregrade verstehen

- 6.1 Die sechs Status
- 6.2 Die Schweregrade
- 6.3 So entsteht der Score
- 6.4 Befunde zur Regel-Reihenfolge

7 Das Fleet-Dashboard

- 7.1 Kennzahlen und Verteilung
- 7.2 Systemische Befunde
- 7.3 Die Tabelle „Geräte & Standorte“
- 7.4 Auf einen Befund filtern
- 7.5 Ein Gerät öffnen und zurückkehren

8 Drucken, PDF und mobile Ansicht

- 8.1 Bericht drucken oder als PDF sichern
- 8.2 Ansicht auf dem Smartphone

9 Fehlermeldungen und Fehlerbehebung (index.html)

- 9.1 Meldungen im Überblick

10 Grenzen der Auswertung

TEIL B – REGEL-EDITOR (RULES-EDITOR.HTML)

11 Konzept und Arbeitsablauf

- 11.1 Wofür der Editor da ist
- 11.2 Der Arbeitsablauf

12 Starten und Katalog laden

- 12.1 Automatisches Laden

- 12.2 Eine andere Katalogdatei öffnen

13 Die Oberfläche im Überblick

- 13.1 Änderungen im Blick behalten

14 Regeln ansehen und bearbeiten

- 14.1 Die Regelliste
- 14.2 Das Regelformular
- 14.3 Schweregrad ändern und Regeln deaktivieren
- 14.4 Parameter

15 Neue Regeln anlegen

- 15.1 Regeln löschen

16 Katalog-Einstellungen

17 Prüfungen bearbeiten und ausprobieren

- 17.1 Vorlagen
- 17.2 „Aufbau prüfen“ – was es findet und was nicht
- 17.3 Der Probelauf
- 17.4 Die Sprachreferenz

18 Die Regelsprache

- 18.1 Aufbau und Ablauf
- 18.2 Werte: Angaben aus der Config lesen
- 18.3 XML-Pfade und Plattformen
- 18.4 Faktenquellen
- 18.5 Bedingungen
- 18.6 Ergebnis-Fälle
- 18.7 Textvorlagen für Befunde
- 18.8 Reihenfolge-Prüfungen (typ „reihenfolge“)
- 18.9 Bausteine
- 18.10 Parameter (@param)
- 18.11 Plattform-abhängige Prüfungen

19 Praxisbeispiele

- 19.1 Beispiel 1: Schweregrad einer Regel ändern
- 19.2 Beispiel 2: Eine Regel ausschalten, die bei dir nicht zutrifft
- 19.3 Beispiel 3: Ein zweiter Internetanschluss
- 19.4 Beispiel 4: Eine Portnummer als sensibel ergänzen
- 19.5 Beispiel 5: WAN-Freigaben ohne Logging finden
- 19.6 Beispiel 6: Eine Einstellung auf einen bestimmten Wert prüfen
- 19.7 Beispiel 7: Eine eigene Portliste als Parameter
- 19.8 Beispiel 8: Einen anderen Ablageort einer Einstellung nachtragen

19.9 Beispiel 9: Verdeckte Block-Regeln nur auf dem WAN melden

20 Als neue Version speichern und einsetzen

20.1 Der Speicher-Dialog

20.2 Was in die Datei geschrieben wird

20.3 Die neue Datei einsetzen

21 Schritt für Schritt: Prüfen, ob die Block-All-Regel am Ende steht

21.1 Was die Regel prüfen soll

21.2 Schritt 1 und 2: Editor öffnen und Regel beschreiben

21.3 Schritt 3 und 4: Prüfung einsetzen und Aufbau prüfen

21.4 Schritt 5: Probelauf mit zwei Test-Configs

21.5 Schritt 6: Regel anlegen

21.6 Schritt 7 und 8: Als neue Version speichern und einsetzen

21.7 Schritt 9: Ergebnis im Bericht kontrollieren

21.8 Schritt 10: Mit der eigenen Firewall prüfen und beheben

21.9 Varianten und Grenzen

22 Fehlermeldungen und Fehlerbehebung (Editor)

22.1 Meldungen des Editors

22.2 Häufige Aufbaufehler

22.3 Wenn die Prüfung nicht tut, was sie soll

23 Grenzen des Editors

TEIL C – PRAXIS

24 Typische Abläufe

24.1 Eine Firewall prüfen und die Ergebnisse abarbeiten

24.2 Viele Standorte vergleichen

24.3 Den Katalog anpassen und einführen

24.4 Den Katalog im Team verwalten

25 Der Selbsttest der Engine

25.1 Wann ausführen?

ANHANG

Anhang A Regelübersicht des Katalogs 1.2.0

A.1 Regeln mit eigenem Schweregrad im Fund

Anhang B Sprachreferenz und Spickzettel

B.1 Faktenquellen und ihre Felder

B.2 Allgemeine Felder

B.3 Operatoren

B.4 Weitere Bedingungen

B.5 Spickzettel

Anhang C Dateien, Ordner und Beispiel-Configs

C.1 Dateien im Paket

C.2 Die Beispiel-Configs

Anhang D Glossar

Anhang E Versionsverlauf des Katalogs

1 Überblick

1.1 Was das Werkzeug tut

Firewall-Validation Portable prüft die Konfiguration einer **OPNsense**- oder **pfSense**-Firewall auf typische Fehlkonfigurationen. Du exportierst die `config.xml` deiner Firewall, lädst sie in den Browser, und das Werkzeug

- erkennt automatisch, ob es sich um OPNsense oder pfSense handelt,
- prüft die Konfiguration gegen einen **Katalog von 34 Regeln in 6 Kategorien** (Management-Ebene, Firewall-Regeln, NAT / Port-Forwarding, Dienst-Exposition, VPN, System-Härtung),
- berechnet daraus einen **Posture-Score von 0 bis 100** und eine **Note von A bis F**,
- erstellt einen Bericht mit Befund, Begründung, empfohlener Maßnahme und Referenz zu jeder Auffälligkeit,
- zeigt beim Laden **mehrerer** Konfigurationen ein **Fleet-Dashboard** für den Vergleich vieler Standorte.

Das Werkzeug besteht aus zwei Programmen, die dieses Handbuch beschreibt:

Programm	Aufgabe	Kapitel
<code>index.html</code>	Auswertung. Konfiguration laden, Bericht und Dashboard lesen.	3 bis 10
<code>rules-editor.html</code>	Regel-Editor. Den Regelkatalog pflegen: Regeln anpassen, deaktivieren, neu schreiben, Einstellungen ändern, neue Version speichern.	11 bis 23

1.2 Die Bausteine und wie sie zusammenspielen

Vier Dateien gehören zusammen und müssen **im selben Ordner** liegen. Jede hat genau eine Aufgabe:

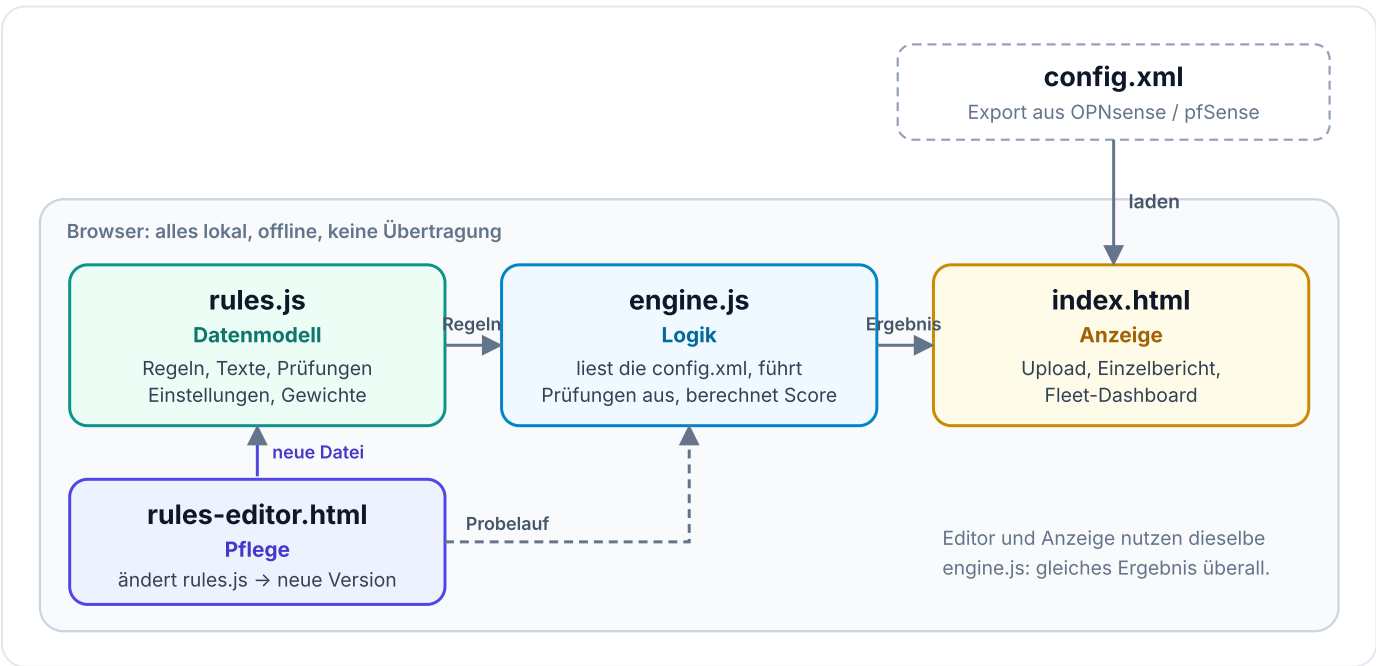


Abbildung 1: Die Dateien des Werkzeugs und ihre Aufgaben. Alles läuft lokal im Browser.

Datei	Rolle	Inhalt
<code>index.html</code>	Anzeige	Upload, Einzelbericht, Fleet-Dashboard. Enthält keine Prüflogik.
<code>engine.js</code>	Logik	Liest die <code>config.xml</code> , übersetzt sie für OPNsense und pfSense in eine einheitliche Sicht, führt die Prüfungen aus und berechnet den Score. Enthält keine Anzeige.
<code>rules.js</code>	Datenmodell	Der Regelkatalog: für jede Regel Titel, Schweregrad, Begründung, Maßnahme, Parameter und die Prüfung selbst ; dazu Einstellungen wie Gewichte und WAN-Interfaces.
<code>rules-editor.html</code>	Pflege	Bearbeitet <code>rules.js</code> und erzeugt eine neue Version davon.

Zusätzlich liegen im Paket: `tests/` (Selbsttest und Beispiel-Configs), `README.md` (Kurzbeschreibung), `TESTPLAN.md` (Testplan für echte Firewalls) und dieses Handbuch.

Zwei Arten von „Regel“ Dieses Handbuch unterscheidet **Regeln des Katalogs** (Prüfvorschriften, z. B. `FW-ANY-ANY`) und **Firewall-Regeln** (die Pass-/Block-Regeln in der Firewall selbst). Wo es nötig ist, steht „Firewall-Regel“ ausdrücklich da; sonst ist die Regel des Katalogs gemeint.

Merksatz Was die Auswertung *anzeigt*, entscheidet `index.html`. Was *geprüft wird*, steht in `rules.js`. Wie die Prüfungen *technisch ausgeführt* werden, steckt in `engine.js`. Für den Alltag musst du nur `index.html` öffnen; den Editor brauchst du erst, wenn du den Katalog ändern willst.

1.3 Datenschutz und Arbeitsweise

- **Alles passiert lokal im Browser.** Die Konfiguration wird nicht hochgeladen, es gibt weder Server noch Konto. Das Werkzeug funktioniert offline und direkt aus dem Dateisystem (`file://`).
- Die `config.xml` enthält **Passwort-Hashes, VPN-Schlüssel und gegebenenfalls PSKs**. Behandle sie wie ein Geheimnis: Audits möglichst auf einem Rechner ohne Internetzugang oder mit gesperrtem Netz durchführen, die Datei nach dem Audit sicher löschen oder verschlüsselt aufbewahren.
- Der **Bericht** enthält keine Passwörter, wohl aber Hostnamen, Regelbeschreibungen, IP-Adressen und Portnummern deiner Firewall. Gib Berichte und Screenshots nur weiter, wenn das vertretbar ist.
- `engine.js` und `rules.js` werden vom Browser als **Programmcode** ausgeführt. Verwende nur Dateien aus vertrauenswürdiger Quelle, gerade bei Katalogen, die dir jemand schickt.

1.4 Was das Werkzeug nicht ersetzt

Das Audit liest ausschließlich die exportierte `config.xml`. Es sieht **nicht**, was zur Laufzeit geschieht: den tatsächlichen Firmware-Stand, geladene Regeln, automatisch erzeugte Regeln oder Plugin-Zustände. Regeln mit dem Status **Manuell** verlangen ausdrücklich eine Prüfung auf der Firewall selbst. Die vollständige Liste der Grenzen steht in Kapitel 10. Ein automatisiertes Audit ersetzt keine manuelle Sicherheitsprüfung.

1.5 Wie du dieses Handbuch benutzt

Du möchtest ...

Lies ...

schnell eine Firewall prüfen	Kapitel 2 (Schnellstart), danach 5 (Einzelbericht)
viele Firewalls vergleichen	Kapitel 7 (Fleet-Dashboard)
Score und Befunde verstehen	Kapitel 6
Schweregrad oder Regeln anpassen	Kapitel 14 und 16
eine eigene Prüfung schreiben	Kapitel 21 (vollständiges Beispiel Schritt für Schritt), 18 (Regelsprache) und 19 (Praxisbeispiele)
etwas Unerwartetes deuten	Kapitel 9 und 22 (Fehlermeldungen)

Zu den Screenshots Alle Abbildungen entstanden mit den mitgelieferten **Test-Configs** aus `tests/configs/`, nicht mit echten Firewalls. Du kannst jede Abbildung mit denselben Dateien nachstellen. Die eingeblendeten Zahlen im Kreis gehören zu den nummerierten Erläuterungen unter dem Bild. Die Oberfläche von `index.html` schreibt Umlaute als *ae, oe, ue* (z. B. „Datei auswaehlen“); das Handbuch zitiert Beschriftungen wörtlich.

2 Schnellstart in fünf Schritten

Dauer: etwa fünf Minuten. Du brauchst einen aktuellen Browser (Firefox, Chrome, Edge oder Safari) und Zugriff auf die Weboberfläche deiner Firewall.

- 1 Ordner bereitstellen** **Arbeitsrechner**
 Entpacke das Paket. Die Dateien `index.html`, `engine.js`, `rules.js` und `rules-editor.html` müssen im **selben Ordner** liegen. Öffne `index.html` per Doppelklick im Browser. Prüfe in der Kopfzeile: *Katalog: OPNsense Security Baseline v1.2.0 (34 Regeln)* bedeutet, dass Katalog und Engine geladen sind.
- 2 Konfiguration exportieren, OPNsense** **OPNsense-Weboberfläche**
System → Konfiguration → Sicherungen (englisch: *System → Configuration → Backups*). Im Bereich *Download* die Option **Verschlüsseln nicht** ankreuzen, dann *Konfiguration herunterladen*. Ergebnis: eine XML-Datei.
- 2 Konfiguration exportieren, pfSense** **pfSense-Weboberfläche**
Diagnostics → Backup & Restore. *Backup area* auf *All*, die Option *Skip RRD data* ankreuzen (macht die Datei deutlich kleiner, der Inhalt für das Audit bleibt gleich), *Download configuration as XML*.
- 3 Datei laden** **Arbeitsrechner, Browser**
 Ziehe die XML-Datei auf den Upload-Bereich oder klicke **Datei auswaehlen**. Der Dateiname muss auf `.xml` enden (kleingeschrieben).

4

Ergebnis lesen

Oben stehen Note und Score, darunter die Übersichtstabelle der Befunde, sortiert nach Dringlichkeit. Die Karten weiter unten erklären jeden Befund und nennen die Maßnahme.

5

Ausprobieren ohne Firewall [Arbeitsrechner](#)

Zum Kennenlernen liegen im Ordner `tests/configs/` Beispiel-Konfigurationen. Lade z. B. `opn-schwach.xml` (viele Befunde) und danach `opn-gehaertet.xml` (keine Befunde). Lädst du **mehrere Dateien gleichzeitig**, erscheint das Dashboard (Kapitel 7).

Teil A – Auswertung mit index.html

Kapitel 3 bis 10

3 Voraussetzungen und Vorbereitung

3.1 Browser und Ordner

- **Browser:** aktuelle Versionen von Firefox, Chrome/Chromium, Edge oder Safari mit aktiviertem JavaScript. Es ist keine Installation, kein Webserver und keine Internetverbindung nötig.
- **Ordner:** `index.html`, `engine.js` und `rules.js` liegen zusammen. Fehlt eine der beiden Dateien, zeigt die Seite beim Öffnen eine rote Meldung (Kapitel 9).
- **Betrieb:** Doppelklick auf `index.html` genügt. Du kannst den Ordner ebenso auf einen beliebigen statischen Webserver legen.

3.2 Konfiguration exportieren

System	Menüpfad und wichtige Einstellungen
OPNsense	<i>System → Konfiguration → Sicherungen, Konfiguration herunterladen. Ohne Verschlüsselung exportieren.</i>
pfSense	<i>Diagnostics → Backup & Restore, Bereich Backup Configuration, Backup area: All, Skip RRD data ankreuzen, Download configuration as XML.</i>

Verschlüsselte Sicherungen Eine verschlüsselte Sicherungsdatei kann das Werkzeug nicht lesen. Exportiere ohne Passwort und behandle die Datei anschließend sorgfältig (Kapitel 1.3).

Mehrere Firewalls: Für jede Firewall eine eigene Datei. Vergib aussagekräftige Dateinamen (z. B. `standort-berlin.xml`). In der Übersicht erscheint pro Gerät der Hostname aus der Konfiguration; fehlt er, wird der Dateiname verwendet.

4 Konfiguration laden

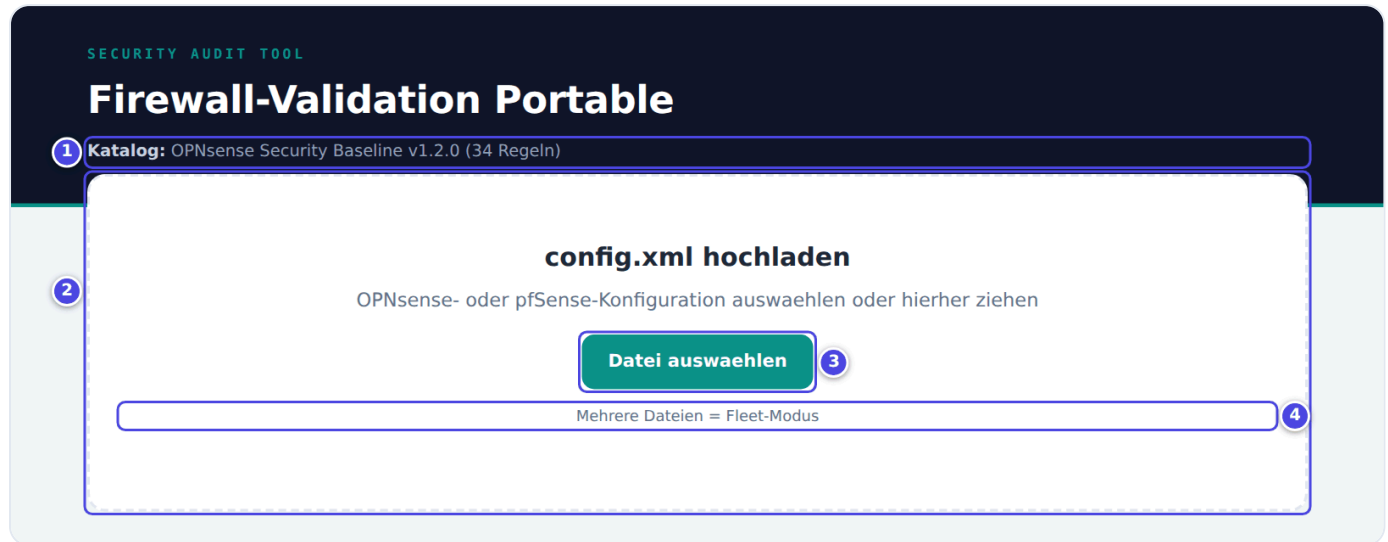


Abbildung 2: Startansicht von index.html vor dem Laden einer Datei.

- 1 **Katalog-Zeile.** Zeigt Name, Version und Regelanzahl des geladenen Katalogs. Steht hier eine Fehlermeldung oder „wird geladen ...“, fehlt `rules.js`.
- 2 **Upload-Bereich.** Eine oder mehrere Dateien hierher ziehen (Drag & Drop). Beim Darüberziehen färben sich Rahmen und Hintergrund grün-türkis.
- 3 **Datei auswählen** öffnet den Dateiauswahldialog Deines Systems. Mehrfachauswahl ist möglich.
- 4 **Fleet-Modus.** Mehrere Dateien auf einmal ergeben statt eines Einzelberichts das Dashboard (Kapitel 7).

4.1 Was beim Laden geschieht

- Nur Dateien, deren Name auf `.xml` in **Kleinbuchstaben** endet, werden berücksichtigt. `config.XML` oder `config.txt` werden ignoriert. Ist keine passende Datei dabei, erscheint *Keine XML-Dateien ausgewählt*.
- Bei **einer** gültigen Datei entsteht der Einzelbericht, bei **mehreren** das Dashboard.
- Schlägt bei mehreren Dateien eine einzelne fehl, erscheint oben eine rote Meldung mit dem Dateinamen. Die übrigen Dateien werden trotzdem ausgewertet. Bleibt danach nur **eine** gültige Datei übrig, siehst du deren Einzelbericht plus die Meldung.
- Die Auswertung dauert nur einen Moment. Es findet keine Übertragung statt.
- Mit **Neue Datei laden** (unten im Bericht) bzw. **Neue Datei(en) laden** (im Dashboard) kehrst du zur Startansicht zurück. Ein Neuladen der Seite (`F5`) tut dasselbe.

Beschädigte oder abgeschnittene Dateien Ist die XML-Datei unvollständig (z. B. durch einen abgebrochenen Download) oder enthält sie einen Syntaxfehler, meldet die Oberfläche in Version 1.2.0 **nicht immer einen Fehler**. Es kann stattdessen ein Bericht über den lesbaren Teil der Datei entstehen. Ein solcher Bericht wirkt plausibel, ist aber unvollständig und meist zu gut. Achte auf Warnzeichen: der Hostname „Firewall“ (das ist der Vorgabetext, wenn die Datei keinen Hostnamen liefert), auffallend viele Regeln mit `N/A` oder ein deutlich besserer Score als erwartet. Im Zweifel den Export wiederholen und die Dateigröße mit der Firewall abgleichen.

5 Der Einzelbericht

5.1 Kopf und Scorecard

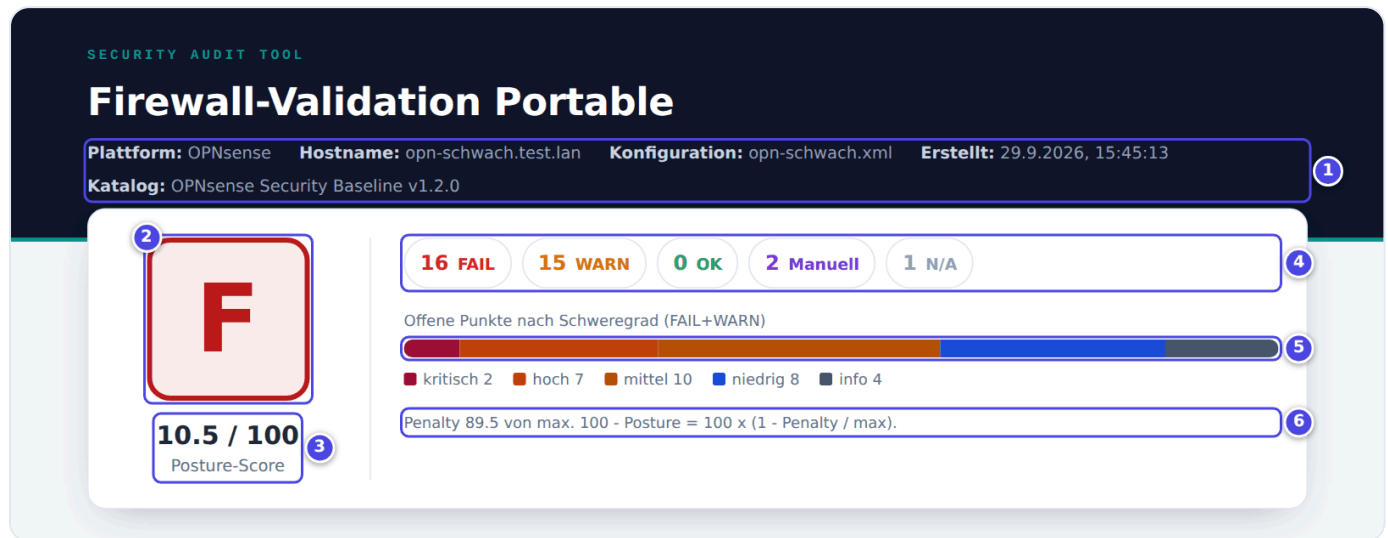


Abbildung 3: Kopfbereich und Scorecard des Einzelberichts (Beispiel: opn-schwach.xml).

- 1 Metadaten.** Plattform (automatisch am Wurzelement der Datei erkannt: OPNsense oder pfSense), Hostname (Hostname und Domäne aus der Konfiguration), Konfiguration (Dateiname), Erstellt (Zeitpunkt der Auswertung) und Katalog (Name und Version).
- 2 Note.** Der Buchstabe A bis F fasst den Score zusammen (Tabelle in Kapitel 6.3). Die Farbe wechselt von grün (A) bis rot (F).
- 3 Posture-Score** von 0 (sehr schlecht) bis 100 (keine bewertbaren Schwächen).
- 4 Statuszähler:** Anzahl der Regeln je Ergebnis: FAIL, WARN, OK, Manuell, N/A. (Was die Status bedeuten: Kapitel 6.1.)
- 5 Schweregradbalken** für die *offenen Punkte*, also FAIL und WARN zusammen, aufgeteilt nach kritisch, hoch, mittel, niedrig und info. So erkennst du auf einen Blick, wo das Gewicht liegt.
- 6 Penalty-Zeile.** Zeigt die Strafpunkte und die höchstmögliche Strafpunktzahl, aus denen der Score berechnet wurde (Kapitel 6.3).

5.2 Die Befundübersicht

UEBERSICHT

Befunde im Ueberblick

STATUS	SCHWERE	ID	TITEL	KATEGORIE
FAIL	kritisch	MGMT-WEBGUI-WAN	WebGUI vom WAN-Interface erreichbar	Management-Ebene
FAIL	kritisch	NAT-MGMT-EXPOSED	Port-Weiterleitung exponiert sensiblen/Management-Port vom WAN	NAT / Port-Forwarding
FAIL	hoch	FW-ANY-ANY	Pass-Regel mit Quelle ANY und Ziel ANY	Firewall-Regeln
FAIL	hoch	FW-WAN-EXPOSURE	WAN-Pass-Regel exponiert internen Dienst oder die Firewall selbst	Firewall-Regeln
FAIL	hoch	MGMT-SSH-WAN	SSH-Dienst auf WAN-Interface gebunden oder per WAN-Regel erreichbar	Management-Ebene
FAIL	hoch	MGMT-WEBGUI-HTTP	WebGUI ohne HTTPS (Klartext)	Management-Ebene
FAIL	hoch	NAT-UPNP	UPnP / NAT-PMP aktiviert	NAT / Port-Forwarding
FAIL	hoch	SVC-SNMP-PUBLIC	SNMP mit Standard-Community 'public'	Dienst-Exposition

Abbildung 4: Tabelle „Befunde im Ueberblick“ (Ausschnitt).

- 1

STATUS: FAIL, WARN, OK, MANUELL, N/A oder FEHLER.
- 2

SCHWERE: der *wirksame* Schweregrad. Er kann vom Katalogwert abweichen (Kapitel 6.2).
- 3

ID: die eindeutige Kennung der Regel, z. B. NAT-MGMT-EXPOSED .
- 4

TITEL: Kurzbeschreibung des Befunds.
- 5

KATEGORIE: Management-Ebene, Firewall-Regeln, NAT / Port-Forwarding, Dienst-Exposition, VPN oder System-Haertung.

Sortierung: zuerst FAIL, dann WARN, dann Fehler, Manuell, OK und N/A. Innerhalb eines Status stehen die schwerwiegendsten zuerst (kritisch vor hoch vor mittel ...), bei gleichem Schweregrad alphabetisch nach ID.

Welche Zeilen sichtbar sind, bestimmt der Filter im Bereich „Detailbefunde“ (nächster Abschnitt). Beim Öffnen ist **Offen (FAIL+WARN)** aktiv, deshalb zeigt die Tabelle zunächst nur die offenen Punkte.

5.3 Detailbefunde und Filter

1

Offen (FAIL+WARN)

Nur FAIL

Nur WARN

Bestanden

Manuell

N/A

Alle

2

3

FAIL

kritisch

MGMT-WEBGUI-WAN

WebGUI vom WAN-Interface erreichbar

4

Management-Ebene

5

- WebGUI an KEIN Interface gebunden -> lauscht auf allen inkl. WAN.
- WAN-Regel #1 erlaubt GUI-Port 80 (Quelle: ANY) - 'GUI von aussen'.

6

Begründung: Ist das Web-Interface direkt aus dem Internet erreichbar, wird das gesamte Geraet zur exponierten Angriffsflaeche (Brute-Force, CVE-Ausnutzung). Management gehoert ins LAN/Management-VLAN oder hinter ein VPN.

7

MASSNAHME
WebGUI nur an interne Interfaces binden (System > Settings > Administration > Listen Interfaces) und keine WAN-Pass-Regel auf den GUI-Port anlegen. Zugriff ueber VPN statt direkter WAN-Freigabe.

8

Referenzen: OPNsense Hardening Guide: Secure access

Abbildung 5: Filterleiste und erste Detailkarte.

Benutzerhandbuch · Firewall-Validation Portable · © 2026 Christian Jelitte · GPL-3.0-or-later

Seite 11 von 86

- 1 **Filterleiste.** Sie wirkt gleichzeitig auf die Übersichtstabelle darüber und auf alle Karten darunter.
- 2 **Aktiver Filter** (dunkel hinterlegt). Beim Öffnen: **Offen (FAIL+WARN)**.
- 3 **Kartenkopf:** Status, Schweregrad, Regel-ID und Titel. Der farbige Streifen am linken Kartenrand entspricht dem Schweregrad.
- 4 **Kategorie** der Regel.
- 5 **Befunde.** Die konkreten Fundstellen in deiner Konfiguration, z. B. Regelnummern, Ports oder Benutzernamen.
- 6 **Begründung:** warum das ein Risiko ist.
- 7 **Massnahme** (grün hinterlegt): was du tun kannst, häufig mit Menüpfad.
- 8 **Referenzen** (falls vorhanden): weiterführende Quellen.

Schaltfläche	Zeigt Regeln mit Status
Offen (FAIL+WARN)	FAIL, WARN und FEHLER (Standard beim Öffnen)
Nur FAIL	FAIL
Nur WARN	WARN
Bestanden	OK
Manuell	MANUELL und FEHLER
N/A	N/A (nicht zutreffend)
Alle	alle Regeln

Tipp Wähle **Alle**, wenn du wissen willst, was *bestanden* wurde oder warum eine Regel *nicht zutrifft*. Bei Regeln mit Status OK, N/A oder MANUELL steht in der Befundzeile eine kurze Notiz, z. B. „SSH nicht aktiviert.“

5.4 Kartenarten im Vergleich

Der Kopf einer Karte zeigt sofort, was die Regel ergeben hat. Die vier folgenden Ausschnitte zeigen, wie sich die Ergebnisse unterscheiden (jeweils nur der obere Kartenteil).

WARN

mittel

SVC-DNS-OPENRESOLVER

Unbound: ACL-Default 'allow' (potenzieller Open Resolver)

Dienst-Exposition

- Unbound lauscht auf 'lan,opt2,wan' - Open-Resolver-Risiko.

WARN: bedingtes Risiko, mit konkretem Befund.

OK

kritisch

MGMT-WEBGUI-WAN

WebGUI vom WAN-Interface erreichbar

Management-Ebene

GUI-Port 443 nicht vom WAN freigegeben.

OK: Kontrolle erfüllt, mit Notiz.

N/A

hoch

MGMT - SSH - WAN

SSH-Dienst auf WAN-Interface gebunden oder per WAN-Regel erreichbar

Management-Ebene

SSH nicht aktiviert.

N/A: Regel trifft nicht zu (hier: SSH ist nicht aktiviert).

MANUELL

info

SYS - BACKUP - SECRETS

config.xml enthaelt Secrets (Hinweis)

System-Haertung

- config.xml enthaelt Passwort-Hashes, Schluessel und ggf. PSKs.

Backups verschluesseln und Zugriff begrenzen.

MANUELL: kann nur auf der Firewall geprüft werden.

Abbildung 6: Karten für WARN, OK, N/A und MANUELL (Beispiel: pf-reihenfolge.xml).

6 Score, Status und Schweregrade verstehen

6.1 Die sechs Status

Status	Bedeutung	Was du tun solltest
FAIL	Schwachstelle vorhanden	Befund lesen, Maßnahme umsetzen oder bewusst begründen. Fließt mit Faktor 1,0 in den Score.
WARN	Bedingtes Risiko, Prüfung empfohlen	Prüfen, ob die Konfiguration beabsichtigt ist. Fließt mit Faktor 0,5 in den Score.
OK	Kontrolle erfüllt	Nichts. Zählt mit Faktor 0 (keine Strafpunkte), aber das Gewicht der Regel steht im Nenner.
N/A	Funktion nicht vorhanden oder nicht zutreffend	Nichts. Die Regel bleibt beim Score außen vor. Ein Beispiel: SSH ist aus, also sind die SSH-Prüfungen nicht anwendbar. Prüfe nur, ob „nicht vorhanden“ tatsächlich stimmt und die Engine nicht an der falschen Stelle gesucht hat (siehe Testplan).
MANUELL	Nicht aus der Config bewertbar	Auf der Firewall selbst prüfen (z. B. Firmware-Stand). Beim Score außen vor.
FEHLER	Die Prüfung ist abgestürzt	Programmfehler. Befundtext notieren, Regel im Editor prüfen (Kapitel 22). Beim Score außen vor.

6.2 Die Schweregrade

Schweregrad	Gewicht	Bedeutung
kritisch	10	Unmittelbare Gefahr, z. B. Verwaltungsoberfläche oder sensibler Dienst aus dem Internet erreichbar.
hoch	6	Erhebliches Risiko, zeitnah beheben.
mittel	3	Relevantes Risiko oder Härtungslücke.

niedrig	1	Kleinere Härtungslücke oder Hygiene.
info	0	Hinweis. Erscheint im Bericht, ändert aber den Score nicht.

Wirksamer Schweregrad. Einzelne Regeln passen den Schweregrad an den konkreten Fund an. Beispiele: Eine any→any -Regel auf dem WAN-Interface gilt als *hoch* statt *mittel*; eine Weiterleitung sensibler Ports gilt als *kritisch*, wenn die Quelle beliebig ist (sonst *hoch*). In Tabelle und Karte steht immer der **wirksame** Wert. Die Gewichte kannst du im Editor ändern (Kapitel 16).

6.3 So entsteht der Score

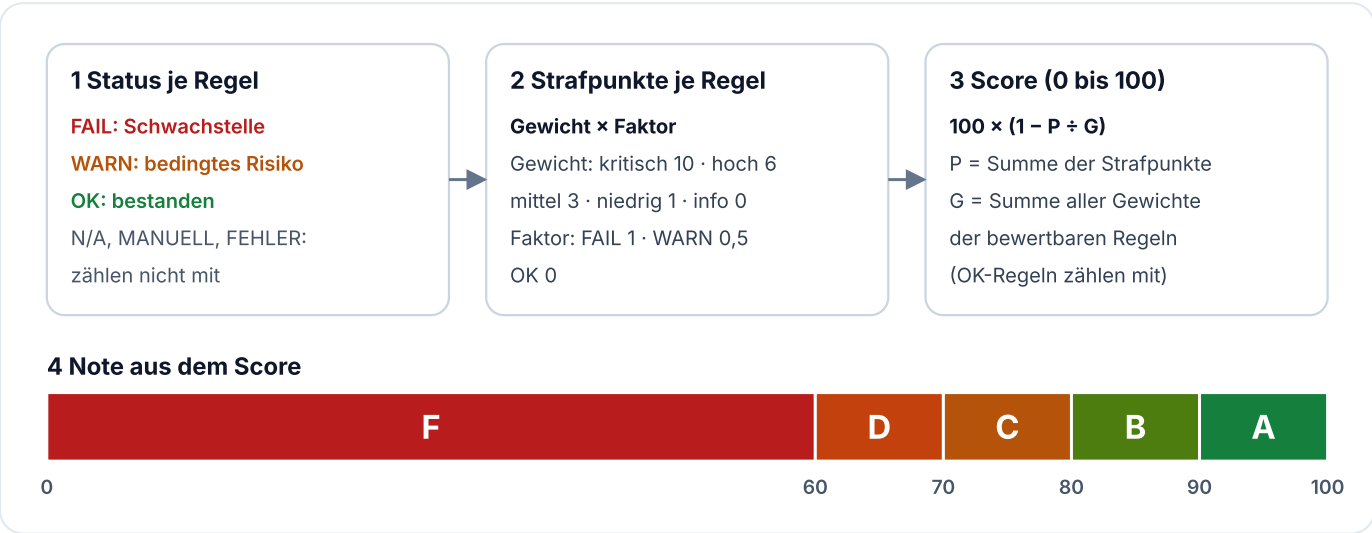


Abbildung 7: Berechnung von Score und Note.

In Worten: Jede bewertbare Regel hat ein **Gewicht** (nach Schweregrad). Ein FAIL kostet das volle Gewicht, ein WARN das halbe, ein OK nichts. Die Summe dieser Strafpunkte wird ins Verhältnis zur höchstmöglichen Strafpunktzahl gesetzt, also der Summe der Gewichte *aller bewertbaren* Regeln. N/A, MANUELL und FEHLER zählen nicht mit. Damit wird nur nach dem bewertet, was tatsächlich zutrifft.

Note	Score von ... bis
A	90 bis 100
B	80 bis unter 90
C	70 bis unter 80
D	60 bis unter 70
F	unter 60

Ein durchgerechnetes Beispiel

Beispiel: die Test-Config `opn-minimal.xml` (fast leere OPNsense-Konfiguration). Wie in der Oberfläche sind Dezimalzahlen mit Punkt geschrieben.

Regel	Status	Schweregrad	Gewicht	Faktor	Strafpunkte
-------	--------	-------------	---------	--------	-------------

MGMT - WEBGUI - WAN	FAIL	kritisch	10	1	10
MGMT - CONSOLE	WARN	niedrig	1	0.5	0.5
SYS - SYSLOG - REMOTE	WARN	niedrig	1	0.5	0.5
16 Regeln mit Status OK	OK	verschieden	48	0	0
nicht bewertbar (2 × MANUELL, 13 × N/A)	zählen weder bei den Strafpunkten noch im Gesamtgewicht				
Summe			60		11

Score = $100 \times (1 - 11 \div 60) = 81.7 \rightarrow$ Note **B**. Die Oberfläche zeigt für diese Config: „Penalty 11 von max. 60“ und „81.7 / 100“.

6.4 Befunde zur Regel-Reihenfolge

Firewall-Regeln werden **je Interface und Richtung von oben nach unten** ausgewertet, die **erste passende Regel gewinnt**. Steht eine Regel, die jeden Verkehr erfasst, zu weit oben, werden alle darunter stehenden Regeln *nie erreicht*. Zwei Regeln des Katalogs suchen genau das:

Falsch: „Block All“ steht zuerst
Beispiel: eine Richtung, von oben nach unten

1 Block alle → alle (alle Ports)

2 Pass TCP 80 nie erreicht

3 Pass TCP 443 nie erreicht

4 Pass UDP 123 nie erreicht

Alles endet schon bei Regel 1.

Richtig: „Block All“ steht zuletzt
Beispiel: eine Richtung, von oben nach unten

1 Pass TCP 80 greift

2 Pass TCP 443 greift

3 Pass UDP 123 greift

4 Block alle → alle (Rest)

Erst die Ausnahmen, dann die Auffang-Regel.

Abbildung 8: Warum die Reihenfolge zählt: Links wird die Freigabe nie erreicht, rechts ist die Reihenfolge korrekt.

Regel	Meldet	Schwere
FW - SHADOWED - BLOCK	Eine Block -Regel wird nie erreicht. Der Schutz besteht nur scheinbar, das Netz ist weiter offen.	mittel
FW - SHADOWED - PASS	Eine Freigabe wird nie erreicht. Meist ein Funktionsfehler, oft eine „Block all“-Regel, die am Anfang statt am Ende steht.	niedrig

Als **Auffang-Regel** gilt eine Regel, die alles erfasst: aktiv, Quelle *any*, Ziel *any* ohne Port, Protokoll *any*, weder Zeitplan noch Tag noch OS-Filter, keine Negation und keine Floating-Regel. Der Befund gilt nur für Regeln auf **demselben Interface in derselben Richtung**; eine Auffang-Regel für IPv4 und IPv6 verdeckt beide IP-Versionen, eine reine IPv6-Regel verdeckt IPv4 nicht.

1

WARN

niedrig

FW-SHADOWED-PASS

Freigabe-Regel wird durch vorangehende Auffang-Regel nie erreicht

Firewall-Regeln

2

- #1 [opt2/in] 'Allow HTTP TCP 80' wird nie erreicht - verdeckt durch #0 block 'Block All default deny'
- #2 [opt2/in] 'Allow NTP UDP 123' wird nie erreicht - verdeckt durch #0 block 'Block All default deny'

Freigabe wirkt nicht: eine vorangehende Regel faengt den gesamten Verkehr ab.

Begründung: Steht eine Regel, die jeden Verkehr erfasst, vor einer Freigabe, wird die Freigabe nie ausgewertet. Typisch ist eine 'Block all'-Regel am Anfang statt am Ende. Das ist vor allem ein Funktionsfehler, macht das Regelwerk aber auch schwer nachvollziehbar.

3

MASSNAHME

Allgemeine Block-Regel ans Ende des Interfaces verschieben (oder ganz entfernen, da beide Systeme am Ende ohnehin implizit blockieren) und die Freigaben davor anordnen.

Referenzen: OPNsense-Dokumentation: Firewall Rules - Processing order, pfSense-Dokumentation: Rule Methodology - First match

Abbildung 9: Befund „Freigabe wird nie erreicht“ (Beispiel: pf-reihenfolge.xml, Interface opt2 mit der Beschreibung „NutzLAN“).

- Kartenkopf:** WARN, Schweregrad niedrig, Regel FW-SHADOWED-PASS.
- Befunde:** je Zeile eine verdeckte Regel mit Nummer, Interface und Richtung, Beschreibung und der Regel, die sie verdeckt. Hier stehen zwei Freigaben (HTTP TCP 80 und NTP UDP 123) hinter einer „Block All“-Regel an erster Stelle: Das Interface ist faktisch komplett gesperrt.
- Massnahme:** die allgemeine Block-Regel ans Ende schieben. Beide Systeme blockieren am Ende ohnehin implizit, die eigene Block-all-Regel ist nur für die Protokollierung nötig.

Was nicht erkannt wird Teilüberdeckungen (z. B. eine Regel für das ganze Netz vor einer Regel für einen einzelnen Host), Floating-Regeln, Regeln auf Interface-Gruppen und zur Laufzeit erzeugte Regeln. „Regelnummer“ bedeutet die laufende Nummer in der Konfigurationsdatei, nicht die Anzeigenummer in der Weboberfläche.

7 Das Fleet-Dashboard

Lädst du **zwei oder mehr** Dateien gleichzeitig, zeigt `index.html` statt eines Einzelberichts das Dashboard. Es aggregiert alle Geräte, macht systemische Schwächen sichtbar und führt per Klick zu jedem Einzelbericht. Für die Abbildungen wurden alle zehn Beispiel-Configs aus `tests/configs/` geladen.

7.1 Kennzahlen und Verteilung

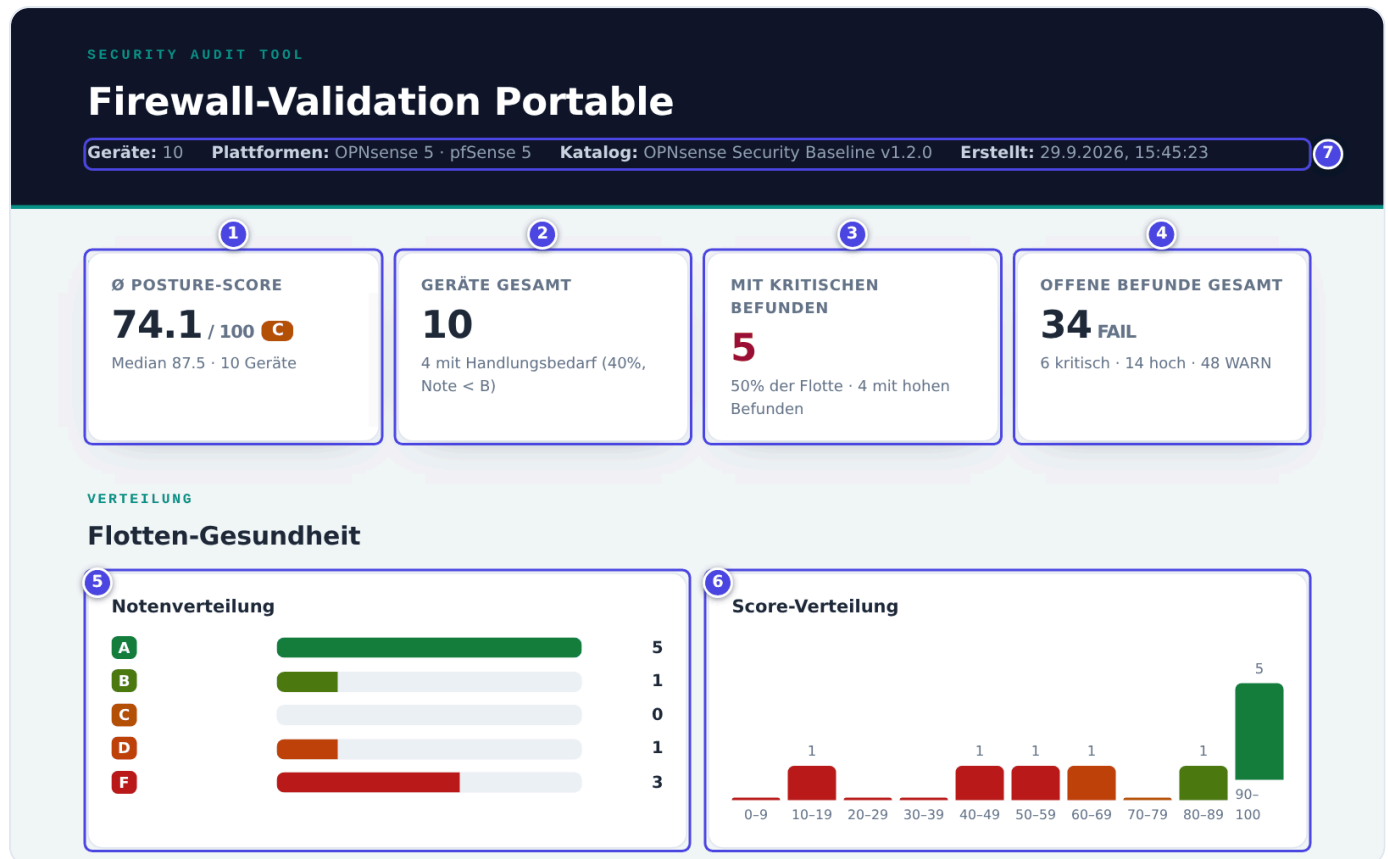


Abbildung 10: Kopf, Kennzahlen und Verteilungen des Dashboards.

- 1 Ø Posture-Score aller Geräte mit Durchschnittsnote, darunter der Median und die Zahl der Geräte.
- 2 Geräte gesamt und wie viele davon Handlungsbedarf haben (Score unter 80, also Note C, D oder F).
- 3 Mit kritischen Befunden: Geräte mit mindestens einem kritischen FAIL-Befund; darunter der Anteil an der Flotte und die Zahl der Geräte mit mindestens einem hohen FAIL-Befund (ein Gerät kann in beiden Zählungen vorkommen).
- 4 Offene Befunde gesamt: Summe aller FAIL-Befunde; darunter, wie viele davon kritisch und hoch sind, und die Summe aller WARN-Befunde.
- 5 Notenverteilung: wie viele Geräte auf jede Note von A bis F entfallen.
- 6 Score-Verteilung: Histogramm in Zehnerschritten (0–9, 10–19 ... 90–100).
- 7 Metazeile: Anzahl der Geräte, Plattformverteilung (OPNsense / pfSense), Katalog und Erstellzeitpunkt.

7.2 Systemische Befunde

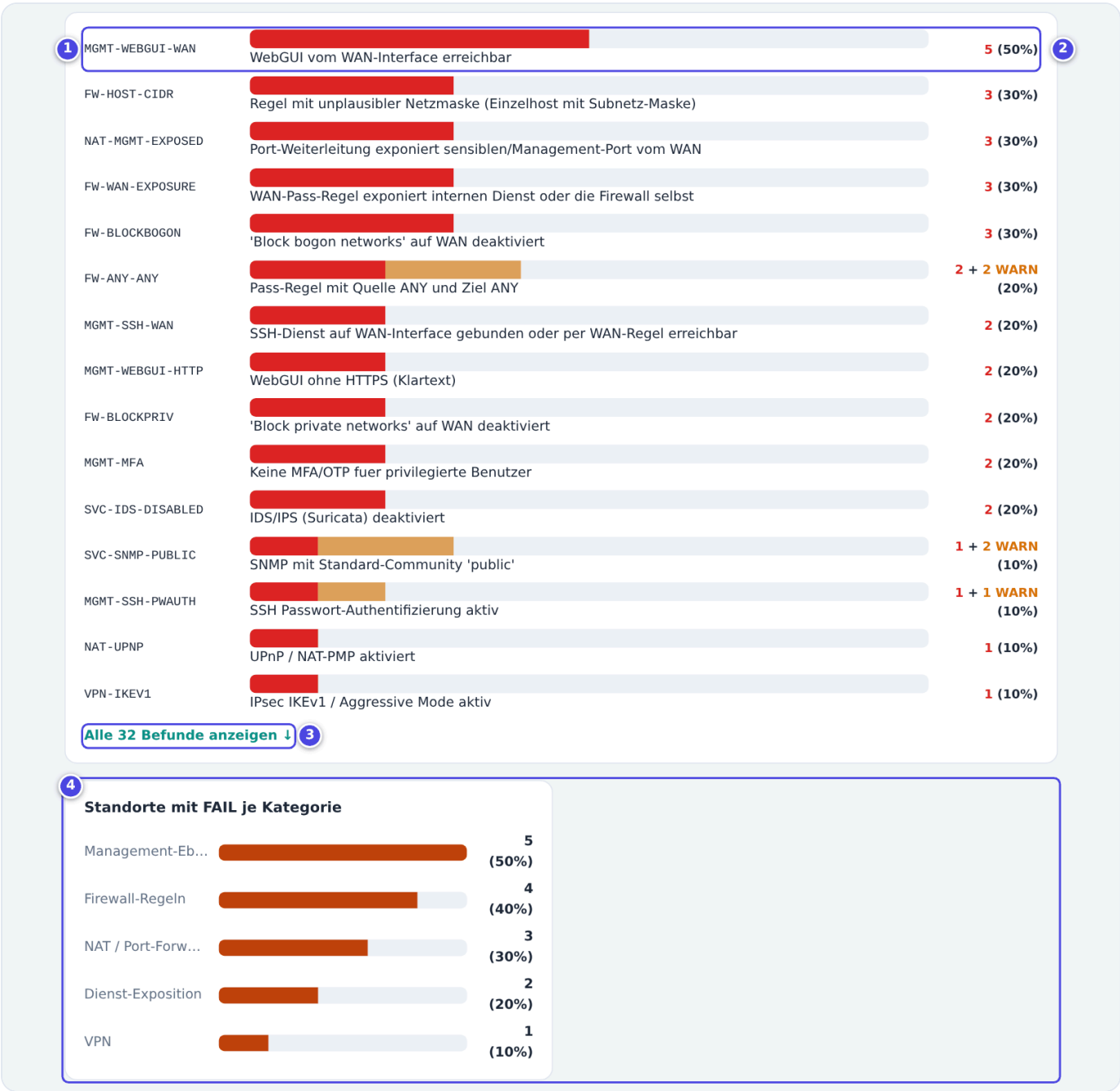


Abbildung 11: Häufigste Befunde flottenweit und Verteilung nach Kategorien.

- 1 Befund-Zeile.** Jede Zeile ist eine Regel, sortiert nach der Zahl betroffener Geräte. Der Balken ist rot für FAIL und orange für WARN. **Ein Klick auf die Zeile** filtert die Geräte-Tabelle darunter auf genau die Geräte mit diesem Befund.
- 2 Anzahl:** betroffene Geräte und Anteil an der Flotte, z. B. „5 (50%)“. Steht dort „2 + 2 WARN (20%)“, haben 2 Geräte ein FAIL und weitere 2 ein WARN.
- 3 Alle ... Befunde anzeigen ↓** klappt die Liste auf. Anfangs werden die 15 häufigsten gezeigt; danach verschwindet die Schaltfläche.
- 4 Standorte mit FAIL je Kategorie:** wie viele Geräte in jeder Kategorie mindestens ein FAIL haben. Dieses Feld ist nur eine Anzeige und nicht anklickbar.

Wozu das gut ist Ein Befund, der bei vielen Standorten auftritt (z. B. MGMT-WEBGUI-WAN bei 50 %), deutet auf eine **gemeinsame Ursache** hin: eine Vorlage, ein Standard-Rollout oder eine Gewohnheit. Diese Ursache einmal zu beheben ist wirksamer, als jedes Gerät einzeln zu bearbeiten.

7.3 Die Tabelle „Geräte & Standorte“

STANDORTE

Geräte & Standorte

Suche: Standort, Hostname,

Alle A B C D F

Alle OPNsense pfSense

nur kritische

Handlungsbedarf (<B)

zeige 1-10 von 10

STANDORT	PLATTFORM	GRUPPE	SCORE ▲	NOTE	KRIT	HOCH	MITTEL	NIEDRIG	FAIL	WARN	STAND
opn-schwach.test.lan opn-schwach.test.lan	OPNsense	(ungruppiert)	10.5	F	2	7	5	2	16	15	29.9.202 15:45:27
pf-schwach.test.lan pf-schwach.test.lan	pfSense	(ungruppiert)	43.2	F	1	3	1	2	7	12	29.9.202 15:45:27
opn-variante.test.lan opn-variante.test.lan	OPNsense	(ungruppiert)	50.6	F	1	3	1	1	6	6	29.9.202 15:45:27
opn-reihenfolge.test.lan opn-reihenfolge.test.lan	OPNsense	(ungruppiert)	68.3	D	1	1	0	1	3	5	29.9.202 15:45:27
opn-minimal.test.lan opn-minimal.test.lan	OPNsense	(ungruppiert)	81.7	B	1	0	0	0	1	2	29.9.202 15:45:27
pf-idsaus.test.lan pf-idsaus.test.lan	pfSense	(ungruppiert)	93.3	A	0	0	1	0	1	2	29.9.202 15:45:27
pf-reihenfolge.test.lan pf-reihenfolge.test.lan	pfSense	(ungruppiert)	95.0	A	0	0	0	0	0	4	29.9.202 15:45:27
pf-minimal.test.lan pf-minimal.test.lan	pfSense	(ungruppiert)	98.2	A	0	0	0	0	0	2	29.9.202 15:45:27
opn-gehaertet.test.lan opn-gehaertet.test.lan	OPNsense	(ungruppiert)	100.0	A	0	0	0	0	0	0	29.9.202 15:45:27
pf-gehaertet.test.lan pf-gehaertet.test.lan	pfSense	(ungruppiert)	100.0	A	0	0	0	0	0	0	29.9.202 15:45:27

< Zurück

Seite 1 / 1

Weiter >

Pro Seite: 50

Abbildung 12: Tabelle der Geräte mit Suche, Filtern, Sortierung und Seitenwechsel.

- Suche** über Standort, Hostname, Gruppe und Plattform (Groß-/Kleinschreibung egal).
- Notenfilter:** Alle oder eine einzelne Note A bis F.
- Plattformfilter:** Alle, OPNsense oder pfSense.

- 4 **nur kritische** : nur Geräte mit mindestens einem kritischen FAIL-Befund. Ein zweiter Klick hebt den Filter auf.
- 5 **Handlungsbedarf (<B)** : nur Geräte mit Score unter 80.
- 6 **Zähler** „zeige 1–10 von 10“. Bei aktiven Filtern erscheint zusätzlich „(gefiltert aus ...)“.
- 7 **Spaltenüberschrift**. Ein Klick sortiert nach dieser Spalte, ein zweiter Klick kehrt die Richtung um. Beim Öffnen ist nach *Score aufsteigend* sortiert: die schwächsten Geräte stehen oben.
- 8 **Seitenwechsel** mit **< Zurück** und **Weiter >**.
- 9 **Pro Seite**: 25, 50, 100 oder Alle.

Spalte	Inhalt
STANDORT	Hostname aus der Konfiguration (oder Dateiname). Der Hostname steht zweimal, einmal als Name und einmal als Kennung.
PLATTFORM	OPNsense oder pfSense.
GRUPPE	In dieser Version stets „(ungruppiert)“. Ordnerstrukturen werden von der Browser-Fassung nicht ausgewertet.
SCORE / NOTE	Posture-Score und Note des Geräts.
KRIT / HOCH / MITTEL / NIEDRIG	Anzahl der FAIL -Befunde je Schweregrad. (Der Schweregradbalken im Einzelbericht zählt dagegen FAIL und WARN.)
FAIL / WARN	Anzahl der Regeln mit diesem Status.
STAND	Zeitpunkt der Auswertung.
BERICHT	Link öffnen : öffnet den Einzelbericht dieses Geräts.

Breite Tabelle Die Tabelle ist breiter als die Seitenspalte und lässt sich **seitlich scrollen**. Die Spalte BERICHT mit dem Link **öffnen** steht ganz rechts. Auf schmalen Fenstern musst du dorthin scrollen.

7.4 Auf einen Befund filtern

STANDORTE

Geräte & Standorte

Suche: Standort, Hostname,

Alle

A

B

C

D

F

Alle

OPNsense

pfSense

nur kritische

Handlungsbedarf (<B)

zeige 1–5 von 5 (gefiltert aus 10)

1

Gefiltert auf Befund: MGMT-WEBGUI-WAN - WebGUI vom WAN-Interface erreichbar x

STANDORT	PLATTFORM	GRUPPE	SCORE ▲	NOTE	KRIT	HOCH	MITTEL	NIEDRIG	FAIL	WARN	STAND
opn-schwach.test.lan opn-schwach.test.lan	OPNsense	(ungruppiert)	10.5	F	2	7	5	2	16	15	29.9.2022 15:45:29
pf-schwach.test.lan pf-schwach.test.lan	pfSense	(ungruppiert)	43.2	F	1	3	1	2	7	12	29.9.2022 15:45:29

Abbildung 13: Tabelle, gefiltert auf einen einzelnen Befund.

- 1 **Filter-Chip** „Gefiltert auf Befund: ID – Titel“ nach einem Klick auf eine Befund-Zeile. Das ✕ hebt den Filter auf.
- 2 **Zähler** zeigt jetzt „zeige 1–5 von 5 (gefiltert aus 10)“. Der Befundfilter lässt sich mit Note, Plattform, Suche und den beiden Schaltflächen kombinieren.

7.5 Ein Gerät öffnen und zurückkehren

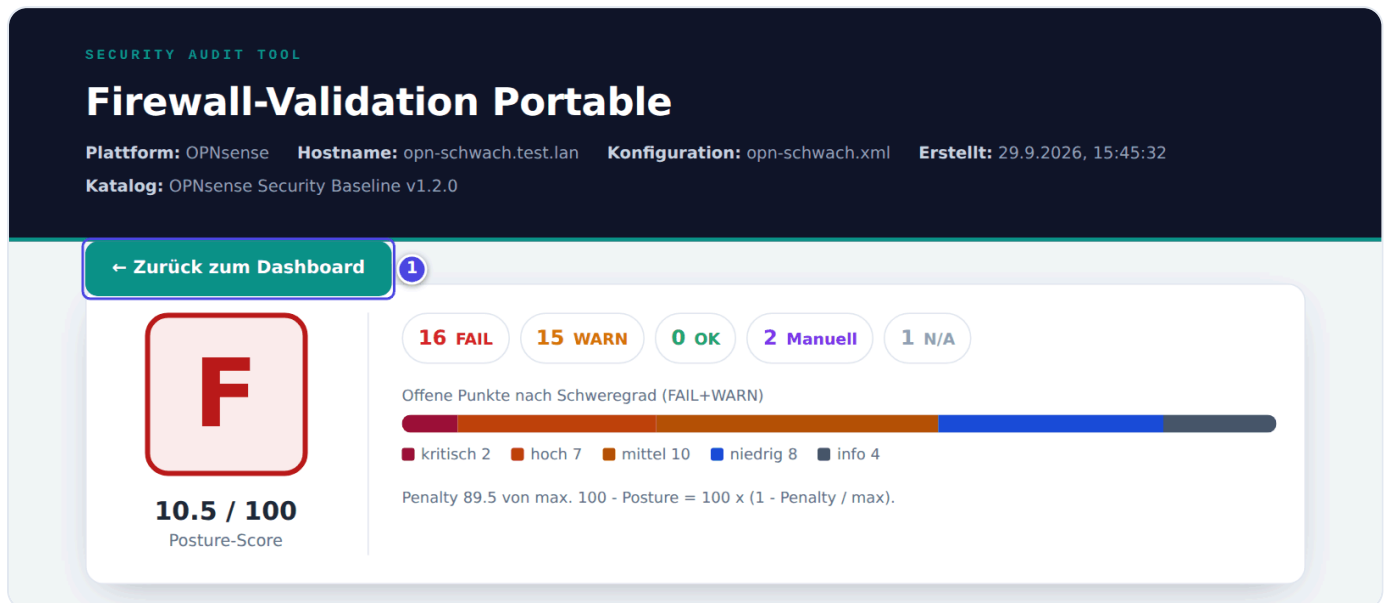


Abbildung 14: Einzelbericht eines Geräts, aufgerufen aus dem Dashboard.

- 1 **← Zurück zum Dashboard** führt zurück. Das Dashboard wird dabei **neu aufgebaut**: Suche, Filter, Sortierung und Seitenauswahl sind danach wieder auf den Ausgangszustand gesetzt. Notiere dir vorher, wonach du gefiltert hattest. Der Bericht selbst funktioniert wie in Kapitel 5.

8 Drucken, PDF und mobile Ansicht

8.1 Bericht drucken oder als PDF sichern

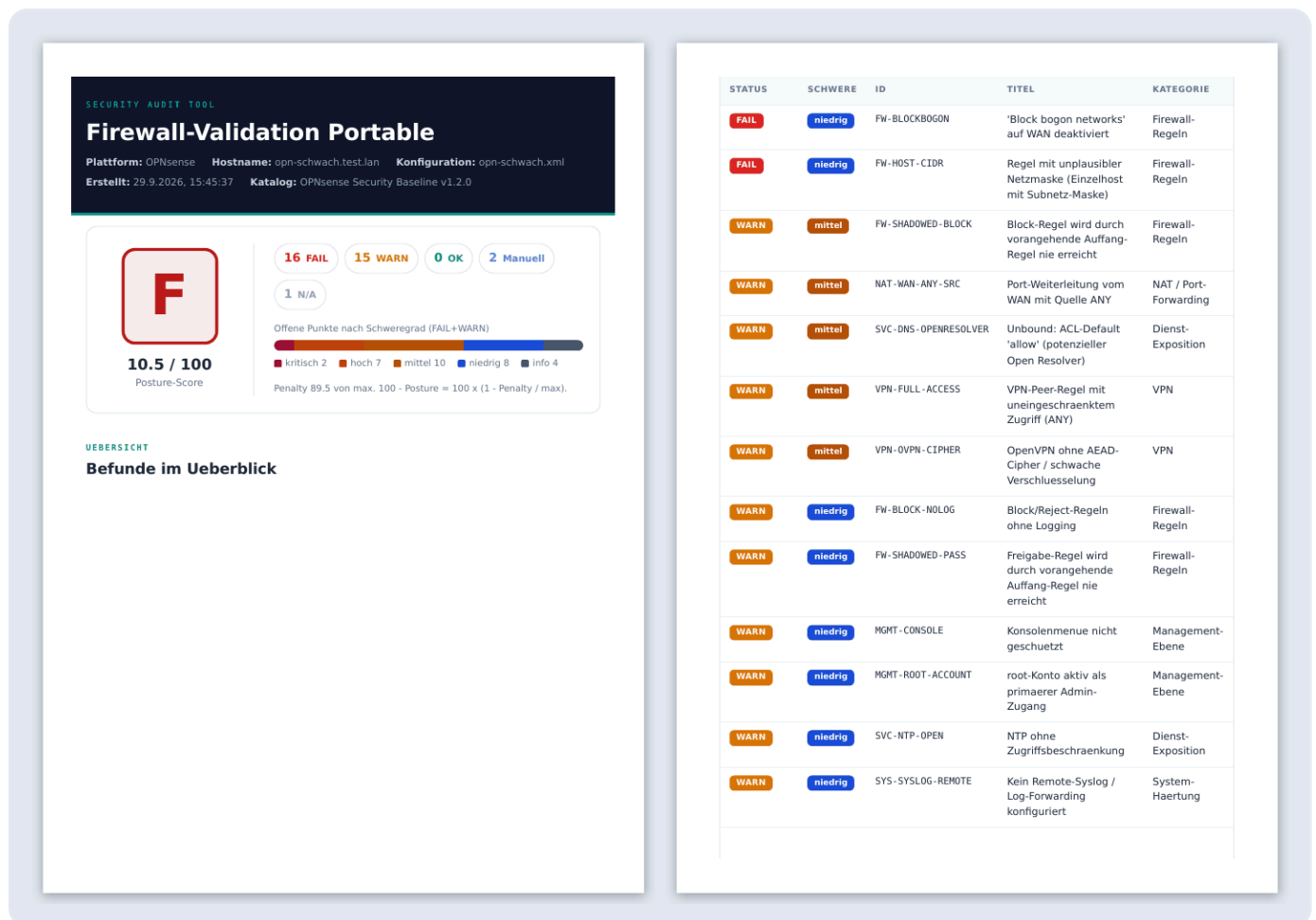


Abbildung 15: Druckansicht (Seite 1 und 3) eines Berichts. Steuerelemente werden ausgeblendet.

- 1 Im Bericht **Strg** + **P** (macOS: **⌘** + **P**) drücken.
 - 2 Als Ziel *Als PDF speichern* oder einen Drucker wählen.
 - 3 *Hintergrundgrafiken* aktivieren, sonst fehlen die Farben der Badges und Balken.
 - 4 *Kopf- und Fußzeilen* nach Bedarf abschalten.
- Für den Druck werden Upload-Bereich und Filterleiste ausgeblendet, die Schrift wird verkleinert und Abschnitte werden möglichst nicht umbrochen.
 - **Gedruckt wird, was der Filter gerade zeigt.** Willst du alle Regeln im Ausdruck, wähle vorher **Alle**; für einen kompakten Ausdruck genügt der Standardfilter „Offen“.
 - Bei langen Berichten kann die Übersichtstabelle erst auf der zweiten Seite beginnen, sodass auf der ersten Platz frei bleibt (beobachtet in Chromium-basierten Browsern). Das ist eine Eigenheit der Seitenumbrüche, keine fehlende Information.

8.2 Ansicht auf dem Smartphone

Die Oberfläche passt sich schmalen Bildschirmen an. Die Kopfangaben stehen untereinander, die Note steht über den Zählern, und breite Tabellen lassen sich seitlich wischen.

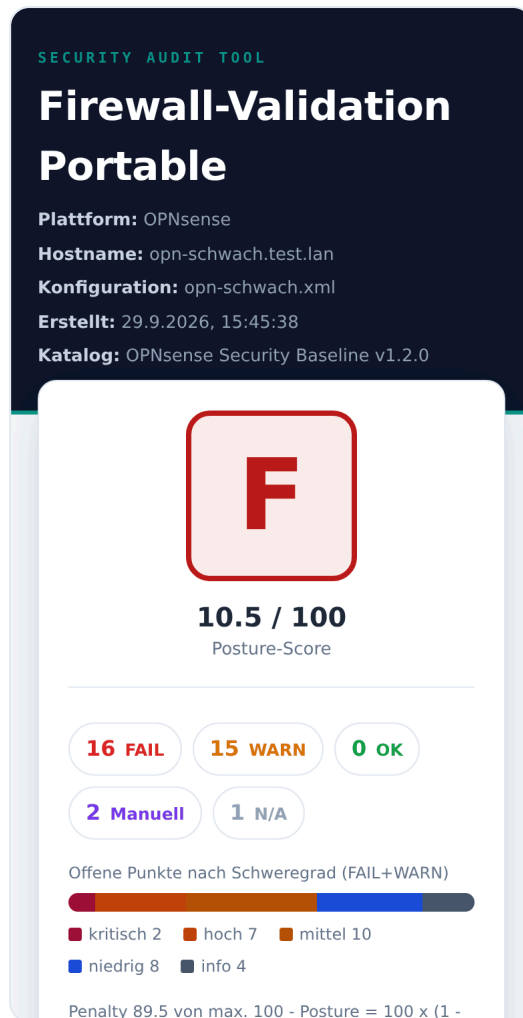


Abbildung 16: Einzelbericht auf einem Smartphone (390 Pixel Breite).

9 Fehlermeldungen und Fehlerbehebung (index.html)

9.1 Meldungen im Überblick

Fehlermeldungen erscheinen in einem roten Banner oben auf der Seite.

Meldung / Beobachtung	Ursache und Abhilfe
<i>rules.js konnte nicht geladen werden. Stelle sicher, dass rules.js im gleichen Verzeichnis liegt wie index.html.</i>	<code>rules.js</code> fehlt, liegt in einem anderen Ordner oder ist beschädigt. Datei neben <code>index.html</code> legen.
<i>engine.js konnte nicht geladen werden. ...</i>	Dasselbe für <code>engine.js</code> . Ohne Engine ist keine Auswertung möglich.
<i>Keine XML-Dateien ausgewaehlt.</i>	Keine der Dateien endet auf <code>.xml</code> (Kleinbuchstaben). Datei umbenennen.
<i>Fehler: datei.xml: Wurzelement ist <...>, erwartet <opnsense> oder <pfsense>.</i>	Die Datei ist keine OPNsense-/pfSense-Konfiguration (oder ein verschlüsselter Export, eine leere Datei oder kein XML). Erneut exportieren, ohne Verschlüsselung. Beispiel unten: eine HTML-Datei statt einer Konfiguration.

Status FEHLER bei einer Regel, Text „Pruefung fehlgeschlagen: ...“	Die Prüfung ist mit einem Fehler abgebrochen, das übrige Audit läuft weiter. Bei eigenen Regeln: Prüfung im Editor mit „Aufbau prüfen“ und „Probelauf“ kontrollieren. Bei mitgelieferten Regeln: Fehlertext notieren und melden.
Bericht wirkt zu gut, Hostname „Firewall“, viele N/A	Möglicherweise unvollständige oder beschädigte Datei (Warnhinweis in Kapitel 4). Export wiederholen.
Nichts passiert nach dem Ablegen	Die Datei endet nicht auf <code>.xml</code> in Kleinbuchstaben, oder es wurde ein Ordner statt einer Datei abgelegt.
Alte Ergebnisse trotz neuer <code>rules.js</code>	Der Browser hat die alte Datei zwischengespeichert. Seite mit <code>Strg</code> + <code>F5</code> neu laden. Die Kopfzeile muss die neue Katalogversion zeigen.

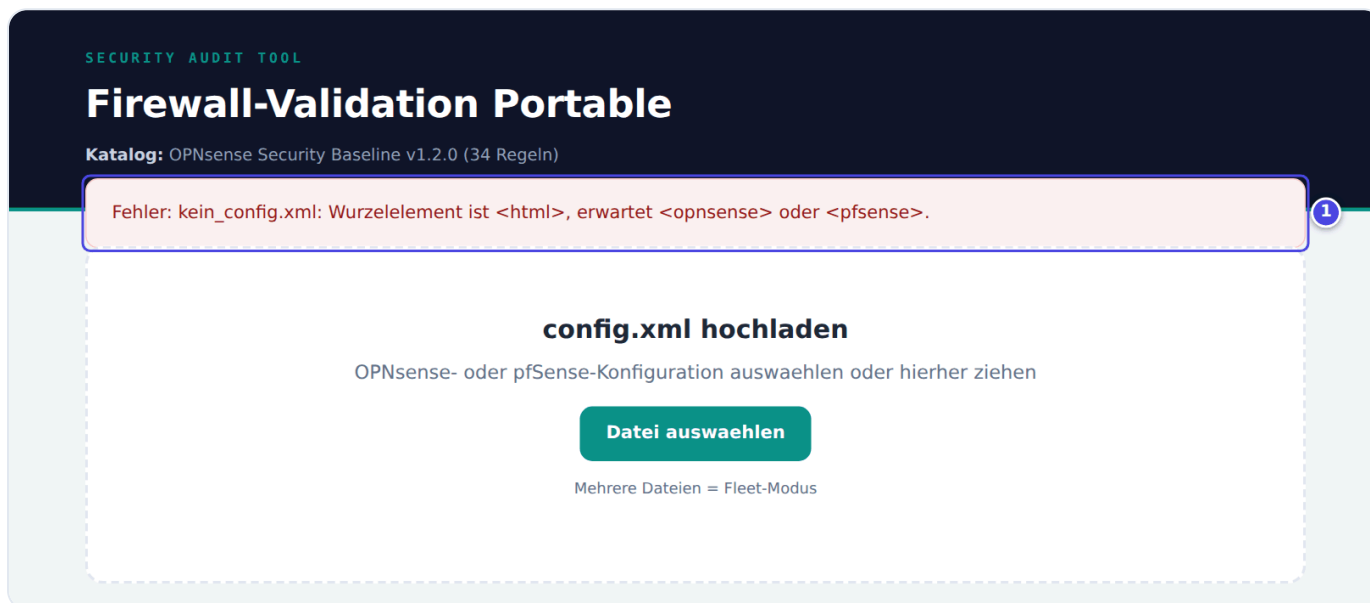


Abbildung 17: Meldung bei einer Datei, die keine Firewall-Konfiguration ist.

- Fehlerbanner** mit Dateiname und Grund. Der Upload-Bereich bleibt bedienbar, du kannst sofort eine andere Datei wählen.

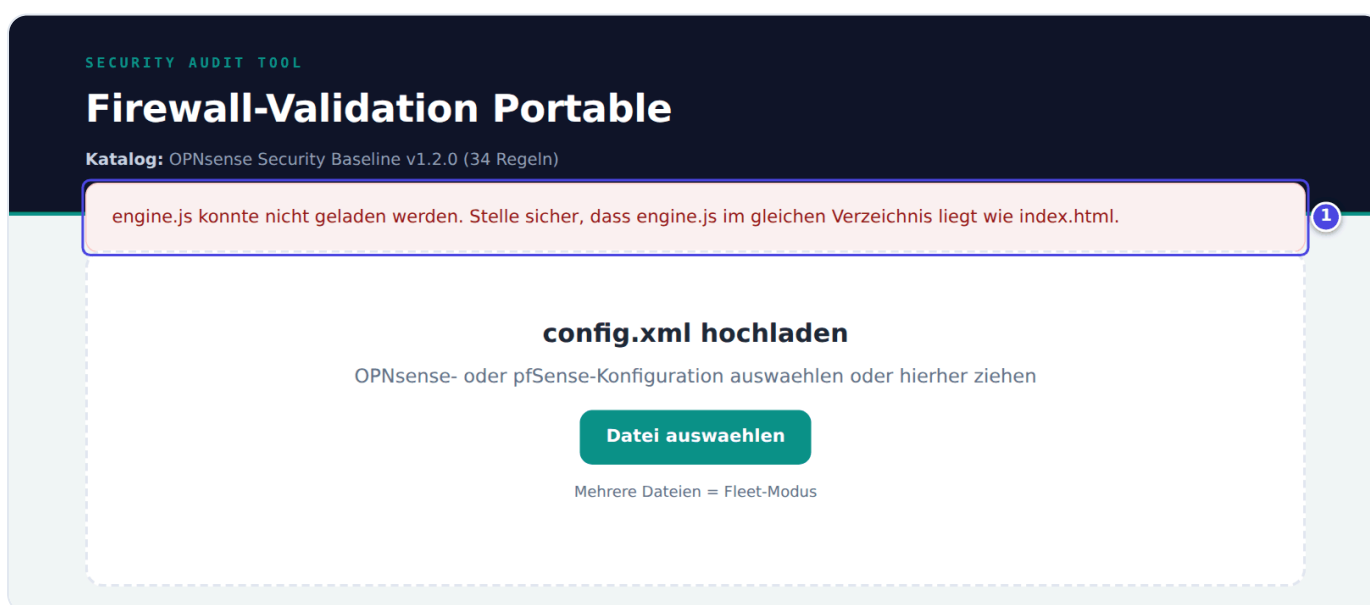


Abbildung 18: Meldung, wenn engine.js fehlt.

- Fehlerbanner** beim Öffnen der Seite. Ergänze die fehlende Datei und lade die Seite neu.

10 Grenzen der Auswertung

Damit du Ergebnisse richtig einordnest, hier, was das Werkzeug bewusst oder technisch bedingt **nicht** leistet:

Grenze	Folge
Nur die <code>config.xml</code> wird gelesen	Laufzeitzustand (tatsächliche Regeln in der Firewall, Firmware-Stand, Plugin-Zustand) ist unsichtbar. Dafür gibt es die Status MANUELL.
Nicht jeder Ablageort der Einstellungen ist abgedeckt	Hersteller verschieben Einstellungen zwischen Versionen. Wahrscheinliche Lücken bei neueren OPNsense-Versionen: Regeln aus <i>Firewall</i> → <i>Automation</i> , 1:1-NAT in der neuen Struktur, das SNMP-Plugin und OpenVPN-Legacy-Server. Bei „N/A“ trotz gesetzter Einstellung: Testplan (<code>TESTPLAN.md</code>) und Kapitel 19.8.
Floating-Regeln, Interface-Gruppen, Teilüberdeckungen	Werden bei den Reihenfolge-Prüfungen nicht einbezogen (Kapitel 6.4).
Bekannte Lücken einzelner Prüfungen	Beispiele: <code>SVC-NTP-OPEN</code> erkennt nur Regeln mit ausdrücklichem Port 123, nicht solche ohne Port; <code>VPN-FULL-ACCESS</code> erkennt VPN-Regeln nur auf zugewiesenen VPN-Interfaces, nicht auf Gruppen-Reitern.
Beschädigte Dateien	Werden nicht immer abgewiesen (Kapitel 4).
Keine Speicherung	Berichte werden nicht abgelegt. Zum Aufbewahren drucken oder als PDF speichern.
Gruppierung von Standorten	Die Browser-Fassung kennt keine Gruppen (Spalte GRUPPE ist konstant).

Einordnung Ein Score von 100 heißt „keine der 34 Prüfungen schlägt an“ und nicht „die Firewall ist sicher“. Ein niedriger Score ist ein Grund, die Befunde zu prüfen, aber jeder Befund braucht Kontext: Eine bewusst offene Web-Weiterleitung ist etwas anderes als eine versehentlich offene Verwaltungsoberfläche.

Teil B – Regel-Editor (rules-editor.html)

Kapitel 11 bis 23

11 Konzept und Arbeitsablauf

11.1 Wofür der Editor da ist

Der Regel-Editor pflegt den **Regelkatalog** `rules.js`. Damit kannst du, ohne Programmcode zu ändern,

- den **Schweregrad** einer Regel anheben oder senken, Regeln **deaktivieren** und Texte anpassen,
- **Einstellungen** ändern: Gewichte, Strafpunktfaktoren, welche Interfaces als WAN gelten, Listen wie sensible Ports,
- **neue Regeln** aus Bausteinen der Regelsprache zusammensetzen und vorher an einer echten Konfiguration ausprobieren,
- Änderungen als **neue, versionierte Katalogdatei** mit Änderungsprotokoll speichern.

Was der Editor **nicht** kann: neue Arten von Prüfungen erfinden (neue Datenquellen, neue Operatoren oder neue Bausteine). Diese erfordern eine Erweiterung von `engine.js` (Kapitel 23).

11.2 Der Arbeitsablauf



Abbildung 19: Vom Ändern bis zur wirksamen Regel: fünf Schritte.

Wichtig: Nichts wird automatisch gespeichert. Der Editor arbeitet mit einer Arbeitskopie im Browser. „Übernehmen“ schreibt eine Änderung nur in diese Arbeitskopie. Erst **Als neue Version speichern ...** erzeugt eine Datei. Schließt du das Fenster vorher, sind die Änderungen weg (der Browser fragt zur Sicherheit nach). Die Datei `rules.js` im Ordner wird vom Editor **nicht** selbst verändert; du ersetzt sie von Hand (Kapitel 20).

12 Starten und Katalog laden

12.1 Automatisches Laden

Öffne `rules-editor.html` per Doppelklick. Liegt `rules.js` im selben Ordner, wird der Katalog **automatisch** geladen (der Editor bindet die Datei wie `index.html` ein). Oben steht dann der Katalogname mit Regelanzahl, rechts die Version, und im Arbeitsbereich „Katalog geladen“. Findet der Editor keine `rules.js`, bleibt er leer und bietet `rules.js öffnen ...` an.

Zusatzdatei engine.js Der Editor lädt außerdem `engine.js` aus demselben Ordner. Sie liefert die Sprachreferenz, die Aufbauprüfung und den Probelauf. Fehlt sie, kannst du weiterhin Texte, Schweregrade und Einstellungen bearbeiten; im Bereich „Prüfung“ erscheint dann der Hinweis „engine.js nicht gefunden: Prüfungen können hier nicht geprüft oder ausprobiert werden.“

12.2 Eine andere Katalogdatei öffnen

- 1 Klicke `rules.js öffnen ...` in der Werkzeugleiste. Liegen ungespeicherte Änderungen vor, fragt der Editor: *Ungespeicherte Änderungen verwerfen und andere Datei öffnen?*
- 2 Wähle die Datei. Erlaubt sind eine `rules.js` (Format `window.RULES_DATA = { ... };`) oder eine reine JSON-Datei mit derselben Struktur, z. B. eine zuvor exportierte `rules.json`.
- 3 Der Editor liest die Datei **nur als Daten** (sie wird nicht ausgeführt) und meldet die Anzahl der Regeln.

Die geladene Datei ist die Vergleichsbasis: Alles, was du änderst, wird später gegen diesen Stand verglichen.

13 Die Oberfläche im Überblick

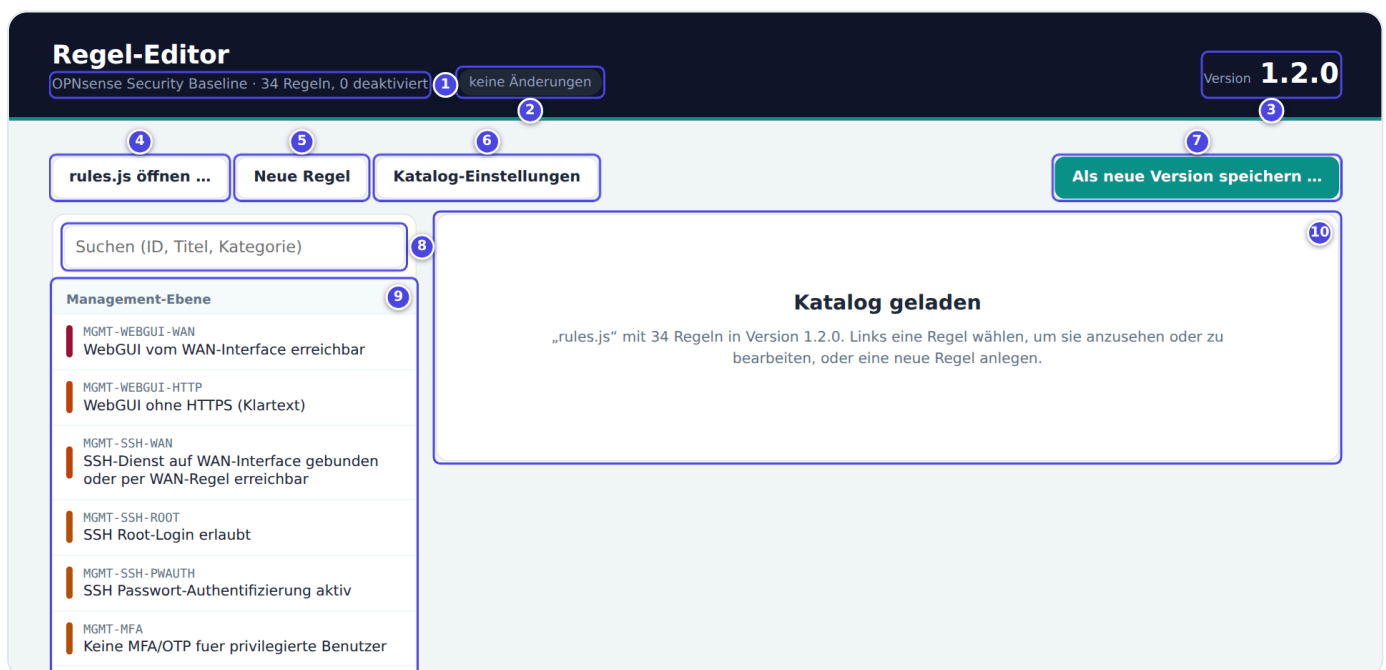


Abbildung 20: Der Regel-Editor unmittelbar nach dem Laden.

- 1 **Katalogname** mit Anzahl der Regeln und der deaktivierten Regeln.

- 2 **Änderungsanzeige.** Zeigt „keine Änderungen“ oder in Gelb „N ungespeicherte Änderung(en)“. Gezählt werden neue, geänderte und entfernte Regeln sowie geänderte Katalog-Einstellungen (Letztere als 1).
- 3 **Version** des Katalogs, wie sie in der geladenen Datei steht.
- 4 **rules.js öffnen ...** lädt eine andere Katalogdatei.
- 5 **Neue Regel** legt eine Regel an (Kapitel 15).
- 6 **Katalog-Einstellungen** öffnet Name, Gewichte, WAN-Interfaces und Strafpunktfaktoren (Kapitel 16).
- 7 **Als neue Version speichern ...** erzeugt die neue Katalogdatei (Kapitel 20).
- 8 **Suchfeld** für ID, Titel und Kategorie (Groß-/Kleinschreibung egal).
- 9 **Regelliste**, nach Kategorien gruppiert, in der Reihenfolge des Katalogs.
- 10 **Arbeitsbereich:** hier erscheint das Formular der gewählten Regel oder die Katalog-Einstellungen.

Die drei Schaltflächen **Neue Regel**, **Katalog-Einstellungen** und **Als neue Version speichern ...** sind erst bedienbar, wenn ein Katalog geladen ist. Auf schmalen Fenstern steht die Regelliste über dem Arbeitsbereich.

13.1 Änderungen im Blick behalten

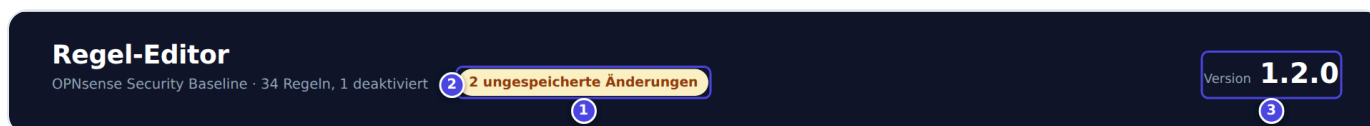


Abbildung 21: Kopfbereich mit ungespeicherten Änderungen (hier: eine Regel geändert, eine deaktiviert).

- 1 **Änderungsanzeige** in Gelb: „2 ungespeicherte Änderungen“.
- 2 **Zähler.** „34 Regeln, 1 deaktiviert“ aktualisiert sich mit jeder Änderung.
- 3 **Version** bleibt die der geladenen Datei, bis du speicherst.

14 Regeln ansehen und bearbeiten

14.1 Die Regelliste

1

Firewall-Regeln

2

FW-ANY-ANY

Pass-Regel mit Quelle ANY und Ziel ANY

geändert**3**

FW-WAN-EXPOSURE

WAN-Pass-Regel exponiert internen Dienst oder die Firewall selbst

FW-BLOCKPRIV

'Block private networks' auf WAN deaktiviert

ohne Prüfung**6**

FW-BLOCKBOGON

'Block bogon networks' auf WAN deaktiviert

FW-HOST-CIDR

Regel mit unplausibler Netzmaske (Einzelhost mit Subnetz-Maske)

~~FW-DISABLED-RULES~~

~~Deaktivierte Firewall-Regeln vorhanden (Hygiene)~~

geändert**4****5**

FW-BLOCK-NOLOG

Block/Reject-Regeln ohne Logging

ohne Prüfung

FW-SHADOWED-BLOCK

Block-Regel wird durch vorangehende Auffang-Regel nie erreicht

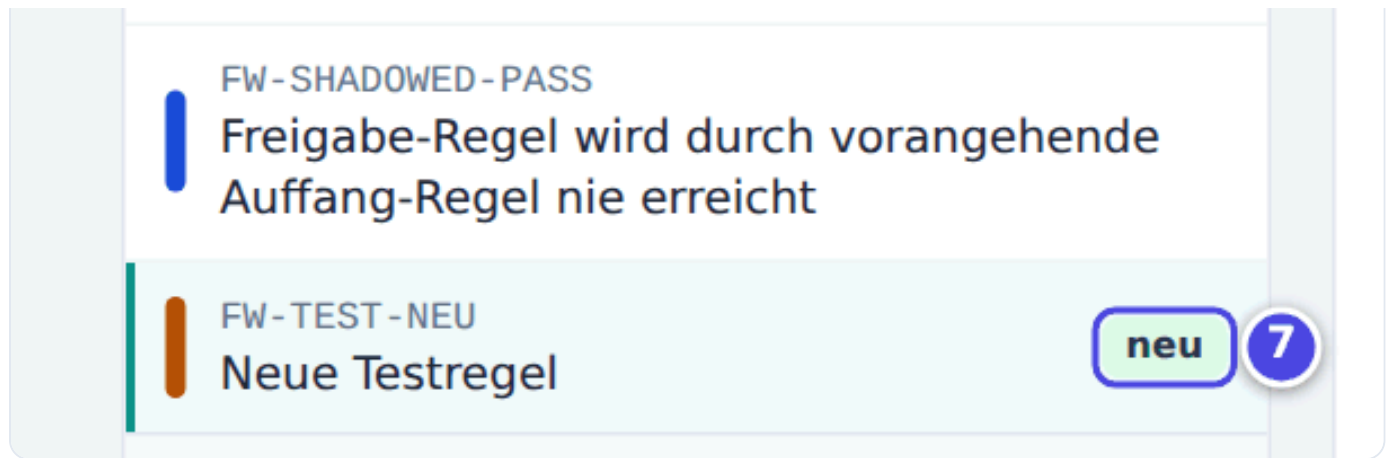


Abbildung 22: Regelliste mit Markierungen (Ausschnitt der Kategorie „Firewall-Regeln“).

- 1 **Kategorie-Überschrift.** Regeln derselben Kategorie stehen zusammen.
- 2 **Farbbalken** = Schweregrad: weinrot kritisch, orange hoch, bernstein mittel, blau niedrig, grau info.
- 3 **„geändert“** (gelb): Die Regel unterscheidet sich vom geladenen Stand.
- 4 Auch ein reines Deaktivieren zählt als Änderung.
- 5 **Durchgestrichen** = Regel ist deaktiviert und wird nicht geprüft.
- 6 **„ohne Prüfung“** (grau): Die Regel hat keine Prüfdefinition und erscheint im Bericht als MANUELL.
- 7 **„neu“** (grün): Regel, die es im geladenen Stand nicht gab. Die ausgewählte Regel hat zusätzlich einen türkisen Balken am linken Rand und hellen Hintergrund.

Ein Klick auf eine Regel öffnet ihr Formular im Arbeitsbereich. Über das Suchfeld grenzt du die Liste ein; „Keine Regel passt zur Suche.“ erscheint, wenn nichts passt.

14.2 Das Regelformular

10 NAT-MGMT-EXPOSED

Änderungen mit „Übernehmen“ in den Katalog schreiben. Gespeichert wird erst über „Als neue Version speichern ...“.

1 ID

NAT - MGMT - EXPOSED

Großbuchstaben, Ziffern und Bindestrich, z. B. SVC-DOT-MISSING. Muss eindeutig sein.

2 Schweregrad

Kritisch (Gewicht 10) ▾

3 Titel

Port-Weiterleitung exponiert sensiblen/Management-Port vom WAN

4 Kategorie

NAT / Port-Forwarding

Vorhandene Kategorie wählen oder eine neue eintippen.

5 ☒ Regel aktiv (deaktivierte Regeln werden nicht geprüft)

6 Begründung

Weiterleitungen von WAN auf Ports wie SSH(22), RDP(3389), SMB(445), DB(3306/5432/1433/27017/6379), VNC(5900) oder Admin-Panels stellen interne Verwaltungsdienste ins Internet.

7 Maßnahme

Solche Weiterleitungen entfernen oder ausschliesslich ueber VPN bereitstellen; falls noetig, Quelle hart auf bekannte IPs begrenzen.

8 Referenzen

Eine Referenz pro Zeile.

9 Parameter (JSON, optional)

```
{  "sensitive_ports": [    22,    23,    135,    138  ]}
```

Wirksam: sensitive_ports – Portnummern (1-65535), deren Weiterleitung vom WAN als sensibel gilt.

Abbildung 23: Formular einer Regel (Beispiel: NAT-MGMT-EXPOSED).

- 1 ID.** Erlaubt sind Großbuchstaben, Ziffern und Bindestrich (Beginn mit Buchstabe oder Ziffer). Kleinbuchstaben werden beim Übernehmen in Großbuchstaben umgewandelt. Die ID muss im Katalog eindeutig sein.
- 2 Schweregrad** mit dem aktuellen Gewicht in Klammern (z. B. „Kritisch (Gewicht 10)“).
- 3 Titel.** Erscheint in der Übersichtstabelle und auf der Karte.
- 4 Kategorie.** Eine Vorschlagsliste bietet die vorhandenen an; ein neuer Name legt eine neue Kategorie an.
- 5 Regel aktiv.** Ohne Haken wird die Regel nicht geprüft und erscheint nirgends im Bericht.
- 6 Begründung:** warum ist das ein Risiko. Erscheint auf der Karte.
- 7 Maßnahme:** was ist zu tun. Erscheint in der grünen Box der Karte.

- 8 **Referenzen**, eine je Zeile. Erscheinen auf der Karte.
- 9 **Parameter (JSON)**. Einstellwerte der Regel; darunter erklärt der Editor, welche davon wirken (Kapitel 14.4).
- 10 **Überschrift** = ID der gewählten Regel.

Unterhalb der Parameter folgt der Bereich **Prüfung** (Kapitel 17) und ganz unten die Schaltflächen:

Schaltfläche	Wirkung
Übernehmen	Prüft die Eingaben und schreibt sie in die Arbeitskopie. Bei Fehlern erscheint die Meldung am betroffenen Feld (rot umrandet) und oben die Meldung „Bitte die markierten Felder prüfen.“ Es wird nichts übernommen.
Änderungen verwerfen	Setzt das Formular auf den Stand der Arbeitskopie zurück, also auf den Zustand nach dem letzten „Übernehmen“.
Auf geladenen Stand zurücksetzen	Erscheint nur, wenn die Regel vom geladenen Stand abweicht. Stellt die Regel so wieder her, wie sie in der geladenen Datei stand.
Regel löschen	Nach Rückfrage („Regel „...“ aus dem Katalog löschen?“) wird die Regel aus der Arbeitskopie entfernt. Ein Rückgängig gibt es nicht; verwirft im Zweifel die Arbeitskopie durch erneutes Laden der Datei.

IDs bestehender Regeln nicht umbenennen. Drei mitgelieferte Regeln (`NAT-MGMT-EXPOSED` , `SVC-SNMP-PUBLIC` , `FW-ANY-ANY`) haben fest verdrahtete Parameter, die die Engine über die **ID** findet. Nach einer Umbenennung gelten wieder die Standardwerte. Außerdem erscheint eine umbenannte Regel im Änderungsprotokoll als „entfernt“ plus „neu“. Willst du eine Regel abwandeln, lege besser eine neue an (Kapitel 15).

14.3 Schweregrad ändern und Regeln deaktivieren

Der **Schweregrad** legt das Gewicht der Regel im Score fest (Kritisch 10, Hoch 6, Mittel 3, Niedrig 1, Info 0; anpassbar, Kapitel 16). Er wirkt in zwei Richtungen: Ein FAIL oder WARN kostet mehr Punkte, *und* auch ein bestandenenes OK erhöht das Gesamtgewicht im Nenner. Eine Änderung verschiebt daher den Score auch bei Geräten, bei denen die Regel bestanden ist.

Nicht jede Änderung des Schweregrads wirkt bei jedem Fund. Einige Regeln bestimmen den Schweregrad im Fund selbst und übersteuern den Katalogwert (Beispiel: `NAT-MGMT-EXPOSED` meldet „kritisch“ bei beliebiger Quelle, sonst „hoch“). Bei diesen Regeln gilt der Katalogwert nur für die Fälle ohne eigene Angabe (z. B. OK). Die betroffenen Regeln und Fälle stehen im Anhang **Anhang A**. Um den wirksamen Schweregrad eines solchen Falls zu ändern, bearbeite das Feld `schweregrad` im `ergebnis` der Prüfung (Kapitel 18.6).

Das **Deaktivieren** ist die sanfte Alternative zum Löschen: Die Regel bleibt im Katalog (und im Editor durchgestrichen sichtbar), wird aber nicht geprüft und erscheint weder im Bericht noch im Score. Die Kopfzeile von `index.html` zählt sie weiterhin mit („34 Regeln“).

14.4 Parameter

Parameter sind einstellbare Werte einer Regel, z. B. eine Portliste. Sie stehen im Feld „Parameter (JSON)“ als JSON-Objekt. Unter dem Feld zeigt der Editor, **welche Parameter tatsächlich wirken**:

Regel	Parameter	Bedeutung und Standard
NAT-MGMT-EXPOSED	sensitive_ports	Ports (1-65535), deren Weiterleitung vom WAN als sensibel gilt. Standard: 22, 23, 135, 139, 445, 1433, 3306, 3389, 5432, 5900, 5901, 6379, 9200, 11211, 27017. Port-Aliase werden aufgelöst und mitgeprüft. Der zweite Eintrag <code>sensitive_port_aliases_hint</code> ist nur ein Hinweistext ohne Wirkung.
SVC-SNMP-PUBLIC	default_communities	SNMP-Community-Namen, die als unsicher gelten (ohne Beachtung der Groß-/Kleinschreibung). Standard: public, private.
FW-ANY-ANY	wan_zone_severity	Schweregrad einer any→any-Regel auf einem WAN-Interface (critical, high, medium, low, info). Standard: high.

Bei diesen drei Parametern prüft der Editor die Eingabe schon beim „Übernehmen“. Ungültige Werte werden abgewiesen:

Parameter (JSON, optional)

1

```
{"sensitive_ports": ["abc", 22]}
```

Wirksam: sensitive_ports - Portnummern (1-65535), deren Weiterleitung vom WAN als sensibel gilt.

2

sensitive_ports: Liste ganzer Zahlen zwischen 1 und 65535 erwartet, z. B. [22, 3389].

Abbildung 24: Abgewiesener Parameterwert: „abc“ ist keine Portnummer.

- 1 **Parameterfeld** ist rot umrandet.
- 2 **Meldung** unter dem Feld: *sensitive_ports: Liste ganzer Zahlen zwischen 1 und 65535 erwartet, z. B. [22, 3389].*

Die Engine selbst fällt bei einem fehlenden oder ungültigen Wert still auf den Standard zurück. Zusätzlich kannst du **eigene Parameter** in beliebigen Regeln anlegen und in deren Prüfung mit `"@param:name"` verwenden (Kapitel 18.10). Für solche eigenen Parameter gibt es keine Wertprüfung im Formular; der Probelauf zeigt, ob sie wie gedacht wirken.

15 Neue Regeln anlegen

Neue Regel

Die Regel wird am Ende ihrer Kategorie eingefügt.

1

Ohne Prüfung erscheint die Regel im Bericht mit Status MANUELL und zählt nicht in den Score. Mit einer Prüfung (Bereich „Prüfung“ unten, am einfachsten über eine Vorlage) wird sie beim nächsten Audit sofort ausgewertet.

2

ID

FW-WAN-SSH-EIGEN

Großbuchstaben, Ziffern und Bindestrich, z. B. SVC-DOT-MISSING. Muss eindeutig sein.

Schweregrad

Mittel (Gewicht 3)

3

Titel

SSH per WAN-Regel freigegeben (eigene Regel)

Kategorie

4

Firewall-Regeln

Vorhandene Kategorie wählen oder eine neue eintippen.

5

☒ Regel aktiv (deaktivierte Regeln werden nicht geprüft)

Abbildung 25: Formular einer neuen Regel, noch ohne Prüfung.

- Hinweis:** Ohne Prüfung erscheint die Regel im Bericht als MANUELL und zählt nicht in den Score.
- ID.** Frei wählbar. Die mitgelieferten Regeln folgen der Konvention `KATEGORIE-THEMA` (MGMT, FW, NAT, SVC, VPN, SYS).
- Schweregrad** (voreingestellt: Mittel).
- Kategorie.** Die neue Regel wird am Ende dieser Kategorie eingefügt. Bei einer neuen Kategorie steht sie am Ende der Liste.
- Regel aktiv** (voreingestellt: an).

- 1** **Neue Regel** anklicken. Das Formular ist leer, die Schaltfläche heißt jetzt **Regel anlegen**.
- 2** ID, Titel, Kategorie, Schweregrad sowie Begründung und Maßnahme eintragen. Titel und Kategorie sind Pflicht.
- 3** Im Bereich **Prüfung** über **Vorlage einfügen ...** einen Ausgangspunkt wählen und anpassen (Kapitel 17). Ohne Prüfung bleibt die Regel eine reine Erinnerung (MANUELL).
- 4** **Test-Config laden ...** und **Probelauf** ausführen. Prüfe zwei Fälle: eine Konfiguration, die den Befund auslösen *soll*, und eine, die *sauber* ist.
- 5** **Regel anlegen** klicken. Die Regel erscheint mit der Marke „neu“ in der Liste; das Formular bleibt geöffnet und die Schaltfläche heißt nun **Übernehmen**.

Eine neue Regel ist auch dann sinnvoll, wenn du eine vorhandene **abwandeln** willst: Kopiere im bestehenden Formular das Prüffeld, lege eine neue Regel an, füge es ein und passe es an. Lege die neue Regel *zusätzlich* an und deaktiviere ggf. die alte.

15.1 Regeln löschen

Über **Regel löschen** im Formular. Die Regel steht danach im Speicher-Dialog unter „Entfernt“ und im Änderungsprotokoll. Möchtest du sie nur vorübergehend ausschalten, deaktiviere sie stattdessen (Kapitel 14.3).

16 Katalog-Einstellungen

Über **Katalog-Einstellungen** in der Werkzeugleiste erreichst du Einstellungen, die für **alle** Regeln gelten.

Katalog-Einstellungen

Name und Beschreibung erscheinen im Bericht. Gewichte und Strafpunktfaktoren bestimmen, wie stark ein Befund in den Posture-Score eingeht; die WAN-Interfaces legen fest, was als Internetseite gilt.

Katalogname

OPNsense Security Baseline

Beschreibung

Quantifiziertes Prüfschema fuer OPNsense-Konfigurationen (config.xml). Jede Regel besitzt einen Schweregrad und ein Gewicht; daraus berechnet die Engine einen normierten Posture-Score (0-100) und eine Note (A-F).

Gewichte je Schweregrad

Kritisch	Hoch	Mittel	Niedrig	Info
10	6	3	1	0

WAN-Interfaces

wan

Interface-Namen aus der config.xml, die als Internetseite gelten, durch Komma getrennt (z. B. wan, opt2 bei zwei Providern). Wirkt auf alle WAN-bezogenen Prüfungen.

Strafpunktfaktor je Status (Anteil des Gewichts)

FAIL	WARN	OK
1	0.5	0

Bisherige Versionen

- 1.2.0** 2026-09-29 – Prüfungen als Daten: 28 Regeln mit Prüfdefinition im Feld 'check', 4 als Baustein. Neu: FW-SHADOWED-BLOCK, FW-SHADOWED-PASS (Regel-Reihenfolge). Korrektur SVC-DNS-OPENRESOLVER: Interface-Liste mit mehreren Kommas wird vollständig ausgewertet.
- 1.1.0** 2026-09-29 – Parameter werden von engine.js ausgewertet: meta.wan_interfaces (neu), meta.scoring (Strafpunktfaktoren), params von NAT-MGMT-EXPOSED, SVC-SNMP-PUBLIC und FW-ANY-ANY. Werte unverändert, Bewertung identisch zu 1.0.0.

Übernehmen

Abbildung 26: Katalog-Einstellungen.

- Katalogname.** Erscheint in der Kopfzeile von `index.html` und im Bericht.
- Beschreibung** des Katalogs.
- Gewichte je Schweregrad** (0 bis 100). Bestimmen, wie stark ein Befund in den Score eingeht. Standard: 10 / 6 / 3 / 1 / 0.
- WAN-Interfaces.** Namen der Interfaces, die als Internetseite gelten, durch Komma getrennt (Standard: `wan`). Verwende die **internen Bezeichner** aus der `config.xml` (Bereich `<interfaces>`): meist `wan`, `lan`, `opt1`, `opt2` ... Bei zwei Providern z. B. `wan`, `opt2`.
- Strafpunktfaktor je Status** (0 bis 1): Anteil des Gewichts, der bei FAIL, WARN und OK als Strafpunkte zählt. Standard: 1 / 0,5 / 0.

- 6 Bisherige Versionen.** Das Änderungsprotokoll des Katalogs, neueste Version oben (nur sichtbar, wenn der Katalog bereits eines enthält).
- 7 Übernehmen** schreibt die Einstellungen in die Arbeitskopie.

Wirkung der WAN-Interfaces: Alle WAN-bezogenen Prüfungen (WAN-Freigaben, NAT-Weiterleitungen, „Block private/bogon networks“, NTP, WebGUI/SSH von außen) richten sich nach dieser Liste. Ein Interface, das hier fehlt, gilt für sie als internes Netz.

Grenzen der Eingabe: Gewichte müssen Zahlen ≥ 0 sein, Strafpunktfaktoren zwischen 0 und 1 liegen und mindestens ein WAN-Interface angegeben sein. Sonst erscheint eine Meldung, und nichts wird übernommen (Kapitel 22).

Score-Vergleiche Ändere Gewichte oder Strafpunktfaktoren nur bewusst und halte fest, dass Scores **nur innerhalb derselben Einstellungen** vergleichbar sind. Ein Score von 80 mit den Standardgewichten ist etwas anderes als 80 mit veränderten Gewichten.

17 Prüfungen bearbeiten und ausprobieren

Jede Regel enthält ihre **Prüfung** als JSON: Sie legt fest, was in der `config.xml` gesucht wird und welches Ergebnis (Status, Befundtext) daraus folgt. Der Bereich „Prüfung“ steht unter dem Formular.

Prüfung

1 Prüfung (JSON)

```

{
  "quelle": "filterregeln",
  "wo": {
    "und": [
      {
        "aktiv": true,
        "aktion": "pass",
        "ist_wan": true
      },
      {
        "feld": "ziel_port",
        "port_ist": [
          "123"
        ]
      }
    ]
  }
}

```

Leer = keine automatische Prüfung (Status MANUELL). Aufbau: Sprachreferenz unten; als Ausgangspunkt eine Vorlage einfügen.

2

Vorlage einfügen ...

3

4

5

Test-Config laden ...

6

7

WARN

Niedrig · OPNsense · opn-schwach.xml · 1 Treffer

- NTP (UDP/123) per WAN erreichbar.

8

Sprachreferenz: Quellen, Felder, Operatoren

9

Übernehmen

Änderungen verwerfen

Regel löschen

Abbildung 27: Bereich „Prüfung“ mit Ergebnis eines Probelaufs (Beispiel: SVC-NTP-OPEN).

- Prüfung (JSON).** Das Textfeld mit der Prüfdefinition. Leer bedeutet: keine automatische Prüfung (MANUELL).
- Vorlage einfügen ...** setzt einen Ausgangspunkt ein. Ist das Feld nicht leer, fragt der Editor vorher: *Die vorhandene Prüfung durch die Vorlage ersetzen?*
- Aufbau prüfen** kontrolliert die Struktur der Definition.
- Probelauf** führt die Prüfung gegen die geladene Test-Config aus.
- Test-Config laden ...** wählt eine `config.xml` für den Probelauf.
- Anzeige der Test-Config.** „keine Test-Config geladen“ oder der Dateiname.
- Ergebnisfeld** mit Status, wirksamem Schweregrad, Plattform, Dateiname, Trefferzahl und den Befunden.
- Sprachreferenz** (aufklappbar): alle Quellen, Felder und Operatoren.
- Übernehmen** schreibt die Prüfung in die Arbeitskopie, wenn der Aufbau stimmt.

17.1 Vorlagen

Vorlage	Wofür
---------	-------

Benutzerhandbuch · Firewall-Validation Portable · © 2026 Christian Jelitte · GPL-3.0-or-later

Seite 39 von 86

Wert oder Schalter in der Config prüfen	Eine Einstellung lesen (Text oder An/Aus) und je nach Wert einen Status vergeben. Mit getrennten Pfaden für OPNsense und pfSense.
Firewall-Regeln filtern (z. B. Port auf WAN)	Alle Firewall-Regeln durchgehen und die treffen, die Bedingungen erfüllen (Aktion, Interface, Port, Logging ...).
NAT-Weiterleitungen filtern	Wie oben, für Port-Forwards (Quelle, Ports, Ziel).
XML-Liste durchlaufen (Einträge einzeln prüfen)	Beliebige Listen der Config prüfen (VPN-Instanzen, Zertifikate ...), getrennt je Plattform.
Reihenfolge von Firewall-Regeln prüfen	Paare „Regel A steht vor Regel B“ finden.
Baustein (Prüfung in Programmcode) aufrufen	Eine der vier fest programmierten Prüfungen unter eigenem Titel und Schweregrad einsetzen.

Die Vorlagen sind bewusst **Platzhalter**: Pfade und Werte darin (z. B. `OPNsense/Modul/general/einstellung`) musst du durch echte ersetzen.

17.2 „Aufbau prüfen“ – was es findet und was nicht

Prüfung (JSON)

```

{
  "quelle": "filterregeln",
  "wo": {
    "feld": "ziel_port"
  },
  "ergebnis": [
    {
      "wenn": "treffen"
    }
  ]
}

```

Leer = keine automatische Prüfung (Status MANUELL). Aufbau: Sprachreferenz unten; als Ausgangspunkt eine Vorlage einfügen.

Vorlage einfügen ...

Aufbau prüfen

Probelauf

Test-Config laden ...

keine Test-Config geladen

Aufbaufehler:

- Unbekannter Schluessel 'tippfehler' fuer Typ abfrage.
- Abfrage → wo: 'feld' braucht genau einen Operator (ist, ist_eine_von, leer, enthaelt_eins_von, liste_enthaelt, groesser, kleiner, port_ist, ports_schneiden).
- ergebnis[1]: 'status' muss einer von FAIL, WARN, PASS, NA, MANUAL sein.
- Der letzte Fall in 'ergebnis' sollte ohne 'wenn' sein (Standardfall).

Abbildung 28: Aufbauprüfung mit mehreren Fehlern.

- Prüffeld** mit absichtlich fehlerhafter Definition.
- Aufbau prüfen** wurde geklickt.
- Fehlerliste** („Aufbaufehler:“), rot hinterlegt, mit einer Zeile je Fehler und Angabe der Stelle.

Die Aufbauprüfung findet **Strukturfehler**: kein gültiges JSON, unbekannte Schlüssel, unbekannte Typen, Quellen und Bausteine, ein Operator ohne `feld` (oder umgekehrt), ein ungültiger Status oder Schweregrad,

ein fehlender Standardfall am Ende von `ergebnis`, ein `@param:`-Verweis auf einen Parameter, den die Regel nicht definiert.

Was sie nicht findet: Tippfehler in **Feldnamen** innerhalb einer Bedingung (z. B. `"aktoin": "pass"`) und **falsche XML-Pfade**. Solche Bedingungen sind gültig, treffen aber nie zu; die Prüfung liefert dann immer „OK“ oder „N/A“. Ob eine Prüfung wirklich tut, was sie soll, zeigt nur der **Probelauf** – am besten mit zwei Configs: eine, die den Befund auslösen soll, und eine, die sauber ist.

17.3 Der Probelauf

Vorlage einfügen ... **4** **Aufbau prüfen** **Probelauf**

Test-Config laden ... Test-Config: pf-reihenfolge.xml

WARN Niedrig · pfSense · pf-reihenfolge.xml · 2 Treffer

- #1 [opt2/in] 'Allow HTTP TCP 80' wird nie erreicht - verdeckt durch #0 block 'Block All default deny'
- #2 [opt2/in] 'Allow NTP UDP 123' wird nie erreicht - verdeckt durch #0 block 'Block All default deny'

Notiz: Freigabe wirkt nicht: eine vorangehende Regel faengt den gesamten Verkehr ab.

Abbildung 29: Ergebnis eines Probelaufs gegen pf-reihenfolge.xml (Regel FW-SHADOWED-PASS).

- 1 Kopfzeile des Ergebnisses:** Status als farbiges Etikett, dahinter wirksamer Schweregrad, Plattform, Dateiname und die Zahl der Treffer.
- 2 Befunde:** je Treffer eine Zeile, so wie sie im Bericht stünden.
- 3 Notiz** der Regel (falls vorhanden).
- 4 Test-Config:** Name der geladenen Datei. Sie bleibt für alle weiteren Probelläufe geladen, bis du die Seite neu lädst.

- 1** **Test-Config laden ...** und eine `config.xml` wählen (auch die Beispiele aus `tests/configs/`). Klickst du **Probelauf** ohne geladene Datei, öffnet sich der Dialog automatisch.
- 2** **Probelauf** klicken. Ausgeführt wird der **aktuelle Inhalt des Formulars** (Titel, Schweregrad, Parameter, Prüfung), auch wenn er noch nicht übernommen wurde. Nichts wird gespeichert.
- 3** Ergebnis lesen, Prüfung anpassen, erneut ausführen. Bei Aufbaufehlern erscheint statt eines Ergebnisses die Fehlerliste.

Ist die Datei keine OPNsense-/pfSense-Konfiguration, meldet der Probelauf „Keine OPNsense- oder pfSense-config.xml.“ Zur Bedeutung der Status siehe Kapitel 6.1; bei Probelläufen ohne Trefferzahl (z. B. bei Bausteinen) fehlt die Angabe „N Treffer“.

17.4 Die Sprachreferenz

1 ▼ Sprachreferenz: Quellen, Felder, Operatoren

Typen: "abfrage" (Standard: Werte lesen, Einträge filtern, Fälle auswerten), "reihenfolge" (Regel A steht vor Regel B), "baustein" (Prüfung in Programmcode: MGMT-WEBGUI-WAN, MGMT-SSH-WAN, MGMT-ROOT-ACCOUNT, FW-HOST-CIDR). Ergebnis-Fälle werden von oben nach unten geprüft, der erste passende gilt. Textvorlagen: {feld}, {feld|Ersatz}, {feld:klein}, {?feld=Text}, {liste}; XML-Pfade: a/b/c, a/* (alle Kinder), //name (irgendwo). Parameter der Regel: "@param:name".

2 Quelle „filterregeln“ - Firewall-Regeln unter Firewall → Rules, in der Reihenfolge der config.xml (Floating-Regeln eingeschlossen).

nr	laufende Nummer ueber alle Regeln (wie im Bericht #n)
pos	Position innerhalb des Interfaces (1, 2, ...)
interface	Interface-Name (wan, lan, opt1 ...)
aktion	pass, block oder reject
richtung	in oder out (leer = in)
protokoll	tcp, udp, any ...
ip_version	inet, inet6 oder inet46
aktiv	true, wenn nicht deaktiviert
log	Logging an
quick	quick gesetzt
floating	Floating-Regel
3 zusatzbedingung	Zeitplan, tagged oder OS-Filter gesetzt
descr	Beschreibung
quelle_any	Quelle any
quelle_negiert	Quelle mit 'not'
ziel_any	Ziel any
ziel_negiert	Ziel mit 'not'

Abbildung 30: Aufgeklappte Sprachreferenz.

1 **Kopfzeile** zum Auf- und Zuklappen. Darunter steht eine Kurzfassung der Typen, Textvorlagen und Pfadschreibweisen.

2 **Quelle** mit Beschreibung (hier `filterregeln`).

3 **Feldliste**. Links der Feldname, wie er in Bedingungen und Vorlagen verwendet wird, rechts die Bedeutung.

Die Referenz wird direkt aus `engine.js` erzeugt und ist deshalb immer aktuell. Eine Kopie steht im Anhang **Anhang B**.

18 Die Regelsprache

Dieses Kapitel erklärt, wie eine Prüfdefinition aufgebaut ist. Die Sprache ist bewusst klein: Sie kann lesen, filtern, vergleichen und ein Ergebnis wählen. Sie hat **keine Variablen, Schleifen oder Berechnungen**. Was darüber hinausgeht, erledigt ein Baustein in `engine.js`.

18.1 Aufbau und Ablauf



Abbildung 31: Aufbau einer Prüfdefinition (Typ „abfrage“) und ihr Ablauf.

Eine Prüfdefinition ist ein JSON-Objekt mit einem `typ` :

typ	Zweck	Erlaubte Schlüssel
abfrage (Standard, darf fehlen)	Werte lesen und/oder Einträge filtern, dann ein Ergebnis wählen.	werte, quelle, xml, wo, vorlage, abfragen, sortierung, trenner, ergebnis
reihenfolge	Paare finden, bei denen eine Regel A vor einer Regel B steht.	werte, quelle, gleich, umfasst, vorher, nachher, vorlage, trenner, ergebnis
baustein	Eine fest programmierte Prüfung aufrufen.	name

Ablauf einer Abfrage:

- 1 Werte lesen** (`werte`): benannte Angaben aus der Config, einmal je Prüfung.
- 2 Einträge sammeln**: aus einer Faktenquelle (`quelle`) oder einer XML-Liste (`xml`). Mit `abfragen` lassen sich mehrere Quellen kombinieren.
- 3 Filtern** (`wo`): Einträge, die die Bedingung erfüllen, sind die **Treffer**. Je Treffer entsteht mit `vorlage` ein Text.
- 4 Sortieren** (optional, `sortierung`): bestimmte Treffer zuerst.
- 5 Ergebnis wählen** (`ergebnis`): Die Fälle werden **von oben nach unten** geprüft; der **erste passende** gilt und liefert Status, Befund und Notiz.

18.2 Werte: Angaben aus der Config lesen

Mit `werte` gibst du Angaben einen Namen, um sie später in Bedingungen und Texten zu verwenden. Es gibt drei Arten (pro Wert genau eine):

Art	Bedeutung	Beispiel
-----	-----------	----------

<code>pfad</code>	Text an diesem Pfad. Eine Liste von Pfaden nimmt den ersten nicht leeren. <code>standard</code> gilt, wenn nichts gefunden wird; <code>klein: true</code> wandelt in Kleinbuchstaben.	<code>{"pfad": "system/webgui/protocol", "standard": "https", "klein": true}</code>
<code>anzahl</code>	Zahl der Elemente an diesem Pfad (auch eine Liste von Pfaden, dann Summe).	<code>{"anzahl": "OPNsense/Syslog/destinations/destination"}</code>
<code>verbinde</code>	Fügt die Texte mehrerer Pfade (nur nicht leere) mit <code>trenner</code> zusammen (Standard: <code>„,“</code>).	<code>{"verbinde": ["syslog/remoteserver", "syslog/remoteserver2"]}</code>

18.3 XML-Pfade und Plattformen

Schreibweise	Bedeutung
<code>a/b/c</code>	Kette von Unterelementen ab der Wurzel der <code>config.xml</code> (oder, bei XML-Listen, ab dem aktuellen Eintrag).
<code>a/*</code>	Alle direkten Unterelemente von <code>a</code> .
<code>//name</code> oder <code>//name/x</code>	Das erste Element <code>name</code> an beliebiger Stelle der Config, danach ggf. weiter mit <code>/x</code> .
<code>{"opnsense": "...", "pfsense": "..."} </code>	Verschiedene Pfade je Plattform. Fehlt die eigene Plattform, gilt der Pfad als „nicht vorhanden“. Erlaubt bei <code>pfad</code> , <code>anzahl</code> , <code>verbinde</code> , <code>xml</code> , <code>eingeschaltet</code> und <code>existiert</code> .

Wie du den richtigen Pfad findest Öffne die exportierte `config.xml` in einem Texteditor, suche nach dem Namen der Einstellung oder einem Wert, den du in der Weboberfläche gesetzt hast, und lies die Elementkette von außen nach innen ab. Wichtig: Der Name des Wurzelements (`<opnsense>` / `<pfsense>`) gehört **nicht** in den Pfad. Achte auf Groß-/Kleinschreibung: Elementnamen wie `OPNsense` (Modulbereich) und `opnsense` (Wurzel) unterscheiden sich.

18.4 Faktenquellen

Statt XML direkt zu lesen, bietet die Engine **Faktenquellen**: Sie übersetzen die Config für OPNsense und pfSense in eine **einheitliche** Form. Eine Regel, die `filterregeln` verwendet, funktioniert daher auf beiden Systemen.

Quelle	Einträge
<code>filterregeln</code>	Firewall-Regeln (<i>Firewall</i> → <i>Rules</i>), in Reihenfolge der Config, inklusive Floating-Regeln.
<code>nat</code>	Port-Weiterleitungen (<i>Firewall</i> → <i>NAT</i> → <i>Port Forward</i>).
<code>benutzer</code>	Lokale Benutzerkonten.
<code>interfaces</code>	Alle Interfaces aus <code><interfaces></code> .
<code>wan_interfaces</code>	Nur die als WAN eingestellten Interfaces, die es in der Config gibt.
<code>ids_instanzen</code>	pfSense: Instanzen des IDS-Pakets (Suricata/Snort).

Die Felder jeder Quelle (z. B. `aktion`, `ist_wan`, `ziel_port`) stehen in der Sprachreferenz im Editor und im Anhang **Anhang B**. Mit `"xml": "pfad"` kannst du stattdessen eine **beliebige Liste** der Config durchlaufen; dann sind die Felder die **direkten Unterelemente** des jeweiligen Eintrags (z. B. `description`, `version`).

Zusätzlich gibt es **allgemeine Felder**, die überall verfügbar sind: `plattform` (opnsense/pfsense), `ssh_aktiv`, `ids_paket`, `wg_vorhanden`, `wg_tunnel`, `wg_peers`, `wg_gesamt`.

18.5 Bedingungen

Bedingungen stehen in `wo` (Filter je Eintrag) und in `wenn` (Auswahl eines Ergebnis-Falls). Sie sind JSON-Objekte; sämtliche Angaben in einem Objekt müssen zugleich zutreffen (UND).

Schreibweise	Bedeutung
<code>{"aktion": "pass"}</code>	Feld gleich Wert. Texte werden ohne Beachtung von Groß-/Kleinschreibung verglichen; Wahrheitswerte als <code>true</code> / <code>false</code> ; Zahlen als Zahl.
<code>{"aktion": ["block", "reject"]}</code>	Feld gleich einem der Werte (ODER innerhalb der Liste).
<code>{"aktiv": true, "ist_wan": true}</code>	Beide Bedingungen (UND).
<code>{"feld": "ziel_port", "port_ist": ["123"]}</code>	Feld mit Operator (genau einer je Objekt). Operatoren: siehe unten.
<code>{"und": [...]}, {"oder": [...]}, {"nicht": { ... }}</code>	Verknüpfungen. <code>und</code> und <code>oder</code> erwarten eine Liste von Bedingungen.
<code>{"eingeschaltet": "system/ssh/enabled"}</code>	Der Schalter am Pfad ist an. Als an gelten <code>1</code> , <code>yes</code> , <code>on</code> , <code>true</code> , <code>enabled</code> und ein leeres Element (so schreibt pfSense viele Häkchen). Fehlt das Element, gilt es als aus.
<code>{"existiert": "snmpd"}</code>	Das Element am Pfad ist vorhanden.
<code>{"eine": {quelle, wo}}, {"keine": {quelle, wo}}</code>	Eine Unterabfrage liefert mindestens einen (<code>eine</code>) bzw. keinen (<code>keine</code>) Treffer.
<code>{"treffer_mit": { ... }}</code>	Mindestens ein Treffer der Hauptabfrage erfüllt die Bedingung (nur in <code>wenn</code>).
<code>"treffer", "keine_treffer", "quellen_leer"</code>	Als ganzer Wert von <code>wenn</code> : Es gibt Treffer / keine Treffer / die Quelle hatte gar keine Einträge.

Operatoren (mit `"feld"`)

Operator	Bedeutung
<code>ist</code>	gleich (Text ohne Gross/klein, true/false, Zahl)
<code>ist_eine_von</code>	gleich einem Wert der Liste
<code>leer</code>	true: Feld ist leer; false: Feld hat Inhalt
<code>enthaelt_eins_von</code>	Text enthaelt einen der Teiltexthe
<code>liste_enthaelt</code>	durch Komma/Leerzeichen getrennte Liste enthaelt einen der Werte

<code>groesser</code>	Zahl groesser
<code>kleiner</code>	Zahl kleiner
<code>port_ist</code>	Port bzw. aufgeloster Port-Alias enthaelt einen der Ports
<code>ports_schneiden</code>	Ports (mit Bereichen und Aliassen) ueberschneiden sich mit der Liste; Treffer als {treffer_ports}

Woher kommen die Feldnamen? In `wo` sind es die Felder des gerade geprüften Eintrags. In `wenn` (ohne Eintrag) sind es die benannten `werte`, die allgemeinen Felder und die von der Abfrage gemerkten Sonderwerte (etwa `treffer_ports`). Bei `xml`-Listen ist ein Feld das gleichnamige Unterelement des Eintrags; `eingeschaltet` und `existiert` beziehen sich dort ebenfalls auf den Eintrag. Ein **unbekannter Feldname** liefert leer und führt zu keinem Fehler (siehe Warnung in Kapitel 17.2).

18.6 Ergebnis-Fälle

`ergebnis` ist eine Liste von Fällen. Der **erste** Fall, dessen `wenn` zutrifft, bestimmt das Ergebnis. Ein Fall ohne `wenn` trifft immer zu und gehört als **Standardfall ans Ende**; fehlt er und keiner passt, entsteht der Status FEHLER.

Schlüssel	Bedeutung
<code>wenn</code>	Bedingung (siehe oben). Fehlt sie, trifft der Fall immer zu.
<code>status</code>	Pflicht: <code>FAIL</code> , <code>WARN</code> , <code>PASS</code> , <code>NA</code> oder <code>MANUAL</code> .
<code>schweregrad</code>	Optional: übersteuert den Katalogwert für diesen Fund (<code>critical</code> , <code>high</code> , <code>medium</code> , <code>low</code> , <code>info</code> oder <code>"@param:name"</code>).
<code>befund</code>	Ein Befundtext (Textvorlage).
<code>befunde</code>	Nur <code>"je_treffer"</code> : ein Befund pro Treffer, aus der <code>vorlage</code> der Abfrage.
<code>notiz</code>	Kurze Erläuterung, wird im Bericht unter dem Status angezeigt.

Beispiel: *Bei Treffern WARN mit einem Befund je Treffer, sonst OK:*

```
"ergebnis": [
  {"wenn": "treffer", "status": "WARN", "befunde": "je_treffer", "notiz": "Kurze
  Erläuterung."},
  {"status": "PASS"}
]
```

Reihenfolge der Fälle beachten Lege zuerst die Sonderfälle an (N/A, weil ein Dienst gar nicht vorhanden ist), dann die Befunde von schwer nach leicht, zuletzt den Standardfall. So liest sich die Prüfung von oben nach unten wie eine Entscheidungsliste.

Die Fälle einer Prüfung sind gleichzeitig die Möglichkeiten, wie der Schweregrad im Bericht vom Katalogwert abweichen kann. Deshalb steht dort, wo `schweregrad` gesetzt ist, der **wirksame** Wert (Kapitel 6.2).

18.7 Textvorlagen für Befunde

Befund- und Notiztexte sowie die `vorlage` einer Abfrage dürfen Platzhalter in geschweiften Klammern enthalten:

Platzhalter	Ergebnis
<code>{feld}</code>	Wert des Feldes (leer, wenn nicht vorhanden).
<code>{feld Ersatz}</code>	Wert, bei leerem Feld der Ersatztext.
<code>{feld:klein}</code> , <code>{feld:gross}</code>	Wert in Klein- bzw. Großbuchstaben. Kombinierbar: <code>{feld:klein Ersatz}</code> .
<code>{?feld=Text}</code>	Gibt <code>Text</code> nur aus, wenn das Feld wahr, eine Zahl ungleich 0 oder ein nicht leerer Text (außer „0“) ist.
<code>{liste}</code>	Alle Treffertexte, durch Komma getrennt (Trennzeichen mit <code>trenner</code> änderbar).
<code>{treffer_ports}</code>	Die durch <code>ports_schneiden</code> gefundenen Ports.
<code>{vorher.nr}</code> , <code>{nachher.descr}</code>	Bei Reihenfolge-Prüfungen: Felder der früheren (<code>vorher</code>) bzw. späteren (<code>nachher</code>) Regel. Punktschreibweise.
<code>{source/address}</code>	Bei XML-Listen: Unterelement-Kette des Eintrags.

Beispiel: `"#{nr} [{interface}] {quelle_anzeige} -> {ziel_text}{?quelle_any= [Quelle ANY]} - '{descr}'"` ergibt zum Beispiel `#1 [wan] ANY → wanip:80 [Quelle ANY] - 'GUI von aussen'`.

18.8 Reihenfolge-Prüfungen (typ „reihenfolge“)

Dieser Typ findet Regeln, die eine **frühere** Regel verdeckt. Er durchläuft die Einträge der Quelle in Reihenfolge und sucht für jeden Kandidaten (`nachher`) eine **frühere** Regel (`vorher`), die zu ihm passt:

Schlüssel	Bedeutung
<code>quelle</code>	Faktenquelle, üblicherweise <code>filterregeln</code> .
<code>gleich</code>	Liste von Feldern, die bei beiden Regeln gleich sein müssen, z. B. <code>["interface", "richtung"]</code> . Sie bilden die „Gruppe“, in der die Auswertungsreihenfolge gilt.
<code>umfasst</code>	Derzeit nur <code>["ip_version"]</code> : Die frühere Regel deckt die spätere ab, wenn beide dieselbe IP-Version haben oder die frühere <code>inet46</code> (IPv4+IPv6) ist.
<code>vorher</code>	Bedingung an die frühere Regel (bei den mitgelieferten Regeln: „Auffang-Regel“).
<code>nachher</code>	Bedingung an die spätere Regel (z. B. Aktion Block).
<code>vorlage</code>	Befundtext mit <code>{vorher....}</code> und <code>{nachher....}</code> .

Je Kandidat wird die **erste** passende frühere Regel gemeldet. So sieht die mitgelieferte Prüfung `FW-SHADOWED-BLOCK` aus:

```

{
  "typ": "reihenfolge",
  "quelle": "filterregeln",
  "gleich": ["interface", "richtung"],
  "umfasst": ["ip_version"],
  "vorher": {
    "aktiv": true,
    "floating": false,
    "zusatzbedingung": false,
    "protokoll": "any",
    "quelle_any": true,
    "quelle_negiert": false,
    "ziel_any": true,
    "ziel_negiert": false,
    "ziel_port": ""
  },
  "nachher": {"aktiv": true, "floating": false, "aktion": ["block", "reject"]},
  "vorlage": "#{nachher.nr} [{interface}/{richtung}] '{nachher.descr}' wird nie
erreicht - verdeckt durch #{vorher.nr} {vorher.aktion} '{vorher.descr}'",
  "ergebnis": [
    {
      "wenn": "treffer",
      "status": "WARN",
      "befunde": "je_treffer",
      "notiz": "Block-Regel wirkt nicht: eine vorangehende Regel faengt den gesamten
Verkehr ab."
    },
    {"status": "PASS"}
  ]
}

```

Gelesen: Suche in den Firewall-Regeln nach einer Block-Regel („nachher“), vor der auf demselben Interface und in derselben Richtung („gleich“), für dieselbe IP-Version („umfasst“), eine aktive Auffang-Regel („vorher“) steht. Melde jeden solchen Fund als Befund; gibt es keinen, ist alles in Ordnung.

18.9 Bausteine

Vier Prüfungen sind zu komplex für die Regelsprache und stecken als Programmcode in `engine.js`. Du rufst sie mit `{"typ": "baustein", "name": "..."} auf:`

Baustein	Was er prüft
MGMT-WEBGUI-WAN	WebGUI vom WAN erreichbar: Bindung der Oberfläche an Interfaces <i>und</i> WAN-Regeln auf den (aus der Config abgeleiteten) GUI-Port.
MGMT-SSH-WAN	Dasselbe für SSH, mit dem konfigurierten SSH-Port.
MGMT-ROOT-ACCOUNT	root-Konto aktiv, gegenüber benannten Admin-Konten (verknüpft Benutzer und Gruppen).
FW-HOST-CIDR	Einzelhost mit Subnetzmaske in Regeln, NAT und Aliassen (mit Alias-Auflösung).

Ein Baustein hat immer sein festes Ergebnis; du kannst nur Titel, Schweregrad und Texte der Regel ändern, die ihn aufruft.

18.10 Parameter (@param)

Statt Werte fest in die Prüfung zu schreiben, kannst du sie als **Parameter der Regel** hinterlegen und mit `"@param:name"` verweisen. Der Vorteil: Portlisten, Communities oder Schweregrade lassen sich im Formular ändern, ohne die Prüfung anzufassen.

```
Parameter (JSON):      {"ports": ["22", "2222"]}
Prüfung:               ... {"feld": "ziel_port", "port_list": ["@param:ports"]} ...
```

Ein Verweis in einer Liste wird durch die Werte des Parameters ersetzt (`["@param:ports", "8443"]` ergibt 22, 2222, 8443). Verweist die Prüfung auf einen Parameter, den die Regel nicht definiert, meldet die Aufbauprüfung einen Fehler.

18.11 Plattform-abhängige Prüfungen

OPNsense und pfSense speichern manche Einstellungen an verschiedenen Stellen. Dafür gibt es drei Mittel:

- **Pfade je Plattform** (Kapitel 18.3): `{"opnsense": "OPNsense/IDS/general/enabled", "pfsense": "installedpackages/suricata/..."}`.
- **Getrennte Abfragen** in `abfragen`, jede mit `"plattform": "opnsense"` oder `"pfsense"`. Eine Abfrage für die andere Plattform liefert keine Einträge.
- **Plattform im Ergebnis-Fall**: `{"wenn": {"plattform": "pfsense"}, "status": "NA", ...}`.

19 Praxisbeispiele

Einen vollständigen Durchgang mit allen Schritten und Abbildungen, vom Anlegen bis zum Bericht, zeigt Kapitel 21 am Beispiel „Block-All-Regel muss am Ende stehen“. Alle folgenden Beispiele wurden mit der Engine ausgeführt; die Ergebnisse sind die tatsächlichen Ausgaben des Probelaufs. Du kannst sie in ein neues Regelformular übernehmen (Prüfung und, wo angegeben, Parameter), die genannte Test-Config laden und **Probelauf** klicken.

19.1 Beispiel 1: Schweregrad einer Regel ändern

Ziel: Ein fehlendes Remote-Syslog soll dir wichtiger sein als bisher (Standard: niedrig).

- 1 Regel `SYS-SYSLOG-REMOTE` in der Liste wählen.
- 2 Schweregrad auf „Mittel“ stellen, **Übernehmen**.
- 3 Katalog speichern und einsetzen (Kapitel 20).

Wirkung: Das Gewicht der Regel steigt von 1 auf 3. Bei Geräten *ohne* Remote-Syslog sinkt der Score stärker; bei Geräten *mit* Remote-Syslog sinkt er ebenfalls leicht, weil das Gesamtgewicht im Nenner wächst.

19.2 Beispiel 2: Eine Regel ausschalten, die bei dir nicht zutrifft

Ziel: Du betreibst bewusst kein IDS; die Regel `SVC-IDS-DISABLED` soll den Score nicht mehr drücken.

- 1 Regel wählen, „Regel aktiv“ abwählen, **Übernehmen**.
- 2 Speichern und einsetzen.

Wirkung: Die Regel erscheint weder im Bericht noch im Score. Dokumentiere die Entscheidung im Änderungsprotokoll (Notiz im Speicher-Dialog), damit sie später nachvollziehbar ist.

19.3 Beispiel 3: Ein zweiter Internetanschluss

Ziel: Deine Firewall hat zwei Provider (`wan` und `opt2`); beide sollen als Internetseite gelten.

- 1 **Katalog-Einstellungen** öffnen.
- 2 Bei „WAN-Interfaces“ `wan`, `opt2` eintragen, **Übernehmen**.

Wirkung: „Block private/bogon networks“ wird auf beiden Interfaces geprüft, und Freigaben, Weiterleitungen und Dienste auf `opt2` zählen als aus dem Internet erreichbar. Ein Test mit der Beispiel-Config `opn-schwach.xml`: Mit `wan`, `opt1` als WAN-Interfaces meldet `FW-BLOCKPRIV` „fehlt auf: wan, opt1“, weil `opt1` dann mit geprüft wird.

19.4 Beispiel 4: Eine Portnummer als sensibel ergänzen

Ziel: Auch eine Weiterleitung auf Port 8080 soll als kritisch gelten.

- 1 Regel `NAT-MGMT-EXPOSED` wählen.
- 2 Im Feld „Parameter (JSON)“ in der Liste `sensitive_ports` die Zahl `8080` ergänzen.
- 3 **Übernehmen**. Der Editor prüft, dass alle Einträge ganze Zahlen zwischen 1 und 65535 sind.

Prüfung (in das Feld „Prüfung (JSON)“ einfügen):

```

{
  "quelle": "nat",
  "wo": {
    "und": [
      {"ist_wan": true},
      {"feld": "ports", "ports_schneiden": "@param:sensitive_ports"}
    ]
  },
  "vorlage": "WAN-Forward Port(s) {treffer_ports} -> {ziel} (Quelle: {quelle_anzeige})
- '{descr}'",
  "ergebnis": [
    {
      "wenn": {"treffer_mit": {"quelle_any": true}},
      "status": "FAIL",
      "schweregrad": "critical",
      "befunde": "je_treffer"
    },
    {
      "wenn": "treffer",
      "status": "FAIL",
      "schweregrad": "high",
      "befunde": "je_treffer"
    },
    {"status": "PASS"}
  ]
}

```

Parameter (Feld „Parameter (JSON)“): die Liste um `8080` ergänzt:

```

{"sensitive_ports": [22, 23, 135, 139, 445, 1433, 3306, 3389, 5432, 5900, 5901, 6379,
9200, 11211, 27017, 8080]}

```

Ergebnis des Probelaufs (tatsächliche Ausgabe der Engine):

Test-Config	Status	Schwere	Treffer	Befund
opn-schwach.xml, ergänzt um eine Weiterleitung	FAIL	kritisch	2	WAN-Forward Port(s) 3389 → 192.168.1.50 (Quelle: ANY) - 'RDP' WAN-Forward Port(s) 8080 → 192.168.1.60 (Quelle: ANY) - 'Proxy'
opn-schwach.xml	FAIL	kritisch	1	WAN-Forward Port(s) 3389 → 192.168.1.50 (Quelle: ANY) - 'RDP'

Test-Datei nachstellen Für die erste Zeile: opn-schwach.xml, ergänzt um eine Weiterleitung. Ergänze innerhalb von ... folgende Zeile(n) und lade die Datei als Test-Config:

```
<rule><interface>wan</interface><source><any/></source><destination>
<port>8080</port></destination><target>192.168.1.60</target><local-
port>8080</local-port><descr>Proxy</descr></rule>
```

19.5 Beispiel 5: WAN-Freigaben ohne Logging finden

Ziel: Jede Freigabe aus dem Internet soll protokolliert werden. Neue Regel `FW-WAN-PASS-NOLOG`, Schweregrad niedrig, mit der Vorlage „Firewall-Regeln filtern“ als Ausgangspunkt:

Prüfung (in das Feld „Prüfung (JSON)“ einfügen):

```
{
  "quelle": "filterregeln",
  "wo": {"aktiv": true, "aktion": "pass", "ist_wan": true, "log": false},
  "vorlage": "#{nr} [{interface}] {quelle_anzeige} -> {ziel_text} - '{descr}'",
  "ergebnis": [
    {
      "wenn": "treffer",
      "status": "WARN",
      "befunde": "je_treffer",
      "notiz": "Freigaben aus dem Internet sollten protokolliert werden."
    },
    {"status": "PASS"}
  ]
}
```

Ergebnis des Probelaufs (tatsächliche Ausgabe der Engine):

Test-Config	Status	Schwere	Treffer	Befund
opn-schwach.xml	WARN	niedrig	4	#0 [wan] ANY → any - 'any any WAN' #1 [wan] ANY → wanip:80 - 'GUI von aussen' #2 [wan] ANY → wanip:22 - 'SSH von aussen' ... (4 Befunde)

19.6 Beispiel 6: Eine Einstellung auf einen bestimmten Wert prüfen

Ziel: Die WebGUI soll nicht auf dem Standardport laufen. Neue Regel `MGMT-WEBGUI-PORT`, Schweregrad info, mit der Vorlage „Wert oder Schalter“:

Prüfung (in das Feld „Prüfung (JSON)“ einfügen):

```
{
  "werte": {"port": {"pfad": "system/webgui/port", "standard": "443"}},
  "ergebnis": [
    {
      "wenn": {"port": ["443", "80"]},
      "status": "WARN",
      "befund": "WebGUI nutzt den Standardport {port}."
    },
    {"status": "PASS", "notiz": "WebGUI-Port: {port}."}
  ]
}
```

Ergebnis des Probelaufs (tatsächliche Ausgabe der Engine):

Test-Config	Status	Schwere	Treffer	Befund
opn-schwach.xml	WARN	info	–	WebGUI nutzt den Standardport 443.
opn-variante.xml	OK	info	–	WebGUI-Port: 8443.

19.7 Beispiel 7: Eine eigene Portliste als Parameter

Ziel: Du willst prüfen, ob bestimmte Dienste vom WAN erreichbar sind, und die Ports später ohne Eingriff in die Prüfung ändern können. Neue Regel `FW-WAN-PORTLISTE`, Schweregrad mittel:

Prüfung (in das Feld „Prüfung (JSON)“ einfügen):

```
{
  "quelle": "filterregeln",
  "wo": {
    "und": [
      {"aktiv": true, "aktion": "pass", "ist_wan": true},
      {"feld": "ziel_port", "port_ist": ["@param:ports"]}
    ]
  },
  "vorlage": "#{nr} [{interface}] {quelle_anzeige} -> {ziel_text} - '{descr}'",
  "ergebnis": [
    {
      "wenn": "treffer",
      "status": "WARN",
      "befunde": "je_treffer",
      "notiz": "Dienst aus der Portliste vom WAN erreichbar."
    },
    {"status": "PASS"}
  ]
}
```

Parameter (Feld „Parameter (JSON)“):

```
{"ports": ["80", "8080"]}
```

Ergebnis des Probelaufs (tatsächliche Ausgabe der Engine):

Test-Config	Status	Schwere	Treffer	Befund
opn-schwach.xml	WARN	mittel	1	#1 [wan] ANY → wanip:80 - 'GUI von aussen'
opn-gehaertet.xml	OK	mittel	0	–

19.8 Beispiel 8: Einen anderen Ablageort einer Einstellung nachtragen

Ziel: Zeigt der Testplan, dass deine OPNsense-Version die SNMP-Einstellungen im Plugin `os-net-snmp` ablegt (Bereich `OPNsense/netsnmp`) statt unter `snmpd`, meldet `SVC-SNMP-PUBLIC` „N/A“, obwohl SNMP läuft. Du kannst das im Editor selbst beheben: In der vorhandenen Regel `SVC-SNMP-PUBLIC` werden die Pfade ergänzt.

Hinweis Das Beispiel geht von einer **angenommenen** Struktur aus (`OPNsense/netsnmp/general/enabled` und `.../community`). Ermittle die echten Pfade deiner Version aus einem Export (Kapitel 18.3).

Gegenüber der mitgelieferten Prüfung sind **drei Stellen** geändert:

- 1 `werte → community → pfad` enthält jetzt eine **Liste** mit beiden Ablageorten; der erste nicht leere gilt.
- 2 Der erste Fall („nicht konfiguriert“ → N/A) prüft beide Bereiche: nur wenn **weder** `snmpd` **noch** `OPNsense/netsnmp` existiert, gilt SNMP als nicht konfiguriert.
- 3 Die Fälle FAIL und WARN fragen „aktiv?“ nun an beiden Stellen ab (`oder`).

Prüfung (in das Feld „Prüfung (JSON)“ einfügen – bei R8 ersetzt sie die vorhandene Prüfung der Regel):

```

{
  "werte": {
    "community": {"pfad": ["snmpd/rocommunity", "OPNsense/netsnmp/general/community"]}
  },
  "ergebnis": [
    {
      "wenn": {
        "nicht": {
          "oder": [{"existiert": "snmpd"}, {"existiert": "OPNsense/netsnmp"}]
        }
      },
      "status": "NA",
      "notiz": "SNMP nicht konfiguriert."
    },
    {
      "wenn": {
        "und": [
          {
            "feld": "community",
            "ist_eine_von": "@param:default_communities"
          },
          {
            "oder": [
              {"eingeschaltet": "snmpd/enable"},
              {"eingeschaltet": "OPNsense/netsnmp/general/enabled"}
            ]
          }
        ]
      },
      "status": "FAIL",
      "schweregrad": "high",
      "befund": "SNMP aktiv mit Standard-Community '{community}'."
    },
    {
      "wenn": {"feld": "community", "ist_eine_von": "@param:default_communities"},
      "status": "WARN",
      "befund": "SNMP-Standard-Community '{community}' konfiguriert."
    },
    {
      "wenn": {
        "oder": [
          {"eingeschaltet": "snmpd/enable"},
          {"eingeschaltet": "OPNsense/netsnmp/general/enabled"}
        ]
      },
      "status": "WARN",
      "befund": "SNMP aktiv. SNMPv3 pruefen."
    },
    {"status": "PASS", "notiz": "Keine Standard-Community."}
  ]
}

```

Parameter (Feld „Parameter (JSON)“):

```
{"default_communities": ["public", "private"]}
```

Ergebnis des Probelaufs (tatsächliche Ausgabe der Engine):

Test-Config	Status	Schwere	Treffer	Befund
opn-minimal.xml, ergänzt um einen Net-SNMP-Abschnitt	FAIL	hoch	–	SNMP aktiv mit Standard-Community 'public'.
opn-minimal.xml	N/A	hoch	–	SNMP nicht konfiguriert.

Test-Datei nachstellen Für die erste Zeile: opn-minimal.xml, ergänzt um einen Net-SNMP-Abschnitt. Ergänze vor dem schließenden folgende Zeile(n) und lade die Datei als Test-Config:

```
<OPNsense><netsnmp><general><enabled>1</enabled><community>public</community></general></netsnmp></OPNsense>
```

19.9 Beispiel 9: Verdeckte Block-Regeln nur auf dem WAN melden

Ziel: Die mitgelieferte Reihenfolge-Prüfung meldet dir auch interne Interfaces; du willst nur die verdeckten Block-Regeln auf dem WAN sehen. Neue Regel, Prüfung = Kopie von `FW-SHADOWED-BLOCK` mit einer zusätzlichen Angabe in `nachher` (`"ist_wan": true`):

Prüfung (in das Feld „Prüfung (JSON)“ einfügen):


```

{
  "typ": "reihenfolge",
  "quelle": "filterregeln",
  "gleich": ["interface", "richtung"],
  "umfasst": ["ip_version"],
  "vorher": {
    "aktiv": true,
    "floating": false,
    "zusatzbedingung": false,
    "protokoll": "any",
    "quelle_any": true,
    "quelle_negiert": false,
    "ziel_any": true,
    "ziel_negiert": false,
    "ziel_port": ""
  },
  "nachher": {
    "aktiv": true,
    "floating": false,
    "aktion": ["block", "reject"],
    "ist_wan": true
  },
  "vorlage": "#{nachher.nr} [{interface}/{richtung}] '{nachher.descr}' wird nie
erreicht - verdeckt durch #{vorher.nr} {vorher.aktion} '{vorher.descr}'",
  "ergebnis": [
    {
      "wenn": "treffer",
      "status": "WARN",
      "befunde": "je_treffer",
      "notiz": "Block-Regel wirkt nicht: eine vorangehende Regel faengt den gesamten
Verkehr ab."
    },
    {"status": "PASS"}
  ]
}

```

Ergebnis des Probelaufs (tatsächliche Ausgabe der Engine):

Test-Config	Status	Schwere	Treffer	Befund
opn-schwach.xml	WARN	mittel	1	#6 [wan/in] 'Block ohne Log' wird nie erreicht - verdeckt durch #0 pass 'any any WAN'
opn-reihenfolge.xml	OK	mittel	0	–

20 Als neue Version speichern und einsetzen

20.1 Der Speicher-Dialog

The dialog box is titled "Als neue Version speichern". It contains the following elements:

- 1. Version range: "von 1.2.0 auf" followed by an input field containing "1.3.0".
- 2. Release type: Three buttons labeled "Patch", "Minor" (which is selected), and "Major".
- 3. Changes list: A box containing "Neu: SVC-DOT-MISSING" and "Geändert: SVC-SNMP-PUBLIC".
- 4. Change note: A text area with the content "DoT-Regel ergänzt, SNMP abgestuft".
- 5. Filename: An input field containing "rules.js".
- 6. Action buttons: Three buttons at the bottom: "rules.js herunterladen", "Als rules.json (Python-Tool)", and "Abbrechen".

Below the dialog box, there is a note: "Zum Einsatz die Datei als rules.js neben index.html ablegen (alte Datei ersetzen)."

Abbildung 32: Dialog „Als neue Version speichern“.

- 1 **Versionsnummer.** „von ... auf ...“: links die Version der geladenen Datei, rechts der Vorschlag. Das Feld ist editierbar; erwartet wird das Format `X.Y.Z`.
- 2 **Patch / Minor / Major** setzt den Vorschlag neu. Patch erhöht die dritte Stelle (1.2.0 → 1.2.1), Minor die zweite (→ 1.3.0), Major die erste (→ 2.0.0). Berechnet wird immer von der geladenen Version aus.
- 3 **Änderungen** gegenüber dem geladenen Stand: *Neu*, *Geändert*, *Entfernt* (jeweils mit den IDs) und ggf. „Katalog: Einstellungen geändert“. Gibt es keine, steht dort ein entsprechender Hinweis; du kannst trotzdem speichern.
- 4 **Änderungsnotiz** (optional). Landet im Änderungsprotokoll des Katalogs.
- 5 **Dateiname** (Standard `rules.js`).
- 6 **rules.js herunterladen** speichert den Katalog als Katalogdatei für `index.html`.
- 7 **Als rules.json (Python-Tool)** speichert denselben Inhalt als reines JSON (Endung `.json`) für das Python-Audit-Programm. Es wertet das Feld `check` nicht aus; neue Regeln erscheinen dort als MANUAL.

Der **Versionsvorschlag** ist „Minor“, wenn Regeln hinzukamen oder entfielen, Katalog-Einstellungen, Schweregrad, Aktivstatus oder eine Prüfung geändert wurden, sonst „Patch“. Major schlägt der Editor nie vor; das bleibt deine Entscheidung (etwa bei einer geänderten Bedeutung des Scores).

20.2 Was in die Datei geschrieben wird

- Die neue Versionsnummer in `meta.version`.
- Ein neuer Eintrag im Änderungsprotokoll `meta.changeLog` mit Version, Datum (Tag der Speicherung), deiner Notiz sowie den Listen `added`, `changed` und `removed`. Frühere Einträge bleiben erhalten.
- Alle Regeln mit sämtlichen Feldern, auch solchen, die der Editor nicht kennt (sie werden unverändert übernommen).

Nach dem Speichern gilt der neue Stand als Vergleichsbasis: Die Anzeige springt auf „keine Änderungen“, die Version im Kopf ist die neue, alle Marken verschwinden. Weitere Änderungen werden gegen diesen Stand verglichen.

20.3 Die neue Datei einsetzen

1

Alten Katalog sichern Arbeitsrechner

Benenne die bisherige `rules.js` um, z. B. in `rules-1.2.0.js`, oder lege sie in einen Ordner `archiv/`. So kannst du jederzeit zurück.

2

Neue Datei ablegen

Die heruntergeladene Datei (meist im Download-Ordner) in den Werkzeugordner neben `index.html` kopieren und `rules.js` nennen. Hast du schon eine Datei gleichen Namens heruntergeladen, benennt der Browser sie evtl. in `rules (1).js` um; dann von Hand umbenennen.

3

Neu laden

`index.html` (und ggf. den Editor) mit `Strg + F5` neu laden. In der Kopfzeile müssen die neue Katalogversion und die Regelanzahl stehen.

4

Kontrollieren

Eine bekannte Config laden und prüfen, ob die geänderten Regeln wie erwartet reagieren (mit den Beispiel-Configs aus `tests/configs/` gelingt das auch ohne echte Firewall).

Der Selbsttest bleibt unberührt. `tests/selbsttest.html` nutzt eine eigene, festgeschriebene Kopie des Katalogs und prüft die *Engine*, nicht deinen Katalog (Kapitel 25). Eine Katalogänderung bricht ihn deshalb nicht.

21 Schritt für Schritt: Prüfen, ob die Block-All-Regel am Ende steht

Dieses Kapitel führt einmal vollständig durch den Regel-Editor: Du legst eine eigene Regel an, die meldet, wenn eine **Block-All-Regel** (eine Firewall-Regel, die allen Verkehr blockiert) **nicht die letzte Regel** ihres Interfaces ist. Alle Abbildungen und Ergebnisse stammen aus einem tatsächlichen Durchlauf mit den Beispiel-Configs.

21.1 Was die Regel prüfen soll

Firewall-Regeln werden je Interface und Richtung von oben nach unten ausgewertet; die erste passende gewinnt (Abbildung im Kapitel 6.4). Steht eine Block-All-Regel nicht ganz unten, werden alle Regeln darunter nie erreicht.

Begriff	Bedeutung in dieser Regel
Block-All-Regel	aktiv, Aktion <i>block</i> oder <i>reject</i> , Quelle <i>any</i> , Ziel <i>any</i> , kein Port, Protokoll <i>any</i> , ohne Zeitplan, Tag oder OS-Filter, nicht negiert, keine Floating-Regel
Gemeldet wird	jede aktive (nicht Floating-)Regel, die auf demselben Interface in derselben Richtung und mit passender IP-Version hinter einer Block-All-Regel steht, egal ob Freigabe oder Block
Ergebnis	WARN mit einem Befund je betroffener Regel, sonst OK

Abgrenzung zu FW-SHADOWED-PASS Die mitgelieferte Regel `FW-SHADOWED-PASS` meldet nur *Freigaben* hinter einer Auffang-Regel (egal ob Block oder Pass). Die neue Regel betrachtet nur Block-All-Regeln, meldet aber *jede* Regel dahinter und nennt die Block-All-Regel im Befund. Beide können parallel aktiv sein; wie du sie auf „nur Freigaben“ einschränkst, steht in Abschnitt 21.9.

21.2 Schritt 1 und 2: Editor öffnen und Regel beschreiben

- 1

Editor öffnen

Arbeitsrechner, Browser

`rules-editor.html` per Doppelklick öffnen. Kontrolle: Oben steht „34 Regeln, 0 deaktiviert“ und rechts die Version 1.2.0.
- 2

Neue Regel anlegen und beschreiben

Arbeitsrechner, Browser

`Neue Regel` klicken und die Felder wie in der Abbildung ausfüllen.

Neue Regel

Die Regel wird am Ende ihrer Kategorie eingefügt.

Ohne Prüfung erscheint die Regel im Bericht mit Status MANUELL und zählt nicht in den Score. Mit einer Prüfung (Bereich „Prüfung“ unten, am einfachsten über eine Vorlage) wird sie beim nächsten Audit sofort ausgewertet.

ID

Schweregrad

1 FW-BLOCKALL - NICHT - LETZTE

2 Mittel (Gewicht 3)

Großbuchstaben, Ziffern und Bindestrich, z. B. SVC-DOT-MISSING. Muss eindeutig sein.

Titel

3 Block-All-Regel steht nicht am Ende

Kategorie

4 Firewall-Regeln

Vorhandene Kategorie wählen oder eine neue eintippen.

5 ☒ Regel aktiv (deaktivierte Regeln werden nicht geprüft)

Begründung

6 Eine Block-All-Regel (alles blockieren) muss die letzte Regel eines Interfaces sein. Regeln dahinter werden nie ausgewertet.

Maßnahme

7 Block-All-Regel ans Ende des Interfaces verschieben und die Freigaben davor anordnen.

Abbildung 33: Ausgefülltes Formular der neuen Regel.

1 ID: FW-BLOCKALL - NICHT - LETZTE

2 **Schweregrad:** Mittel (Gewicht 3). Ein verdecktes Regelwerk ist ein echtes Risiko, aber nicht unmittelbar ausnutzbar.

3 **Titel:** Block-All-Regel steht nicht am Ende

4 **Kategorie:** Firewall-Regeln (aus der Vorschlagsliste)

5 **Regel aktiv** bleibt angehakt.

6 **Begründung:** z. B. „Eine Block-All-Regel (alles blockieren) muss die letzte Regel eines Interfaces sein. Regeln dahinter werden nie ausgewertet.“

7 **Maßnahme:** z. B. „Block-All-Regel ans Ende des Interfaces verschieben und die Freigaben davor anordnen.“

21.3 Schritt 3 und 4: Prüfung einsetzen und Aufbau prüfen

3 **Prüfung einsetzen** **Arbeitsrechner, Browser**

Im Bereich „Prüfung“ unter **Vorlage einfügen ...** die Vorlage *Reihenfolge von Firewall-Regeln prüfen* wählen. Sie entspricht im Aufbau schon fast dem Ziel. Danach den gesamten Inhalt des Feldes „Prüfung (JSON)“ durch den folgenden Text ersetzen:

```

{
  "typ": "reihenfolge",
  "quelle": "filterregeln",
  "gleich": ["interface", "richtung"],
  "umfasst": ["ip_version"],
  "vorher": {
    "aktiv": true,
    "floating": false,
    "zusatzbedingung": false,
    "aktion": ["block", "reject"],
    "protokoll": "any",
    "quelle_any": true,
    "quelle_negiert": false,
    "ziel_any": true,
    "ziel_negiert": false,
    "ziel_port": ""
  },
  "nachher": {"aktiv": true, "floating": false},
  "vorlage": "Block-All #{vorher.nr} [{interface}/{richtung}] steht nicht am Ende:
dahinter folgt #{nachher.nr} {nachher.aktion} '{nachher.descr}'",
  "ergebnis": [
    {
      "wenn": "treffer",
      "status": "WARN",
      "befunde": "je_treffer",
      "notiz": "Regeln hinter einer Block-All-Regel werden nie erreicht. Die Block-All-
Regel gehoert ans Ende."
    },
    {"status": "PASS"}
  ]
}

```

Teil	Was er bewirkt
"typ": "reihenfolge"	Sucht Paare „frühere Regel – spätere Regel“ statt einzelner Regeln (Kapitel 18.8).
"quelle": "filterregeln"	Arbeitet auf den Firewall-Regeln, für OPNsense und pfSense gleich.
"gleich"	Nur Regeln auf demselben Interface und in derselben Richtung werden verglichen, denn nur dort gilt die Auswertungsreihenfolge.
"umfasst"	Eine Block-All-Regel nur für IPv4 verdeckt keine IPv6-Regel; eine für IPv4+IPv6 verdeckt beide.
"vorher"	Definiert die Block-All-Regel. Die Liste ["block", "reject"] heißt „eine von beiden“. "ziel_port": "" heißt „kein Port“.
"nachher"	Jede aktive Regel, die keine Floating-Regel ist. Für jede wird die erste davor stehende Block-All-Regel gemeldet.

"vorlage"	Text je Befund. <code>{vorher.nr}</code> ist die Nummer der Block-All-Regel, <code>{nachher.nr}</code> und <code>{nachher.descr}</code> beschreiben die verdeckte Regel.
"ergebnis"	Bei Treffern WARN mit einem Befund je Treffer, sonst OK (<code>PASS</code>). Der Fall ohne <code>wenn</code> am Ende ist der Pflicht-Standardfall.

4

Aufbau prüfen Arbeitsrechner, Browser

Aufbau prüfen klicken. Erwartet: „Aufbau in Ordnung.“ Erscheint stattdessen eine rote Liste, fehlt meist ein Komma oder ein Anführungszeichen (Kapitel 22.2).

1 Prüfung (JSON)

```
{
  "typ": "reihenfolge",
  "quelle": "filterregeln",
  "gleich": ["interface", "richtung"],
  "umfasst": ["ip_version"],
  "vorher": {
    "aktiv": true,
    "floating": false,
    "zusatzbedingung": false,
    "aktion": ["block", "reject"],
    "protokoll": "any",
    "quelle_any": true,
    "quelle_negiert": false,
    "ziel_any": true,
    "ziel_negiert": false,
    "ziel_port": ""
  },
  "nachher": {"aktiv": true, "floating": false},
  "vorlage": "Block-All #{vorher.nr} [{interface}/{richtung}] steht nicht am Ende: dahinter folgt #
{nachher.nr} {nachher.aktion} '{nachher.descr}'",
  "ergebnis": [
    {
      "wenn": "treffer",
      "status": "WARN",
      "befunde": "je_treffer",
      "notiz": "Regeln hinter einer Block-All-Regel werden nie erreicht. Die Block-All-Regel
gehört ans Ende."
    },
    {"status": "PASS"}
  ]
}
```

Leer = keine automatische Prüfung (Status MANUELL). Aufbau: Sprachreferenz unten; als Ausgangspunkt eine Vorlage einfügen.

2



3

Aufbau prüfen
Probelauf
Test-Config laden ...

keine Test-Config geladen

4

Aufbau in Ordnung.

Abbildung 34: Prüffeld mit der eingesetzten Definition und erfolgreicher Aufbauprüfung.

- 1 **Prüfung (JSON)** mit dem eingesetzten Text.
- 2 **Vorlage einfügen ...** (Schritt 3).
- 3 **Aufbau prüfen** (Schritt 4).
- 4 **Ergebnis:** „Aufbau in Ordnung.“

Aufbau in Ordnung heißt nicht „funktioniert“. Ein Tippfehler in einem Feldnamen (z. B. `"zeil_any"`) wird nicht gemeldet; die Regel findet dann einfach nie etwas. Deshalb ist der nächste Schritt Pflicht.

21.4 Schritt 5: Probelauf mit zwei Test-Configs

- 5 **Gegenprobe in beide Richtungen** **Arbeitsrechner, Browser**
Test-Config laden ... klicken und `tests/configs/pf-reihenfolge.xml` wählen, dann **Probelauf**. Erwartet: **WARN mit 2 Treffern**. Danach dasselbe mit `opn-gehaertet.xml`. Erwartet: **OK mit 0 Treffern**. Erst wenn beide stimmen, findet die Regel, was sie soll, und schlägt bei einem sauberen Regelwerk nicht an.

Vorlage einfügen ... **Aufbau prüfen** **Probelauf**

1 **Test-Config laden ...** Test-Config: pf-reihenfolge.xml 2

WARN Mittel · pfSense · pf-reihenfolge.xml · 2 Treffer

- Block-All #0 [opt2/in] steht nicht am Ende: dahinter folgt #1 pass 'Allow HTTP TCP 80'
- Block-All #0 [opt2/in] steht nicht am Ende: dahinter folgt #2 pass 'Allow NTP UDP 123'

Notiz: Regeln hinter einer Block-All-Regel werden nie erreicht. Die Block-All-Regel gehoert ans Ende.

Abbildung 35: Probelauf gegen pf-reihenfolge.xml: Die Block-All-Regel #0 auf opt2 verdeckt zwei Freigaben.

- 1 **Test-Config laden ...**
- 2 **Geladene Datei.** Sie bleibt für weitere Probelaufe geladen.
- 3 **Probelauf**
- 4 **Kopfzeile:** Status WARN, Schweregrad Mittel, Plattform, Datei, 2 Treffer.
- 5 **Befunde.** Die Nummern sind die laufenden Nummern in der Config-Datei, beginnend bei 0, nicht die Anzeige in der Weboberfläche.
- 6 **Notiz** aus dem Ergebnis-Fall.

Test-Config laden ... Test-Config: opn-gehaertet.xml

OK Mittel · OPNsense · opn-gehaertet.xml · 0 Treffer

Abbildung 36: Gegenprobe mit opn-gehaertet.xml: keine Treffer.

Ergebnisse der Regel für alle zehn Beispiel-Configs (tatsächliche Ausgabe des Probelaufs):

Test-Config	Status	Treffer	Befunde
-------------	--------	---------	---------

opn-reihenfolge.xml	WARN	2	Block-All #4 [opt1/in] steht nicht am Ende: dahinter folgt #5 pass 'Negierte Quelle' Block-All #4 [opt1/in] steht nicht am Ende: dahinter folgt #6 block 'Gaeste nicht ins LAN'
pf-reihenfolge.xml	WARN	2	Block-All #0 [opt2/in] steht nicht am Ende: dahinter folgt #1 pass 'Allow HTTP TCP 80' Block-All #0 [opt2/in] steht nicht am Ende: dahinter folgt #2 pass 'Allow NTP UDP 123'
opn-gehaertet.xml	OK	0	–
opn-minimal.xml	OK	0	–
opn-schwach.xml	OK	0	–
opn-variante.xml	OK	0	–
pf-gehaertet.xml	OK	0	–
pf-idsaus.xml	OK	0	–
pf-minimal.xml	OK	0	–
pf-schwach.xml	OK	0	–

Bei `opn-reihenfolge.xml` zeigt sich der Unterschied zu `FW-SHADOWED-PASS` : Gemeldet wird neben der Freigabe „Negierte Quelle“ auch die Block-Regel „Gaeste nicht ins LAN“, die hinter der Block-All-Regel ebenfalls wirkungslos ist. Die deaktivierte Block-All-Regel auf `opt2` löst dagegen nichts aus.

21.5 Schritt 6: Regel anlegen

6

In den Katalog übernehmen

Arbeitsrechner, Browser

Regel anlegen

klicken. Meldung: „Regel FW-BLOCKALL-NICHT-LETZTE angelegt.“ Noch ist nichts gespeichert, nur die Arbeitskopie im Browser ist geändert.



Abbildung 37: Die neue Regel steht am Ende der Kategorie „Firewall-Regeln“.

- 1

Neue Regel in der Liste (am Ende ihrer Kategorie).
- 2

Marke „neu“.

Abbildung 38: Kopfbereich nach dem Anlegen.

1 „1 ungespeicherte Änderung“: erinnert daran, dass noch gespeichert werden muss.

21.6 Schritt 7 und 8: Als neue Version speichern und einsetzen

7

Neue Version speichern Arbeitsrechner, Browser

Als neue Version speichern ... klicken. Der Editor schlägt 1.3.0 vor (Minor, weil eine Regel hinzukam). Notiz eintragen und rules.js herunterladen klicken. Danach zeigt der Editor „keine Änderungen“ und Version 1.3.0.

Als neue Version speichern

von 1.2.0 auf 1.3.0 Patch Minor Major

Neu: FW-BLOCKALL-NICHT-LETZTE

Änderungsnotiz (optional, landet im Changelog des Katalogs)

FW-BLOCKALL-NICHT-LETZTE ergänzt: Block-All-Regel muss am Ende stehen

Dateiname

rules.js

Zum Einsatz die Datei als rules.js neben index.html ablegen (alte Datei ersetzen).

rules.js herunterladen Als rules.json (Python-Tool) Abbrechen

Abbildung 39: Speicher-Dialog mit der neuen Regel.

1 **Version:** von 1.2.0 auf 1.3.0.

2 **Minor** ist vorgewählt.

3 **Änderungen:** „Neu: FW-BLOCKALL-NICHT-LETZTE“.

4 **Notiz**, z. B. „FW-BLOCKALL-NICHT-LETZTE ergänzt: Block-All-Regel muss am Ende stehen“. Sie landet mit Datum im Änderungsprotokoll.

5 **Dateiname** rules.js .

6 rules.js herunterladen

8

Neue Datei einsetzen Arbeitsrechner, Dateimanager

Im Werkzeugordner die bisherige rules.js in rules-1.2.0.js umbenennen (Sicherung). Die heruntergeladene Datei aus dem Download-Ordner in den Werkzeugordner kopieren und, falls der Browser sie rules (1).js genannt hat, in rules.js umbenennen.

21.7 Schritt 9: Ergebnis im Bericht kontrollieren

9

Neu laden und prüfen Arbeitsrechner, Browser

index.html mit `Strg` + `F5` neu laden. Die Kopfzeile muss „v1.3.0 (35 Regeln)“ zeigen. Dann tests/configs/pf-reihenfolge.xml laden: Die neue Regel erscheint unter **Nur WARN**.



Abbildung 40: Kopfzeile von index.html mit dem neuen Katalog.

1

Katalog v1.3.0 (35 Regeln): Die neue Datei ist geladen.

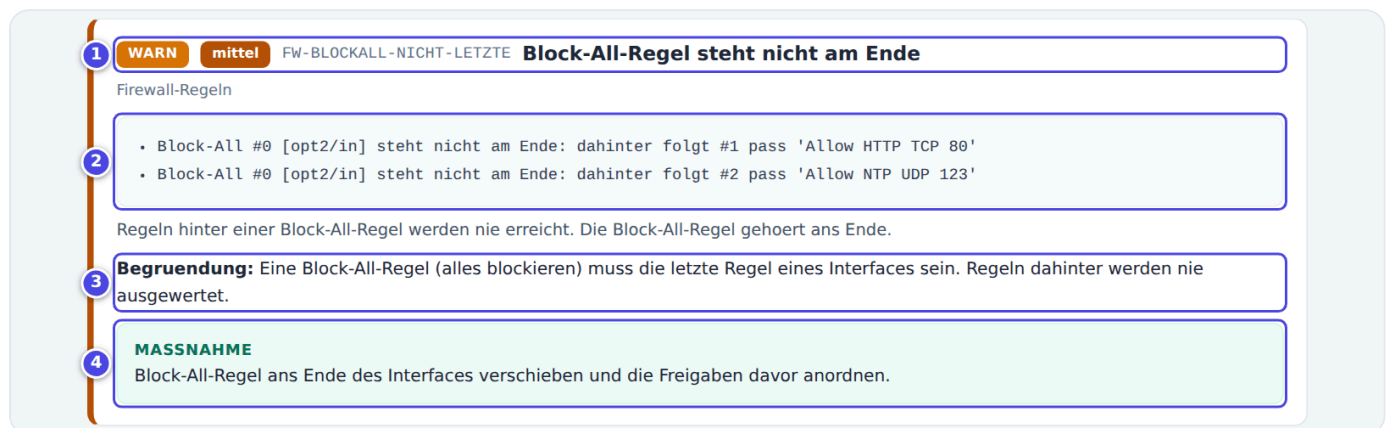


Abbildung 41: Die neue Regel im Bericht für pf-reihenfolge.xml.

1

Kartenkopf: WARN, mittel, ID und Titel.

2

Befunde aus der Vorlage, dazu die Notiz.

3

Begründung aus dem Formular.

4

Maßnahme aus dem Formular.

Der Score von pf-reihenfolge.xml sinkt durch die zusätzliche Regel von 95.0 auf 92.9: Das WARN kostet 1,5 Strafpunkte (halbes Gewicht von 3).

21.8 Schritt 10: Mit der eigenen Firewall prüfen und beheben

10

Konfiguration exportieren OPNsense- bzw. pfSense-Weboberfläche

Wie in Kapitel 3.2 beschrieben, ohne Verschlüsselung.

11

Auswerten Arbeitsrechner, Browser

Datei in index.html laden und die Karte FW-BLOCKALL-NICHT-LETZTE lesen. Notiere dir aus jedem Befund das Interface (in eckigen Klammern) und die Beschreibung der Block-All-Regel.

12

Reihenfolge korrigieren, OPNsense **OPNsense-Weboberfläche**

Firewall → Regeln → [Interface]: die Block-All-Regel ans Ende der Liste verschieben (je nach Version per Ziehen, Verschiebe-Schaltfläche oder Sequenznummer), speichern und die Änderungen anwenden.

12

Reihenfolge korrigieren, pfSense **pfSense-Weboberfläche**

Firewall → Rules → [Interface-Reiter]: die Zeile der Block-All-Regel per Ziehen ans Ende schieben, Save und anschließend Apply Changes klicken.

13

Erneut prüfen **Firewall, danach Arbeitsrechner**

Neu exportieren und laden. Erwartet: FW-BLOCKALL-NICHT-LETZTE mit OK.

Vor dem Verschieben prüfen Regeln, die bisher hinter der Block-All-Regel standen, waren wirkungslos. Rücken sie nach vorn, werden sie aktiv. Prüfe vor dem Verschieben, ob jede davon wirklich gewollt ist, besonders Freigaben.

21.9 Varianten und Grenzen

Variante	Änderung in der Prüfung	Wirkung (getestet)
Nur WAN-Interfaces	in "vorher" ergänzen: "ist_wan": true	Aufbau in Ordnung. Die Beispiel-Configs enthalten keine Block-All-Regel auf dem WAN mit Regeln dahinter, daher dort 0 Treffer. Mit einer eigenen Config gegenprüfen.
Nur verdeckte Freigaben	in "nachher" ergänzen: "aktion": "pass"	Aufbau in Ordnung. pf-reihenfolge.xml 2 Treffer, opn-reihenfolge.xml 1 Treffer (die Block-Regel dahinter entfällt).

Grenzen der Regel:

- Sie prüft nicht, ob überhaupt eine Block-All-Regel *existiert*. Beide Systeme blockieren am Ende ohnehin implizit; eine eigene Block-All-Regel dient vor allem der eigenen Protokollierung.
- Floating-Regeln, Regeln auf Interface-Gruppen und zur Laufzeit erzeugte Regeln werden nicht betrachtet.
- Eine Block-Regel, die nur fast alles erfasst (z. B. Quelle any, Ziel ein ganzes Netz), gilt nicht als Block-All. Teilüberdeckungen erkennt die Regelsprache nicht.
- Eine Block-All-Regel mit Zeitplan zählt bewusst nicht, weil sie nicht immer greift.

22 Fehlermeldungen und Fehlerbehebung (Editor)

22.1 Meldungen des Editors

Kurzmeldungen erscheinen unten rechts (dunkel bzw. bei Fehlern rot), Feldmeldungen direkt unter dem betroffenen Feld.

Meldung	Ursache und Abhilfe
Kein Katalog gefunden (erwartet: <code>window.RULES_DATA = {...};</code> oder reines JSON).	Die gewählte Datei ist keine Katalogdatei. Die richtige <code>rules.js</code> (oder <code>rules.json</code>) wählen.
Unerwartete Struktur: 'meta' und 'rules' fehlen.	Das JSON ist gültig, hat aber nicht den Aufbau eines Katalogs.
Fehlertext des Browsers zu JSON (z. B. „Unexpected token ...“) beim Öffnen	Die Katalogdatei ist beschädigt oder von Hand falsch bearbeitet (fehlendes Komma, überzähliges Komma, unvollständig). Vorherige Sicherung verwenden.
Ungültige ID.	Nur Großbuchstaben, Ziffern und Bindestrich, am Anfang Buchstabe oder Ziffer.
Diese ID gibt es bereits.	Andere ID wählen.
Titel fehlt. / Kategorie fehlt.	Pflichtfelder ausfüllen.
Kein gültiges JSON-Objekt: ...	Das Parameterfeld enthält kein gültiges JSON oder kein Objekt (<code>{ ... }</code>). Anführungszeichen und Kommas kontrollieren.
<code>sensitive_ports</code> : Liste ganzer Zahlen zwischen 1 und 65535 erwartet ..., ähnlich für <code>default_communities</code> und <code>wan_zone_severity</code>	Ungültiger Wert für einen fest verdrahteten Parameter (Kapitel 14.4).
Kein gültiges JSON: ... (Feld Prüfung)	Syntaxfehler im Prüffeld. Typisch: fehlende Anführungszeichen, ein Komma zu viel am Listenende, Klammern nicht geschlossen.
Rote Fehlerliste „Aufbaufehler:“	Strukturfehler in der Prüfung (Tabelle unten).
Bitte die markierten Felder prüfen.	Mindestens ein Feld ist ungültig; die Meldungen stehen an den roten Feldern.
Gewichte müssen Zahlen ≥ 0 sein.	Katalog-Einstellungen: ungültiges Gewicht.
Mindestens ein WAN-Interface angeben.	Katalog-Einstellungen: das Feld darf nicht leer sein.
Strafpunktfaktoren müssen zwischen 0 und 1 liegen.	Katalog-Einstellungen: Wert außerhalb 0 bis 1.
Version im Format X.Y.Z angeben.	Speicher-Dialog: die Versionsnummer besteht aus drei Zahlen mit Punkten, z. B. 1.3.0.
Keine OPNsense- oder pfSense-config.xml.	Probelauf: die geladene Test-Config hat ein anderes Wurzelement.
<code>engine.js</code> nicht gefunden.	Probelauf ohne <code>engine.js</code> ; Datei neben den Editor legen und neu laden.

22.2 Häufige Aufbaufehler

Meldung (Auszug)	Bedeutung und Abhilfe
------------------	-----------------------

Unbekannter Schluessel 'x' fuer Typ abfrage.	Schreibfehler oder ein Schlüssel, der zu einem anderen Typ gehört (Tabelle in Kapitel 18.1).
Abfrage: unbekannte Quelle 'x' (vorhanden: ...).	Quelle aus der Liste wählen.
'feld' braucht genau einen Operator (...).	Bei "feld" muss genau ein Operator im selben Objekt stehen, nicht keiner und nicht zwei.
Operator 'x' ohne 'feld'.	Der Operator gehört zu einem "feld" im selben Objekt.
ergebnis[n]: 'status' muss einer von FAIL, WARN, PASS, NA, MANUAL sein.	Status großgeschrieben und aus dieser Liste.
Der letzte Fall in 'ergebnis' sollte ohne 'wenn' sein (Standardfall).	Ergänze am Ende einen Fall ohne wenn .
Parameter 'x' ist in dieser Regel nicht definiert (unter 'params' anlegen).	Der Verweis @param:x braucht einen Eintrag im Parameterfeld.
'ergebnis' fehlt (Liste von Faellen).	Jede Prüfung (außer Baustein) braucht ergebnis .
Entweder 'abfragen' oder 'quelle'/'xml' verwenden, nicht beides.	Eine einzelne Abfrage oder eine Liste von Abfragen, nicht beides zugleich.
Wert 'x': genau eine Art angeben (pfad, anzahl oder verbinde).	Ein benannter Wert hat genau eine dieser Arten.
Unbekannter Baustein 'x' (vorhanden: ...).	Nur die vier fest programmierten Bausteine sind aufrufbar.

Zur Zuordnung zeigen die Meldungen die Stelle, z. B. `ergebnis[2]` → `wenn` (zweiter Fall, Feld `wenn`).

22.3 Wenn die Prüfung nicht tut, was sie soll

Beobachtung	Wahrscheinliche Ursache
Probelauf liefert immer OK mit 0 Treffern	Ein Feldname oder Wert in <code>wo</code> ist falsch geschrieben (wird nicht als Fehler gemeldet), oder das Feld hat einen anderen Typ (z. B. <code>ziel_port</code> ist ein Text, nicht ein Bereich). Mit einer Config prüfen, die den Befund <i>garantiert</i> enthält, und die Bedingung schrittweise verkleinern.
Probelauf liefert N/A trotz gesetzter Einstellung	Der XML-Pfad stimmt nicht mit dem Ablageort deiner Version überein (Kapitel 18.3). Pfad in der Config nachsehen.
Status FEHLER mit „Pruefung fehlgeschlagen: ...“	Die Prüfung wurde ausgeführt, ist aber auf einen Fehler gelaufen, z. B. „Kein Fall in 'ergebnis' trifft zu“ (Standardfall fehlt) oder ein nicht definierter Parameter.
Änderung aus dem Editor wirkt in <code>index.html</code> nicht	Die neue Datei wurde nicht als <code>rules.js</code> neben <code>index.html</code> abgelegt, oder der Browser zeigt die alte Fassung (mit <code>Strg</code> + <code>F5</code> neu laden). Die Katalogversion in der Kopfzeile von <code>index.html</code> verrät, welche Datei geladen ist.
Neue Regel erscheint im Bericht als MANUELL	Sie hat keine Prüfung (Marke „ohne Prüfung“ in der Liste) oder das Prüffeld wurde nicht übernommen.

23 Grenzen des Editors

- **Keine neuen Prüfungsarten:** Neue Faktenquellen (etwa Regeln aus *Firewall* → *Automation*), neue Operatoren, neue Bausteine oder Berechnungen erfordern eine Erweiterung von `engine.js`. Mit den vorhandenen Bausteinen lässt sich aber ein großer Teil der Praxisfälle abbilden (Kapitel 19).
- **Keine grafische Prüfungs-Oberfläche:** Prüfungen werden als JSON geschrieben. Vorlagen, Aufbauprüfung, Sprachreferenz und Probelauf sollen das erleichtern.
- **Kein Rückgängig:** Es gibt kein schrittweises Zurück, nur „Auf geladenen Stand zurücksetzen“ je Regel oder Neuladen der Datei.
- **Keine Sortierung:** Die Reihenfolge der Regeln im Katalog ist die Reihenfolge im Editor; sie lässt sich nicht verschieben. Neue Regeln kommen ans Ende ihrer Kategorie.
- **Kein Zusammenführen:** Zwei unabhängig bearbeitete Katalogdateien lassen sich nicht vereinen. Bearbeite immer die aktuellste Datei und speichere fortlaufend.
- **Das Python-Programm** wertet die Prüfdefinitionen (`check`) nicht aus. Ein exportiertes `rules.json` liefert dort Titel, Schweregrad und Texte, neue Regeln erscheinen als MANUAL.
- **Sicherheit:** Ein Katalog ist Programmcode. Öffne und verwende nur Dateien, deren Herkunft du kennst (Kapitel 1.3).

Teil C – Praxis

24 Typische Abläufe

24.1 Eine Firewall prüfen und die Ergebnisse abarbeiten

- 1 Exportieren** **OPNsense- bzw. pfSense-Weboberfläche**
Konfiguration ohne Verschlüsselung sichern (Kapitel 3.2). Datei mit Standort und Datum benennen, z. B. `berlin-2026-09-29.xml`.
- 2 Laden** **Arbeitsrechner, Browser**
`index.html` öffnen, Datei ablegen. Kopfzeile kontrollieren: stimmen Plattform und Hostname?
- 3 Kritisches zuerst**
Im Standardfilter „Offen“ von oben nach unten lesen: die Sortierung stellt schwere Befunde nach vorn. Beginne mit FAIL bei kritisch und hoch.
- 4 Befunde einordnen**
Zu jedem Befund gehört die Frage: *Ist das Absicht?* Eine gewollte Web-Weiterleitung mit Quelle ANY ist ein WARN, den du begründet stehen lässt; eine offene Verwaltungsoberfläche nicht. Nutze Begründung und Maßnahme der Karte.
- 5 Manuelle Punkte erledigen** **Firewall**
Filter „Manuell“: Firmware-Stand und Ähnliches direkt auf der Firewall prüfen.
- 6 Dokumentieren**
Bericht als PDF sichern (Kapitel 8.1), Ausnahmen mit Begründung festhalten.
- 7 Umsetzen und erneut prüfen** **Firewall, danach Arbeitsrechner**
Maßnahmen auf der Firewall umsetzen, erneut exportieren, erneut laden und den Score vergleichen. Die Ausgangsdatei danach sicher löschen.

24.2 Viele Standorte vergleichen

- 1** Für jeden Standort eine Datei mit eindeutigem Namen exportieren; Sorge dafür, dass die Konfigurationen einen Hostnamen enthalten (das Dashboard zeigt ihn als Standortnamen).
- 2** Alle Dateien im Auswahldialog markieren (`Strg` + `A` im Ordner) und gemeinsam laden.
- 3** Zuerst die **systemischen Befunde** lesen (Kapitel 7.2): Häufige Befunde haben oft eine gemeinsame Ursache.

- 4 Mit `nur kritische` und `Handlungsbedarf (<B)` die dringendsten Geräte eingrenzen und deren Einzelberichte öffnen.
- 5 Für den Verlauf über Zeit gibt es keine Speicherung. Sichere das Dashboard als PDF und vergleiche Ø-Score und Notenverteilung von Auswertung zu Auswertung.

Grenze der Vergleichbarkeit Scores sind nur vergleichbar, wenn dieselbe Katalogversion mit denselben Einstellungen verwendet wurde. Halte die Katalogversion (steht im Bericht) bei jeder Sicherung fest.

24.3 Den Katalog anpassen und einführen

- 1 **Ziel festlegen.** Was soll sich ändern (Schweregrad, neue Regel, Einstellung)? Notiere es in einem Satz; er wird später zur Änderungsnotiz.
- 2 **Im Editor ändern** (Kapitel 14 bis 17).
- 3 **Mit Test-Configs prüfen.** Bei einer neuen oder geänderten Prüfung im Probelauf zwei Fälle testen: `opn-schwach.xml` bzw. `pf-schwach.xml` (soll auslösen) und `opn-gehaertet.xml` bzw. `pf-gehaertet.xml` (soll sauber bleiben). Bei plattformabhängigen Prüfungen beide Plattformen.
- 4 **Speichern** mit Notiz (Kapitel 20). Die Versionsnummer folgt deinem Schema.
- 5 **Einsetzen** und die alte Datei sichern. Kopfzeile von `index.html` kontrollieren.
- 6 **Mit echten Configs gegenprüfen.** Eine eigene Prüfung ist erst verlässlich, wenn sie an *echten* Exporten deiner Firewall-Versionen bestätigt wurde. Der Testplan (`TESTPLAN.md`) beschreibt, wie du Einstellungen gezielt setzt, exportierst und kontrollierst.

24.4 Den Katalog im Team verwalten

- **Eine Hauptdatei.** Bestimme, wer den Katalog pflegt. Der Editor kann keine Änderungen zusammenführen; parallele Bearbeitungen überschreiben sich (Kapitel 23).
- **Alle Versionen aufheben.** Lege jede gespeicherte Datei mit Versionsnummer im Namen ab (`rules-1.3.0.js`) und setze die jeweils aktuelle als `rules.js` ein.
- **Das Änderungsprotokoll pflegen.** Die Notiz im Speicher-Dialog wird später zum Nachweis, *warum* etwas geändert wurde. Formuliere sie so, dass sie ohne Vorwissen verständlich ist („SYS-SYSLOG-REMOTE auf mittel: neue Vorgabe der Betriebsleitung“).
- **Herkunft prüfen.** Verwende nur Katalogdateien, deren Absender du kennst; sie werden als Programmcode ausgeführt.

25 Der Selbsttest der Engine

Im Ordner `tests/` liegt die Seite `selbsttest.html`. Sie prüft die **Engine** (`engine.js`): Sie wertet die zehn Beispiel-Configs aus und vergleicht jedes einzelne Ergebnis (Status, Schweregrad, Befunde, Notiz) und den

Score mit einer gespeicherten Referenz. Dafür nutzt sie eine eigene, festgeschriebene Kopie des Katalogs (`tests/katalog-referenz.js`), damit Änderungen an `rules.js` das Ergebnis nicht beeinflussen.

Selbsttest der Prüf-Engine

Wertet die mitgelieferten Test-Configs mit `engine.js` aus und vergleicht jedes Ergebnis (Status, Schweregrad, Befunde, Notiz, Score) mit der gespeicherten Referenz. Nach jeder Änderung an `engine.js` erneut öffnen.

1

Alle 340 Ergebnisse stimmen mit der Referenz überein
10 Test-Configs · Katalog 1.2.0

2

Test-Config	Plattform	Score erwartet / ist	Regeln	Abweichungen	
3	opn-gehaertet	opnsense	100 / 100	34	keine
	opn-minimal	opnsense	81.7 / 81.7	34	keine
	opn-reihenfolge	opnsense	68.3 / 68.3	34	keine
	opn-schwach	opnsense	10.5 / 10.5	34	keine
	opn-variante	opnsense	50.6 / 50.6	34	keine
	pf-gehaertet	pfsense	100 / 100	34	keine
	pf-idsaus	pfsense	93.3 / 93.3	34	keine
	pf-minimal	pfsense	98.2 / 98.2	34	keine
	pf-reihenfolge	pfsense	95 / 95	34	keine
	pf-schwach	pfsense	43.2 / 43.2	34	keine

Prüflogik bewusst geändert? Dann die aktuellen Ergebnisse als neue Referenz übernehmen: herunterladen und die Datei `tests/referenz.js` damit ersetzen.

Abbildung 42: Selbsttest: alle Ergebnisse stimmen mit der Referenz überein.

- 1
- Gesamturteil. Grün: „Alle 340 Ergebnisse stimmen mit der Referenz überein“ (10 Configs × 34 Regeln). Rot: „N von 340 Ergebnissen weichen ab“.
- 2
- Spalte „Abweichungen“. „keine“ in Grün; sonst in Rot je Abweichung die Regel mit erwartetem und tatsächlichem Status.
- 3
- Zeile je Test-Config mit Plattform, Score (erwartet / ist) und Zahl der Regeln.
- 4
- Referenz erneuern. Speichert die aktuellen Ergebnisse als neue `referenz.js`.

25.1 Wann ausführen?

Situation	Selbsttest?
Du hast <code>engine.js</code> geändert oder ein Update des Werkzeugs eingespielt	Ja. Erwartet: alles grün.
Du hast <code>rules.js</code> mit dem Editor geändert	Nicht nötig; er prüft den Katalog nicht. Nutze den Probelauf (Kapitel 17.3).
Du hast bewusst die Logik der Engine geändert	Ja. Abweichungen sind dann erwartet; kontrolliere sie und erneuere die Referenz.

Referenz erneuern: `Aktuelle Ergebnisse als referenz.js speichern` klicken (bei Abweichungen fragt die Seite nach: „N Ergebnisse weichen ab. Nur fortfahren, wenn diese Änderung gewollt ist. Neue Referenz speichern?“), die heruntergeladene Datei nach `tests/referenz.js` kopieren und die Seite neu laden. Eine Abweichung bedeutet **nicht zwingend einen Fehler**: Sie zeigt nur, dass sich ein Ergebnis gegenüber der Referenz verändert hat.

Die Seite meldet „Test nicht startbar“, wenn `engine.js` oder die Testdaten fehlen (der Ordner `tests/` muss neben `engine.js` liegen).

Anhang

ANHANG A Regelübersicht des Katalogs 1.2.0

Alle Regeln des mitgelieferten Katalogs, wie sie in der Datei `rules.js` stehen. „Prüfart“ nennt, wie die Regel geprüft wird: *Abfrage* (mit Quelle), *Reihenfolge* oder *Baustein*. „Parameter“ nennt die Einstellwerte, die im Formular veränderbar sind.

ID	Titel	Schwere	Prüfart	Parameter
MANAGEMENT-EBENE				
MGMT-WEBGUI-WAN	WebGUI vom WAN-Interface erreichbar	kritisch	Baustein	–
MGMT-WEBGUI-HTTP	WebGUI ohne HTTPS (Klartext)	hoch	Abfrage (Werte)	–
MGMT-SSH-WAN	SSH-Dienst auf WAN-Interface gebunden oder per WAN-Regel erreichbar	hoch	Baustein	–
MGMT-SSH-ROOT	SSH Root-Login erlaubt	mittel	Abfrage (Werte)	–
MGMT-SSH-PWAUTH	SSH Passwort-Authentifizierung aktiv	mittel	Abfrage (Werte)	–
MGMT-MFA	Keine MFA/OTP fuer privilegierte Benutzer	mittel	Abfrage (benutzer)	–
MGMT-ROOT-ACCOUNT	root-Konto aktiv als primaerer Admin-Zugang	niedrig	Baustein	–
MGMT-CONSOLE	Konsolenmenue nicht geschuetzt	niedrig	Abfrage (Werte)	–
FIREWALL-REGELN				
FW-ANY-ANY	Pass-Regel mit Quelle ANY und Ziel ANY	mittel	Abfrage (filterregeln)	wan_zone_severity
FW-WAN-EXPOSURE	WAN-Pass-Regel exponiert internen Dienst oder die Firewall selbst	hoch	Abfrage (filterregeln)	–
FW-BLOCKPRIV	'Block private networks' auf WAN deaktiviert	mittel	Abfrage (wan_interfaces)	–
FW-BLOCKBOGON	'Block bogon networks' auf WAN deaktiviert	niedrig	Abfrage (wan_interfaces)	–
FW-HOST-CIDR	Regel mit unplausibler Netzmaske (Einzelhost mit Subnetz-Maske)	niedrig	Baustein	–
FW-DISABLED-RULES	Deaktivierte Firewall-Regeln vorhanden (Hygiene)	info	Abfrage (filterregeln)	–
FW-BLOCK-NOLOG	Block/Reject-Regeln ohne Logging	niedrig	Abfrage (filterregeln)	–

FW-SHADOWED-BLOCK	Block-Regel wird durch vorangehende Auffang-Regel nie erreicht	mittel	Reihenfolge	–
FW-SHADOWED-PASS	Freigabe-Regel wird durch vorangehende Auffang-Regel nie erreicht	niedrig	Reihenfolge	–
NAT / PORT-FORWARDING				
NAT-MGMT-EXPOSED	Port-Weiterleitung exponiert sensiblen/Management-Port vom WAN	kritisch	Abfrage (nat)	sensitive_ports, sensitive_port_aliases_hint
NAT-WAN-ANY-SRC	Port-Weiterleitung vom WAN mit Quelle ANY	mittel	Abfrage (nat)	–
NAT-UPNP	UPnP / NAT-PMP aktiviert	hoch	Abfrage (Werte)	–
NAT-1TO1	1:1-NAT konfiguriert (Review)	info	Abfrage (XML)	–
DIENST-EXPOSITION				
SVC-SNMP-PUBLIC	SNMP mit Standard-Community 'public'	hoch	Abfrage (Werte)	default_communities
SVC-DNS-OPENRESOLVER	Unbound: ACL-Default 'allow' (potenzieller Open Resolver)	mittel	Abfrage (Werte)	–
SVC-NTP-OPEN	NTP ohne Zugriffsbeschränkung	niedrig	Abfrage (filterregeln)	–
SVC-IDS-DISABLED	IDS/IPS (Suricata) deaktiviert	mittel	Abfrage (Werte)	–
SVC-IDS-NOIPS	IDS aktiv, aber IPS-Blockmodus aus (nur Erkennung)	niedrig	Abfrage (Werte)	–
VPN				
VPN-IKEV1	IPsec IKEv1 / Aggressive Mode aktiv	hoch	Abfrage (XML)	–
VPN-0VPN-CIPHER	OpenVPN ohne AEAD-Cipher / schwache Verschlüsselung	mittel	Abfrage (XML)	–
VPN-FULL-ACCESS	VPN-Peer-Regel mit uneingeschränktem Zugriff (ANY)	mittel	Abfrage (filterregeln)	–
VPN-WG-PRESENT	WireGuard aktiv (Allowed-IPs prüfen)	info	Abfrage (Werte)	–
SYSTEM-HAERTUNG				
SYS-FIRMWARE	Firmware-Aktualität prüfen (Live-Check nötig)	info	Abfrage (Werte)	–
SYS-SYSLOG-REMOTE	Kein Remote-Syslog / Log-Forwarding konfiguriert	niedrig	Abfrage (Werte)	–
SYS-API-KEYS	API-Schlüssel auf Benutzerkonten vorhanden	info	Abfrage (benutzer)	–
SYS-BACKUP-SECRETS	config.xml enthält Secrets (Hinweis)	info	Abfrage (Werte)	–

A.1 Regeln mit eigenem Schweregrad im Fund

Bei diesen Regeln übersteuert die Prüfung den Katalog-Schweregrad in bestimmten Fällen (Kapitel 14.3). Der Katalogwert gilt nur für die übrigen Fälle.

Regel	Katalogwert	Fall (wenn ...)	Schweregrad im Fund
FW-ANY-ANY	mittel	{"treffer_mit": {"ist_wan": true}}	@param:wan_zone _severity (Standard: hoch)
FW-WAN-EXPOSURE	hoch	es gibt Treffer	hoch
NAT-MGMT-EXPOSED	kritisch	{"treffer_mit": {"quelle_any": true}}	kritisch
NAT-MGMT-EXPOSED	kritisch	es gibt Treffer	hoch
SVC-SNMP-PUBLIC	hoch	{"und": [{"feld": "community", "ist_eine_von": "@param:default_communities"}, {"eingesc ...	hoch

ANHANG B Sprachreferenz und Spickzettel

Die folgenden Tabellen werden aus `engine.js` erzeugt und entsprechen der Sprachreferenz im Editor.

B.1 Faktenquellen und ihre Felder

Quelle `filterregeln`

Firewall-Regeln unter Firewall → Rules, in der Reihenfolge der config.xml (Floating-Regeln eingeschlossen).

Feld	Bedeutung
nr	laufende Nummer ueber alle Regeln (wie im Bericht #n)
pos	Position innerhalb des Interfaces (1, 2, ...)
interface	Interface-Name (wan, lan, opt1 ...)
aktion	pass, block oder reject
richtung	in oder out (leer = in)
protokoll	tcp, udp, any ...
ip_version	inet, inet6 oder inet46
aktiv	true, wenn nicht deaktiviert
log	Logging an
quick	quick gesetzt
floating	Floating-Regel
zusatzbedingung	Zeitplan, tagged oder OS-Filter gesetzt
descr	Beschreibung

quelle_any	Quelle any
quelle_negiert	Quelle mit 'not'
ziel_any	Ziel any
ziel_negiert	Ziel mit 'not'
quelle_anzeige	Quelle als Text, any als ANY
quelle_text	Quelle als Text
ziel_text	Ziel als Text
ziel_port	Zielport bzw. Port-Alias (leer = alle)
ist_wan	Interface zaehlt als WAN (Katalog-Einstellung)
ist_vpn	Interface ist ein VPN-Interface

Quelle nat

Port-Weiterleitungen unter Firewall → NAT → Port Forward.

Feld	Bedeutung
nr	laufende Nummer
interface	Interface
quelle_any	Quelle any
quelle_anzeige	Quelle als Text, any als ANY
quelle_text	Quelle als Text
ports	Zielport(s) bzw. Port-Alias
ziel	interne Ziel-IP
ziel_port_intern	interner Port
descr	Beschreibung
ist_wan	auf WAN

Quelle benutzer

Lokale Benutzerkonten.

Feld	Bedeutung
name	Anmeldename
uid	Benutzer-ID
aktiv	nicht deaktiviert
otp	OTP-Seed hinterlegt
admin	Mitglied einer Gruppe mit page-all oder uid 0
gruppen_admin	Mitglied einer Gruppe mit page-all

`api_schluesssel`

Anzahl API-Schluesssel

Quelle `interfaces`

Alle Interfaces aus `<interfaces>`.

Feld	Bedeutung
<code>name</code>	interner Name (wan, lan, opt1 ...)
<code>descr</code>	Beschreibung
<code>geraet</code>	Geraetenname (igb0 ...)
<code>ist_wan</code>	zaehlt als WAN
<code>ist_vpn</code>	VPN-Interface
<code>block_private</code>	'Block private networks' an
<code>block_bogons</code>	'Block bogon networks' an

Quelle `wan_interfaces`

Die WAN-Interfaces in der Reihenfolge der Katalog-Einstellung (nur vorhandene). Felder wie 'interfaces'.

Felder wie bei `interfaces`.

Quelle `ids_instanzen`

pfSense: Interface-Instanzen des IDS-Pakets (Suricata/Snort).

Feld	Bedeutung
<code>interface</code>	Interface
<code>aktiv</code>	Instanz aktiviert
<code>blockiert</code>	Block Offenders bzw. Inline-Modus

B.2 Allgemeine Felder

Feld	Bedeutung
<code>plattform</code>	opnsense oder pfsense
<code>ssh_aktiv</code>	SSH-Dienst aktiviert
<code>ids_paket</code>	pfSense: 'Suricata' oder 'Snort', sonst leer
<code>wg_vorhanden</code>	WireGuard-Konfiguration vorhanden
<code>wg_tunnel</code>	Anzahl WireGuard-Tunnel/Instanzen
<code>wg_peers</code>	Anzahl WireGuard-Peers
<code>wg_gesamt</code>	Tunnel + Peers

B.3 Operatoren

Operator	Bedeutung
<code>ist</code>	gleich (Text ohne Gross/klein, true/false, Zahl)
<code>ist_eine_von</code>	gleich einem Wert der Liste
<code>leer</code>	true: Feld ist leer; false: Feld hat Inhalt
<code>enthaelt_eins_von</code>	Text enthaelt einen der Teiltexte
<code>liste_enthaelt</code>	durch Komma/Leerzeichen getrennte Liste enthaelt einen der Werte
<code>groesser</code>	Zahl groesser
<code>kleiner</code>	Zahl kleiner
<code>port_ist</code>	Port bzw. aufgeloester Port-Alias enthaelt einen der Ports
<code>ports_schneiden</code>	Ports (mit Bereichen und Aliassen) ueberschneiden sich mit der Liste; Treffer als {treffer_ports}

B.4 Weitere Bedingungen

Schreibweise	Bedeutung
<code>und</code>	alle Bedingungen der Liste
<code>oder</code>	mindestens eine Bedingung der Liste
<code>nicht</code>	Bedingung trifft nicht zu
<code>feld + Operator</code>	{ "feld": "ports", "ports_schneiden": [22,3389] }
<code>eingeschaltet</code>	Schalter unter dem XML-Pfad ist an (1, yes, on, enabled oder leeres Element)
<code>existiert</code>	XML-Pfad ist vorhanden
<code>eine</code>	mindestens ein Eintrag der Abfrage {quelle xml, wo}
<code>keine</code>	kein Eintrag der Abfrage {quelle xml, wo}
<code>treffer_mit</code>	mindestens ein Treffer erfuehlt die Bedingung
<code>"treffer" / "keine_treffer" / "quellen_leer"</code>	als Wert von 'wenn'

B.5 Spickzettel

Grundgerüst „abfrage“

```
{
  "werte": { "name": { "pfad": "a/b/c", "standard": "x" } },
  "quelle": "filterregeln",
  "wo": { "aktiv": true, "aktion": "pass" },
  "vorlage": "#{nr} [{interface}] {descr}",
  "ergebnis": [
    { "wenn": "treffer", "status": "WARN", "befunde": "je_treffer" },
    { "status": "PASS" }
  ]
}
```

Grundgerüst „reihenfolge“

```
{
  "typ": "reihenfolge", "quelle": "filterregeln",
  "gleich": ["interface", "richtung"], "umfasst": ["ip_version"],
  "vorher": { ... }, "nachher": { ... },
  "vorlage": "#{nachher.nr} hinter #{vorher.nr}",
  "ergebnis": [ ... ]
}
```

Bedingungen

```
{"feld": "aktion", "ist": "pass"} oder {"aktion": "pass"}
{"aktion": ["block", "reject"]} einer von
{"und": [ ... ]} {"oder": [ ... ]} {"nicht": { ... }}
{"feld": "ziel_port", "port_ist": ["22"]}
{"eingeschaltet": "system/ssh/enabled"}
{"existiert": "snmpd"}
{"keine": {"quelle": "ids_instanzen", "wo": { ... }}}
"treffer" "keine_treffer" "quellen_leer"
```

Texte und Werte

```
{feld} {feld|Ersatz} {feld:klein} {?feld=Text} {liste}
"@param:name" Parameter der Regel
{"opnsense": "a/b", "pfsense": "c/d"} Pfad je Plattform
Status: FAIL WARN PASS NA MANUAL
Schweregrad: critical high medium low info
```

C.1 Dateien im Paket

Datei / Ordner	Inhalt
<code>index.html</code>	Auswertung (Kapitel 3 bis 10).
<code>engine.js</code>	Logik: Konfiguration lesen, Prüfungen ausführen, Score berechnen.
<code>rules.js</code>	Regelkatalog (Daten und Prüfdefinitionen).
<code>rules-editor.html</code>	Regel-Editor (Kapitel 11 bis 23).
<code>HANDBUCH.html</code> , <code>HANDBUCH.pdf</code>	Dieses Handbuch (HTML mit eingebetteten Bildern bzw. als PDF).
<code>README.md</code>	Kurzbeschreibung des Werkzeugs.
<code>LICENSE</code> , <code>NOTICE</code>	Lizenztext der GNU GPL Version 3 und der Urheberhinweis mit der Zusatzbedingung nach Abschnitt 7(b): Der Hinweis „Firewall-Validation Portable - Copyright (C) 2026 Christian Jelitte“ bleibt in den Dateien und im Lizenzhinweis am unteren Rand von <code>index.html</code> und <code>rules-editor.html</code> erhalten. Der Regel-Editor schreibt ihn beim Speichern einer neuen <code>rules.js</code> automatisch mit.
<code>site/</code> , <code>.github/workflows/pages.yml</code>	Projektseite für GitHub Pages mit Demo (siehe <code>README.md</code>). Für den Betrieb des Werkzeugs nicht nötig.
<code>TESTPLAN.md</code>	Testplan mit 107 Testfällen für die Prüfung mit echten Firewalls.
<code>tests/selbsttest.html</code>	Selbsttest der Engine (Kapitel 25).
<code>tests/configs/*.xml</code>	Zehn Beispiel-Konfigurationen (siehe unten).
<code>tests/fixtures.js</code> , <code>referenz.js</code> , <code>katalog-referenz.js</code>	Testdaten des Selbsttests. Nicht von Hand bearbeiten.

C.2 Die Beispiel-Configs

Die Beispiel-Konfigurationen sind **synthetisch**: Sie sind von Hand so geschrieben, dass sie gezielt bestimmte Prüfungen auslösen. Sie enthalten keine echten Daten. Die Scores stammen von der Engine mit dem Katalog 1.2.0.

Datei	Plattform	Zweck	Score	Note	FAIL	WARN
<code>opn-schwach.xml</code>	OPNsense	OPNsense mit möglichst vielen Fehlkonfigurationen (WebGUI per HTTP, SSH am WAN, any→any auf WAN, RDP-Weiterleitung, SNMP „public“, UPnP, IKEv1 ...).	10.5	F	16	15
<code>pf-schwach.xml</code>	pfSense	pfSense mit vielen Fehlkonfigurationen (HTTP, SNMP „public“, UPnP, Suricata, WireGuard, schwacher OpenVPN-Cipher ...).	43.2	F	7	12

opn- variante.xml	OPNsense	OPNsense in einer Zwischenstufe (WebGUI auch am WAN gebunden, SSH-Weiterleitung für einen Partner, IDS ohne IPS).	50.6	F	6	6
opn- reihenfolge.xml	OPNsense	OPNsense mit Fällen zur Regel-Reihenfolge (verdeckte Block-Regeln, verschiedene Richtungen, IPv4/IPv6, Zeitplan, deaktivierte Regel).	68.3	D	3	5
opn-minimal.xml	OPNsense	Fast leere OPNsense-Konfiguration (nur Hostname und Domäne).	81.7	B	1	2
pf-idsaus.xml	pfSense	pfSense mit installiertem, aber deaktiviertem IDS-Paket (Snort).	93.3	A	1	2
pf- reihenfolge.xml	pfSense	pfSense-Fall: Auf dem Interface opt2 („NutzLAN“) steht eine „Block All“-Regel vor zwei Freigaben (HTTP, NTP).	95	A	0	4
pf-minimal.xml	pfSense	Fast leere pfSense-Konfiguration.	98.2	A	0	2
opn- gehaerttet.xml	OPNsense	OPNsense, sauber gehärtet. Erwartung: keine Befunde.	100	A	0	0
pf- gehaerttet.xml	pfSense	pfSense, sauber gehärtet. Erwartung: keine Befunde.	100	A	0	0

ANHANG D

Glossar

Audit
Die Prüfung einer Konfiguration gegen den Regelkatalog.
Auffang-Regel
Eine Firewall-Regel, die jeden Verkehr erfasst (Quelle any, Ziel any ohne Port, Protokoll any). Alles, was auf demselben Interface in derselben Richtung danach steht, wird nie erreicht.
Baustein
Eine fest programmierte Prüfung in <code>engine.js</code> , die eine Regel des Katalogs aufrufen kann.
Befund
Eine konkrete Fundstelle in einer Konfiguration, z. B. „Regel #3 erlaubt Port 22 vom WAN“.
config.xml
Die Konfigurationsdatei einer OPNsense- oder pfSense-Firewall (Export aus der Weboberfläche).
Engine
Die Prüflogik (<code>engine.js</code>). Liest die Konfiguration, führt Prüfungen aus und berechnet den Score.
Faktenquelle

Eine für beide Plattformen vereinheitlichte Sicht auf Teile der Konfiguration, z. B. <code>filterregeln</code> .
Fleet
Die Gesamtheit mehrerer Firewalls. Der „Fleet-Modus“ zeigt das Dashboard.
Floating-Regel
Firewall-Regel, die nicht an ein einzelnes Interface gebunden ist. Sie wird von den Reihenfolge-Prüfungen nicht einbezogen.
Firewall-Regel
Eine Regel der Firewall (Pass/Block ...) im Gegensatz zur <i>Regel des Katalogs</i> .
Gewicht
Zahl je Schweregrad (10/6/3/1/0), die bestimmt, wie stark eine Regel im Score zählt.
Katalog
Die Sammlung aller Prüfregeln samt Einstellungen (<code>rules.js</code>).
Note
Buchstabe A bis F, abgeleitet aus dem Score.
Parameter
Einstellwert einer Regel des Katalogs, z. B. eine Portliste.
Penalty / Strafpunkte
Gewicht × Faktor je Regel (FAIL 1, WARN 0,5, OK 0). Ihre Summe geht in den Score ein.
Plattform
OPNsense oder pfSense; wird an der Config erkannt.
Posture-Score
Kennzahl von 0 bis 100: Anteil der bewertbaren Regelgewichte, die nicht als Strafpunkte anfallen.
Probelauf
Ausführung einer (ungespeicherten) Prüfung im Editor gegen eine geladene Test-Config.
Regel (des Katalogs)
Eine Prüfvorschrift mit ID, Schweregrad, Texten und Prüfdefinition. Zu unterscheiden von der Firewall-Regel.

Schweregrad
kritisch, hoch, mittel, niedrig oder info.
Status
Ergebnis einer Regel: FAIL, WARN, OK, N/A, MANUELL oder FEHLER.
WAN
Die Internetseite der Firewall. Welche Interfaces dazu zählen, legen die Katalog-Einstellungen fest.
Wirksamer Schweregrad
Der im Fund tatsächlich angewandte Schweregrad, der vom Katalogwert abweichen kann.

ANHANG E Versionsverlauf des Katalogs

Das Änderungsprotokoll aus `meta.changelog` der mitgelieferten `rules.js`:

Version	Datum	Änderung
1.2.0	2026-09-29	Pruefungen als Daten: 28 Regeln mit Pruefdefinition im Feld 'check', 4 als Baustein. Neu: FW-SHADOWED-BLOCK, FW-SHADOWED-PASS (Regel-Reihenfolge). Korrektur SVC-DNS-OPENRESOLVER: Interface-Liste mit mehreren Kommas wird vollstaendig ausgewertet. Neu: <code>FW-SHADOWED-BLOCK</code> , <code>FW-SHADOWED-PASS</code>
1.1.0	2026-09-29	Parameter werden von engine.js ausgewertet: meta.wan_interfaces (neu), meta.scoring (Strafpunktfaktoren), params von NAT-MGMT-EXPOSED, SVC-SNMP-PUBLIC und FW-ANY-ANY. Werte unveraendert, Bewertung identisch zu 1.0.0.